

Masterarbeit

Auf dem Weg zum autonom fahrenden Fahrrad: Eine videogestützte Regelung des Weges mit Webbots

Amine Karabila

Sommersemester 2025

1 Abstract

Diese Masterarbeit beschäftigt sich mit der Entwicklung einer kombinierten Regelungsstrategie für ein autonom fahrendes, selbstbalancierendes Fahrrad, basierend auf den Ergebnissen dreier vorangegangener Forschungsarbeiten. Ziel dieser Arbeit ist die erfolgreiche Integration einer PID-Balanceregung mit einer MPC-basierten Fahrtrichtungsregelung, die mithilfe eines YOLO-Modells zur visuellen Spurdetektion ergänzt wird. Zur Validierung und Optimierung der kombinierten Regelung wurde die Simulationsumgebung Webots verwendet, wodurch die Robustheit und Funktionalität der entwickelten Regelungsalgorithmen realitätsnah überprüft werden konnten. Im Rahmen der Arbeit wurde eine zweistufige Regelungsarchitektur umgesetzt, welche die präzise Balancierung des Fahrrads mittels eines PID-Reglers und die zuverlässige Spurverfolgung sowie Navigation mithilfe eines MPC-Reglers sicherstellt. Die gewonnenen Simulationsergebnisse bestätigen die Machbarkeit und Effizienz des entwickelten Ansatzes und legen damit eine solide Grundlage für die zukünftige Implementierung der Regelung in einem realen autonomen Fahrrad.

Contents

1	Abstract	2
2	Einleitung	5
2.1	Problemstellung	5
2.2	Stand der Technik	5
2.3	Bisherige Arbeiten	6
2.3.1	Anna Ranz (2021) – Hardware-Grundlagen	7
2.3.2	Amine Karabila (2022) – Erste funktionierende Regelung	7
2.3.3	Jonah Zander (2023) – Optimierte PID-Balanceregung	8
2.3.4	Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung	8
3	Grundlagen	10
3.1	PID-Regelung – Prinzip und Eignung für das Balance-Problem	10
3.2	MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem	11
3.3	YOLO-Grundlagen und Segmentierung	12
4	Webots-Simulationsumgebung	14
4.1	Überblick	14
4.2	Struktur der Simulationsumgebung	15
4.3	Das Fahrradmodell	17
4.4	Sensoren	18
4.4.1	Inertialmesseinheit (IMU)	18
4.4.2	Kamera (Videosensor)	20
4.4.3	Motorsensoren (Inkrementalgeber)	22
4.5	Aktuatoren	24
4.6	Controller	28
4.7	Angebundene GUI	31
5	Kombinierte Regelung	33
5.1	Portierung des Balance-PID (Zander)	33
5.2	Portierung des Fahrtrichtungs-MPC (Giray)	34
5.3	Strategien der Spurführung	37
5.3.1	YOLO-Segmentierung (YOLOv8) + MPC	37
5.3.2	Fallback-Strategie (Kanten/ROI) + MPC	41
5.4	Gesamtregelung	43

6	Ergebnisse	44
6.1	Fahrrad Datenblatt	44
6.2	Teststrecken	45
6.2.1	Kurvenfahrt	45
6.2.2	S-Kurve	50
6.2.3	Gerade Strecke mit Seitenwind	54
6.2.4	Gerade Strecke mit Höhenunterschied	58
6.3	Rechenzeit & Echtzeitfähigkeit in Webots (Vergleich zum Realfahrrad)	64
7	Grenzen der Arbeit	67
8	Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)	68
9	Fazit	69
10	Literaturverzeichnis	70

2 Einleitung

2.1 Problemstellung

In diesem Kapitel wird die zentrale Herausforderung dieser Arbeit präsentiert. Ohne regelnden Eingriff kippt ein Fahrrad im Stand oder bei niedriger Geschwindigkeit. Erst durch die Kombination aus gyroskopischen Effekten der rotierenden Räder, dem Nachlauf der Lenkgeometrie und einem aktiven Regler lässt sich eine stabile Fahrt realisieren. Die vorliegenden Arbeiten verfolgen daher das Ziel, ein autonomes fahrendes Fahrrad zu entwickeln, das sich sowohl selbst balancieren als auch eine vorgegebene Strecke verfolgen kann. Hierfür wurde eine hochrealistische Simulationsumgebung in Webots aufgebaut, die eine duale Reglerarchitektur abbildet. Ein schneller Balance-Controller, implementiert in C, stabilisiert den Rollwinkel mit einer Abtastzeit von 2 ms (500 Hz) und gibt einen Lenkwinkel an den Antrieb weiter. Parallel dazu extrahiert ein Vision-Controller in Python mit niedrigerer Frequenz (20 Hz) Informationen aus Kamerabildern, plant eine Trajektorie und überträgt entsprechende Lenkwinkel- und Geschwindigkeitsbefehle. Die Herausforderung besteht darin, diese beiden Regelschleifen so zu koppeln, dass das Fahrzeug stabil bleibt und zugleich auf visuelle Reize reagiert. Darüber hinaus soll die simulierte Physik so realitätsnah sein, dass die Ergebnisse auf reale Fahrversuche übertragbar sind.

Abgrenzung zu realitätsnahen und simulationsbasierten Ansätzen Im Gegensatz zu experimentellen Untersuchungen an realen Fahrradplattformen (z. B. Arbeiten von Zander [35] und Karabila [21]) liegt der Fokus dieser Arbeit auf einer vollständig simulationsbasierten Methodik unter Nutzung von Webots. Frühere Simulationsstudien (z. B. Zander [35], Karabila [21]) berücksichtigen dynamisch variierende Geschwindigkeiten, etwa aufgrund von Gefälle oder Bremsmanövern. In der hier vorliegenden Simulation hingegen wird die Geschwindigkeit als konstant angenommen. Auch die Sensorik und Regelungshardware unterscheiden sich wesentlich: Reale Versuchsaufbauten (etwa bei Karabila [21]) nutzten einen BNO-005 IMU-Sensor zur Erfassung von Eulerwinkeln sowie PWM-basierte Motoransteuerung. Die Webots-Simulation ermöglicht hingegen direkten Zugriff auf Zustandsvariablen, ohne den Einsatz realer Hardwarekomponenten oder deren Signalverarbeitung. *Zusammenfassend zeichnet sich diese Arbeit durch ihre rein simulationsbasierte Herangehensweise, die Annahme konstanter Geschwindigkeit, den Verzicht auf adaptive Regler sowie den Ausschluss hardwarebedingter Schnittstellen (Sensorik, PWM, ...) aus.*

2.2 Stand der Technik

Die Regelung eines selbstbalancierenden Fahrrads basiert auf zwei zentralen Grundlagen:

- (i) einem hinreichend genauen dynamischen Modell, das die wesentlichen Fahr- und Stabilitätseigenschaften beschreibt, und
- (ii) einer abgestimmten Reglerarchitektur, die sowohl die Aufrechterhaltung des Gleichgewichts als auch die zuverlässige Spurführung gewährleistet.

Auf der Modellseite etablierten Meijaard et al. den Benchmark der linearisierten Whipple-Gleichungen, der bis heute als Referenz für Analyse, Identifikation und Reglersynthese dient [25]. Ergänzend zeigte Kooijman et al., dass Selbststabilität *ohne* gyroskopische oder Caster-Effekte ¹ möglich ist,

¹„Caster-Effekte“ entstehen, wenn der Aufstandspunkt des Vorderrads hinter der Lenksäule liegt, wodurch das Rad sich durch die Bewegung selbständig in Fahrtrichtung ausrichtet :contentReference.

womit die Interaktion von Massenverteilung, Nachlauf und Lenkkinematik als zentrale Gestaltungsgrößen belegt wurde [22]. Diese Ergebnisse motivieren regelungstechnische Ansätze, die das Prinzip „*Steer into the Fall*“ explizit nutzen und die Lenkung als primären Stellgrad einsetzen.

Für die aktive Balanceregung über die Lenkung liegt mit Andreo et al. eine LPV-Formulierung² vor, die die geschwindigkeitsabhängige Dynamik berücksichtigt und Stabilität in einem definierten Geschwindigkeitsbereich nachweist [1]. Damit wird ein wichtiger Praxisaspekt adressiert, da Fahrräder ihre Vorwärtsgeschwindigkeit ständig ändern und eine rein LTI-Betrachtung³ nur lokal gültig ist. Als Alternative wurden Balanceregler mit Reaktionsrädern untersucht. Kanjanawanishkul vergleicht LQR- und nichtlineare MPC-Ansätze unter Berücksichtigung von Aktuatorsättigungen und zeigt, dass bereits einfache Quadratikregler bei Stand- oder Niedriggeschwindigkeit Stabilisierung ermöglichen, jedoch mit zusätzlicher Masse und höherem Energiebedarf verbunden sind [20]. Ein dritter Ansatz, der in der Robotik verbreitet ist, nutzt Control-Moment-Gyroskope (CMG). Park und Yi belegen experimentell, dass ein scissored-pair-CMG störende Quermomente kompensiert und so eine robuste Rollstabilisierung erlaubt, die geeignet für stationäre oder sehr langsame Fahrzustände ist, allerdings mit erheblichem mechanischem Aufwand [28].

Für die gekoppelte Aufgabe aus Balance *und* Spurführung hat sich eine Kaskadenarchitektur bewährt. Eine schnelle innere Balanceschleife (PID/LQR) und eine langsamere äußere Spurführungsschleife (häufig MPC) wurden von Persson et al. entworfen. Der äußere MPC optimiert dabei unter harten Nebenbedingungen (Roll-, Lenk- und Heading-Grenzen sowie Positionsbeschränkungen) die Soll-Rollwinkel und damit die Kursführung, während ein innerer PID die Soll-Rollwinkel über die Lenkrate stabilisiert [30]. Die Autoren zeigen auf realitätsnahen Strecken (OpenStreetMap-Go-Kart-Track), dass die Kaskade enge Radien und variable Geschwindigkeiten meistert und zugleich eine saubere Trennung der Zeitskalen ermöglicht. Dieser Aufbau ist unmittelbar anschlussfähig an *do-mpc*-basierte Implementierungen, da Optimierungs-, Nebenbedingungs- und Prädiktionshorizonte explizit parametrierbar sind.

Parallel zur Reglerseite hat sich die Umfeldwahrnehmung stark weiterentwickelt. Ein prominenter Systemdemonstrator ist das Tsinghua-„Tianjic“-Fahrrad, das Kameraperzeption (Objekterkennung/-tracking), Sprachkommandos, Hindernisvermeidung und Balance auf einem eingebetteten Chip integriert [29]. Die Studie ist weniger eine methodische Neuentwicklung der Fahrdynamik, zeigt aber überzeugend, dass visuelle Spurführung und Balanceregung *in Echtzeit* auf kompakten Plattformen zusammengeführt werden können, somit ein wichtiger Befund für kamerabasierte Spurhaltung.

2.3 Bisherige Arbeiten

Die im vorherigen Abschnitt dargestellten Forschungsarbeiten zeigen, dass sowohl klassische als auch moderne Regelungsverfahren prinzipiell geeignet sind, ein selbstbalancierendes Fahrrad zu stabilisieren und mit Trajektorienfolgeregelungen zu kombinieren. Während sich diese Studien überwiegend auf externe Forschungsgruppen beziehen, existieren auch an der Goethe-Universität Frankfurt eine Reihe aufeinander aufbauender Abschlussarbeiten, die wesentlich zur Entwicklung des hier untersuchten Systems beigetragen haben.

²LPV (Linear Parameter-Varying) beschreibt Systeme, deren Dynamik von zeit- oder zustandsabhängigen Parametern beeinflusst wird und erlaubt so eine erweiterte Stabilitätsanalyse über verschiedene Betriebsbereiche hinweg.

³LTI (Linear Time-Invariant) beschreibt Systeme mit zeitlich unveränderlichen Parametern; diese Annahme ist nur lokal gültig, wenn die Geschwindigkeit des Fahrrads konstant bleibt.

2.3.1 Anna Ranz (2021) – Hardware-Grundlagen

Die Bachelorarbeit von Anna Ranz markierte den praktischen Ausgangspunkt des Projekts und schuf eine erstmals vollständig lauffähige Experimentalplattform, auf der regelungstechnische Konzepte an realer Hardware untersucht werden konnten [31]. Zentrales Element ist ein *BeagleBone Black* als Steuereinheit, der den Lenkantrieb über einen *MD49*-Controller (Motor: *EMG49*) via **UART** ansteuert und die Inertialmessdaten einer *BNO055*-IMU über **I²C** erfasst. Ergänzt wird die Architektur durch einen Nabenmotor am Hinterrad, der eine konstante Vorwärtsfahrt sicherstellt. In dieser Kombination standen erstmals mechanische Plattform, Sensoren und Aktoren mit klar definierten Schnittstellen zur Verfügung, um Stabilisierung und Lenkregelung systematisch zu evaluieren.

Im Rahmen der Inbetriebnahme implementierte Ranz erste PID-Regelkreise, validierte die IMU-Sensorik in separaten Testprogrammen und analysierte die Dynamik des Gesamtsystems mit Blick auf Massenträgheit, Raddynamik und Lenkkinematik. Besondere Aufmerksamkeit galt dem Zusammenspiel von Trägheitsmoment, Lenkwinkel und Rückstellkräften, da diese Größen die Querstabilität wesentlich bestimmen. Ergänzende Messreihen dienten der Bewertung der BNO055 hinsichtlich Genauigkeit und Latenz der Rollwinkelbestimmung. Eine sorgfältige Kalibrierung stellte sicher, dass die Datenqualität den Anforderungen eines geschlossenen Regelkreises genügt.

Die Arbeit dokumentiert die Hardware umfassend anhand von Schaltplänen, 3D-Modellen und Messaufnahmen und bildet damit die hardwareseitige Referenzbasis für alle nachfolgenden Entwicklungen [31]. Zugleich wurden systemische Grenzen der ersten Ausbaustufe transparent herausgearbeitet, darunter die begrenzte Reaktionsgeschwindigkeit des *MD49*-Motorcontrollers sowie Latenzen in der **UART**-Kommunikation und somit konkrete Ansatzpunkte für spätere Verbesserungen identifiziert.

Insgesamt kennzeichnet die Arbeit von Ranz den Übergang von theoretischen Vorüberlegungen hin zu einer belastbaren Forschungsplattform.

2.3.2 Amine Karabila (2022) – Erste funktionierende Regelung

Amine Karabila schloss 2022 unmittelbar an den Hardware-Prototyp von Ranz an und überführte das System von einzelnen Stellversuchen zu einem geschlossenen, echtzeitfähigen Regelkreis. Kernbestandteil ist eine dreistufige P-Regelung auf dem *BeagleBone Black*, die den Rollwinkel der BNO055-IMU [2] und die Lenkerstellung des MD49-Encoders [8] auswertet, während ein hinterer Nabenmotor die Vortriebsdrehzahl konstant hält [21]. Damit gelang erstmals eine softwareseitig realisierte Balancekontrolle, die den Forschungsprototyp in eine echte Versuchsplattform überführte.

Ein erstes Hindernis war die Strombegrenzung des MD49, die den EMG49-Lenkmotor zu langsam ansprechen ließ. Karabila ersetzte deshalb den Leistungsteil durch eine BTS7960-H-Brücke [18] und verwendete den MD49 nur noch zur hochauflösenden Wegmessung. Nicht-blockierende **UART**-Aufrufe verkürzten die Encoderabfrage von rund 4 ms auf knapp 1 ms, sodass die Gesamttransportzeit des Reglers auf etwa 2 ms sank. Damit wird deutlich, dass die Stabilität des Regelkreises aus dem Zusammenspiel leistungsfähiger Algorithmen und einer auf minimale Latenzen ausgelegten Hard-/Software resultiert.

Die erste Regelversion koppelte die Motorgeschwindigkeit über einen gestreckten tanh-Verlauf an die Verweildauer des Rollwinkels in einer „habitablen Zone“. Außerhalb des Labors erwies sich dieses Verfahren jedoch als zu träge. Der Lenker schlug regelmäßig an die Endanschläge, und

Stützräder mussten das Fahrrad vor dem Kippen bewahren. Die finale Lösung verknüpfte dagegen den gemessenen Rollwinkel direkt mit einem geschwindigkeitsabhängigen Soll-Encoderwert. Dadurch pendelte der Lenker nur noch um wenige Grad um die Nullposition.

Validiert wurde der Regler im Hof des Informatikgebäudes ($\approx 9.5 \times 26.5$ m, fünf Prozent Steigung). Bei Sollgeschwindigkeiten von 6–10 km/h blieb das Fahrrad während aller aussagekräftigen Fahrten aufrecht.

Damit bewies Karabilas Arbeit erstmals die Realisierbarkeit einer selbstbalancierenden Regelung und schuf die Grundlage für die PID-Optimierungen von Zander sowie die MPC-Navigation von Giray. Insbesondere machte sie deutlich, dass die Regelgüte stark von niedrigen Latenzen und einer robusten Kopplung zwischen Sensoren und Aktoren abhängt, ein zentrales Motiv, das auch in den nachfolgenden Arbeiten beibehalten und weiterentwickelt wurde.

2.3.3 Jonah Zander (2023) – Optimierte PID-Balanceregung

Zander knüpfte 2023 an die Vorarbeiten an und formulierte als Kernziel, das Fahrrad mindestens 60 s auf ebener Strecke ohne Bodenkontakt der Stützräder stabil fahren zu lassen [35]. Zur Umsetzung entwickelte Zander eine modular aufgebaute C-Software auf dem BeagleBone Black (Debian), in der Multi-Threading Sensor-/Aktor-I/O, Reglerkern und Logging entkoppelt. Zeitkritische Abtast- und Vorverarbeitungsaufgaben wurden in die Programmable Realtime Units (PRUs) verlagert, wodurch deterministische Lesezyklen für Fahr- und Lenkwinkel im zweistelligen kHz-Bereich möglich wurden. Die effektive Transportverzögerung zwischen Messung und Stellgröße sank dadurch deutlich, was die Stabilitätsreserven des Reglers erhöhte [35].

Die Balanceregung realisierte Zander als klassisch ausgelegten PID-Regler, dessen Parameter iterativ auf Basis umfangreicher Mess- und Logdaten optimiert wurden. Im Vergleich zur zuvor eingesetzten dreistufigen P-Logik [21] reduziert der I-Anteil den stationären Regelfehler (Ausregelgüte), während der D-Anteil die Einschwingdämpfung erhöht. In der Software wurden robuste Elemente berücksichtigt, unter anderem Begrenzung und Glättung des Lenkmotor-Solls (Rate- und Amplitudenlimits), sorgfältige Filterung der Eingangssignale (Encoder/IMU) zur Rauschunterdrückung bei erhaltener Phasenreserve sowie latenzarme Pfade zwischen PRU-Puffern und Reglerkern zur Minimierung von Jitter. Das Loggingsystem zeichnete synchronisierte Sensor- und Aktordaten auf und erlaubte eine systematische Auswertung der Reglerwirkung [35].

Neben dem Reglerkern stärkte Zander die Betriebssicherheit. Dabei wurden zwei Notausschalter (separate Lenkungs- bzw. Gesamtabschaltung), eine Reißleine zur sofortigen Trennung der Lenkungsversorgung in kritischen Situationen sowie ein Fail-Safe für den RC-Empfänger (Stop bei Signalverlust) begrenzen Risiken unkontrollierter Lenkereinschläge und mechanischer Schäden eingesetzt [35].

In experimentellen Fahrttests blieb das Fahrrad über mehr als 60 s stabil in Balance, ohne dass die Stützräder den Boden berührten. Bei konstanter Geschwindigkeit auf ebenem Untergrund waren zudem gesteuerte Richtungsänderungen per Fernbedienung möglich, ohne die Balance zu verlieren [35]. Damit erzielte Zander den bis dahin längsten autonom balancierten Fahrbetrieb des Prototyps.

2.3.4 Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung

Aufbauend auf der in [35] erreichten stabilen Balanceregung verlagerte Giray (2024) [10] den Schwerpunkt auf die autonome Fahrtrichtungswahl mittels Computer Vision und modellprädiktiver

Regelung. Ziel war eine monokamera-basierte Pipeline, die in Echtzeit auf eingebetteter Hardware lauffähig ist und aus Bilddaten eine Referenztrajektorie ableitet, der das Fahrrad folgen kann, ohne die Balance zu verlieren.

Die Wahrnehmungskomponente setzt auf ein effizientes Segmentierungsmodell aus der YOLOv8-Familie [19]. Nach einer Evaluation alternativer Verfahren (u. a. FastSAM [36]) fiel die Wahl aufgrund des guten Verhältnisses von Genauigkeit und Latenz auf ein Ultralytics-Modell in der Segmentation-Variante ⁴. Das Netz wurde mit einem angepassten Datensatz trainiert und für die Inferenz mittels TensorRT auf dem NVIDIA Jetson Nano beschleunigt, sodass die Bildauswertung in Zyklen von etwa 100 ms erfolgen konnte. Aus den segmentierten Szenen werden Stützpunkte extrahiert und über Spline-Interpolation zu einer glatten Referenzspur verdichtet, die die gewünschte Fahrlinie repräsentiert.

Für die Fahrregelung implementierte Giray einen Model Predictive Controller (MPC) auf Basis eines Einspurmodells [17]. Der MPC minimiert über einen endlichen Horizont Abweichungen von der Referenzspur bei gleichzeitig moderatem Stellaufwand und berücksichtigt Nebenbedingungen des Systems. In jedem Rechenzyklus wird das Optimierungsproblem mit der aktuellen Zustandsschätzung gelöst. Modellierung, Trajektorien-Interpolation und MPC-Optimierung wurden zunächst in Python umgesetzt und auf die Jetson-Plattform portiert.

Die Pipeline detektierte in den Simulationen Fahrbahnkanten und einfache Hindernisse zuverlässig. Daraus ergaben sich stabile Referenztrajektorien. Zugleich wurden Grenzen sichtbar, da eine explizite Kollisionsvermeidung noch nicht integriert wurde. Eine Tiefeninformation war nur indirekt über Bildgeometrie vorhanden. Nach Giray ist mit neueren Segmentierungsansätzen und leistungsfähigeren Edge Plattformen eine weitere Reduktion der Latenz zu erwarten [36] [19].

Für die vorliegende Arbeit ist die Einordnung klar: Die von Zander übernommene PID-Balanceregung bildet die innere Schleife, während Girays MPC-basierter Spurhalter als äußere Schleife in *Webots* integriert wird. Diese Schichtung erlaubt es, die Kopplung von Balancehaltung und visionbasierter Trajektorienverfolgung systematisch zu untersuchen und in einer realitätsnahen Simulation zu kalibrieren, bevor reale Fahrversuche erfolgen.

⁴Gemeint ist die YOLOv8-Ausprägung, die neben Klassen/Boxen pro Objekt Pixelmasken (Semantik/Instanz) vorhersagt.

3 Grundlagen

Dieses Kapitel führt die theoretischen und methodischen Grundlagen ein, die für das Verständnis der im Folgenden vorgestellten Reglerarchitekturen und Bildverarbeitungsverfahren notwendig sind. Zunächst wird die PID-Regelung als klassischer Vertreter der linearen Regelungstechnik erläutert und ihre Eignung zur Stabilisierung des unteraktuierten Balance-Problems des Fahrrads diskutiert. Anschließend folgt eine Einführung in das Prinzip der modellprädiktiven Regelung (MPC), die eine vorausschauende Berechnung von Stellgrößen erlaubt und somit für die Fahrtrichtungsregelung prädestiniert ist. Abschließend werden die Grundlagen der Objekterkennung mit YOLO sowie der Bildsegmentierung vorgestellt, um die visuelle Wahrnehmung und Spurführung des autonomen Fahrrads zu ermöglichen.

3.1 PID-Regelung – Prinzip und Eignung für das Balance-Problem

Die Proportional-Integral-Differential (PID)-Regelung ist der am weitesten verbreitete Ansatz der klassischen Regelungstechnik. Ein PID-Regler ermittelt die Stellgröße $u(t)$ aus dem Fehler $e(t)$ zwischen Sollwert SP und dem gemessenen Istwert PV nach

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt} \quad (1)$$

wobei K_P , K_I und K_D die Verstärkungsparameter für proportionale, integrale und derivative Anteile sind. Der proportionale Anteil erzeugt eine Stellgröße proportional zur momentanen Fehlergröße und reduziert so die Anstiegszeit. Der Integrator akkumuliert den Fehler und eliminiert stationäre Abweichungen, kann aber langsame Dynamik verursachen [16]. Die Derivative wirkt der Änderung des Fehlers entgegen und dämpft dadurch Überschwinger; sie erhöht jedoch die Empfindlichkeit gegenüber Messrauschen, weshalb in vielen Anwendungen filternde Maßnahmen erforderlich sind.

PID-Regler werden eingesetzt, wenn eine robuste und einfach implementierbare Rückführung gefordert ist und das Systemverhalten hinreichend durch lineare Modelle beschrieben werden kann. In der Literatur wird die PID-Regelung häufig als Standardlösung für nicht integrierende Prozesse bezeichnet. Ihre Parametrierung erfordert eine sorgfältige Abwägung zwischen Ansprechverhalten und Stabilitätsreserve: Eine Erhöhung des proportionalen Anteils verringert die Regelabweichung, kann aber zu Instabilitäten führen; ein zu hoher Integrationsanteil kann Integrator-Windup verursachen; die derivativen Anteile sind besonders rauschanfällig.

Beim selbstbalancierenden Fahrrad lässt sich die Balanceproblematik auf das klassische inverses-Pendel-Problem zurückführen. Ein Rad, dessen Schwerpunkt über der Aufstandsfläche liegt, befindet sich in einem instabilen Gleichgewicht; kleinste Störungen führen zum Kippen. Um das System aufrecht zu halten, muss der Regler in Echtzeit Stellmomente applizieren, die den kippspezifischen Drehmomenten entgegenwirken. Die Anforderungen an geringe Latenz und hohe Abtastrate werden etwa beim inversen Pendel mit Updatefrequenzen von mehreren Kilohertz umgesetzt. Dies verdeutlicht, dass auch beim Fahrrad eine schnelle Regelungsschleife erforderlich ist, wie sie beispielsweise in den Bachelorarbeiten von Karabila und Zander implementiert wurde. Beide Arbeiten basieren auf P- bzw. PID-Reglern, die den Neigungswinkel mittels IMU messen und über den Lenkmotor oder den Antrieb korrigieren [3].

Insgesamt ist die PID-Regelung für das Balance-Problem des autonomen Fahrrads wegen ihrer geringen Komplexität und Rechenlast, der einfachen Implementierung auf Mikrocontrollern und der guten Robustheit gegenüber Modellunsicherheiten geeignet. Die Nichtlinearität des Fahrverhaltens

und die starken Störmomente durch Schräglagen erfordern jedoch eine sorgfältige Abstimmung der Parameter und gegebenenfalls Erweiterungen wie Anti-Windup-Mechanismen oder Filterung der derivativen Komponente, um die Regelung stabil und performant zu halten.

3.2 MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem

Model predictive control (MPC) nutzt ein internes Modell des Systems, um dessen zukünftiges Verhalten über einen endlichen Zeithorizont vorherzusagen. Auf Grundlage dieser Vorhersage und des aktuellen gemessenen bzw. geschätzten Zustands werden optimale Stellgrößen berechnet, die einen definierten Kostenfunktional minimieren und gleichzeitig die Systembeschränkungen einhalten. Nach jedem Abtastintervall wird nur die erste Stellgröße angewendet und die Optimierung mit vorwärts verschobenem Horizont neu gelöst, weshalb man auch von *receding horizon control* spricht. Zu den wesentlichen Vorteilen von MPC zählen die proaktive Reaktion auf künftige Störungen, die Möglichkeit nichtlineare Modelle ohne Linearisierung zu berücksichtigen und die explizite Berücksichtigung von physikalischen und sicherheitsrelevanten Beschränkungen. Im Gegensatz zu reaktiven PID-Reglern kann MPC damit frühzeitig gegensteuern und gleichzeitig harte Grenzen, etwa Stellweg- oder Geschwindigkeitsbeschränkungen, einhalten [9].

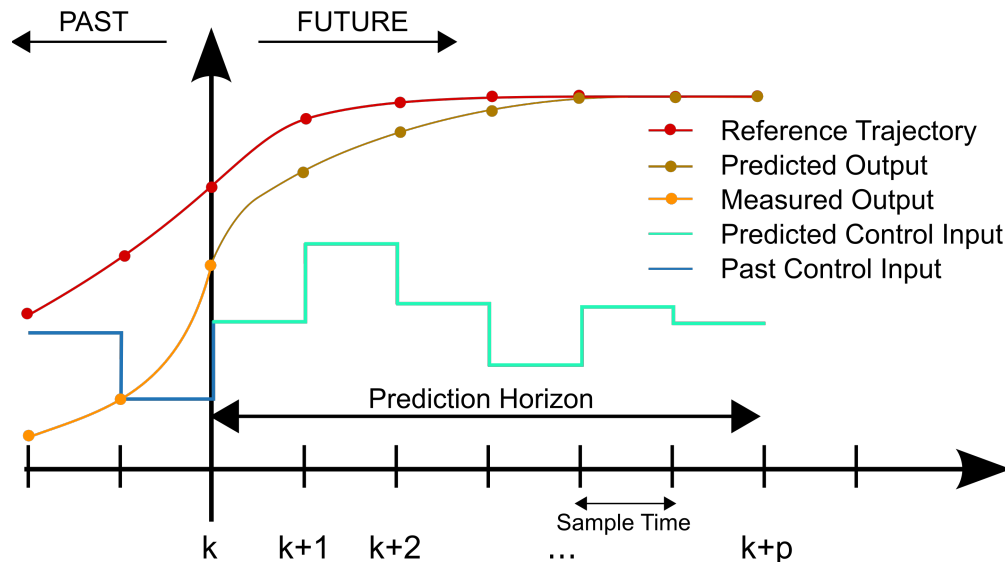


Figure 1: MPC-Regler

Beim selbstbalancierenden Fahrrad besteht das Fahrtrichtungsproblem darin, aus Kamerainformationen einen Sollkurs abzuleiten und das Fahrzeug mit begrenztem Lenkradius und Lenkrate entlang dieser Trajektorie zu führen. Der in der Vision-Regelung verwendete `VisionMPCController` modelliert das Fahrzeug als einfaches Fahrraddynamikmodell mit Zustandsvektor $x = [x, y, v, \psi]$ für Position, Geschwindigkeit und Gierwinkel sowie Eingangsvektor $u = [a, \delta]$ für Beschleunigung und Lenkwinkel. Aus dem aktuellen Zustand und dem lateral gemessenen Fehler wird ein Referenzpfad erzeugt (Ziel-Yaw proportional zum Fehler), der über einen Horizont T vorwärts projiziert wird. Anschließend löst der MPC ein konvexes Optimierungsproblem, das die Abweichung von diesem Pfad, die Eingangsgrößen sowie deren zeitliche Änderung in einer quadratischen Kostenfunktion gewichtet und durch Beschränkungen auf maximalen Lenkwinkel, Lenkrate, Geschwindigkeitsbereich und Beschleunigung ergänzt. Stehen geeignete Optimierungslöser (z. B. `cvxpy`) zur Verfügung,

werden aus diesem Problem kontinuierliche Stellgrößen $\delta \in [-\delta_{\max}, \delta_{\max}]$ und a berechnet; andernfalls fällt der Controller auf eine einfache P-Regelung zurück. Vor der Weitergabe an den schnellen Balance-Controller werden die resultierenden

Lenkwinkel und Beschleunigungen normiert, geglättet und auf Änderungsraten begrenzt, um ruckfreie Übergänge zu gewährleisten [12].

Die Eignung der MPC Regelung für das Fahrtrichtungsproblem ergibt sich aus der Vorhersagefähigkeit und der Einbeziehung von Beschränkungen. Das Fahrradsystem ist nichtlinear und unteraktuiert; um einer gekrümmten Spur zu folgen, muss der Regler zukünftige Kurvenverläufe berücksichtigen und gleichzeitig mechanische Limits (Lenkeinschlag, Lenkrate) respektieren. MPC kann diese Randbedingungen explizit in das Optimierungsproblem integrieren und die Stellgrößen so wählen, dass sowohl die Trajektorienabweichung als auch der Stellaufwand minimiert werden. In Yasin Girays Masterarbeit wurde der visionbasierte MPC-Regler mit einem Horizont von $T = 3$ und den genannten Zuständen und Eingängen implementiert und mit Gewichten auf Zustände, Eingänge und deren Differenzen parametrisiert. Simulationen zeigten, dass das Fahrrad mithilfe des MPC gleichmäßig entlang der aus der Bildsegmentierung abgeleiteten Spur navigierte und dabei die mechanischen Limits einhielt. Somit stellt MPC einen geeigneten Ansatz dar, um das autonome Fahrrad stabil und vorausschauend [32] entlang einer vorgegebenen Fahrtrichtung zu steuern.

3.3 YOLO-Grundlagen und Segmentierung

You Only Look Once (YOLO) ist eine Reihe von Echtzeit-Objekterkennungsverfahren, die auf Convolutional Neural Networks basieren. Das ursprüngliche YOLO wurde 2015 von Redmon et al. vorgestellt und erfordert im Gegensatz zu region-proposal-basierten Ansätzen nur einen Vorwärtsthroughlauf durch das Netz, um Klassen und Bounding Boxes für alle Objekte in einem Bild zu bestimmen [34]. Neuere Versionen wie YOLOv8 erweitern diese Architektur und bieten neben der Objekterkennung auch Bildklassifikation und Instanzsegmentierung; sie verwenden eine moderne Backbone-Struktur, einen ankerfreien Detektionskopf und eine verbesserte Verlustfunktion und sind auf verschiedenen Hardwareplattformen effizient einsetzbar [27].

Bei der Bildsegmentierung wird jedem Pixel eines Bildes ein Label zugeordnet. Semantic Segmentation ordnet jedem Pixel eine Objektklasse zu, ohne zwischen einzelnen Instanzen zu unterscheiden, während Instance Segmentation jedes Pixel einer individuellen Objektinstanz zuordnet [33]. Panoptic Segmentation kombiniert beide Ansätze, indem sie sowohl die Klasse als auch die Instanz für jedes Pixel identifiziert [33]. In der Instanzsegmentierung liefert das Netzwerk neben Bounding Boxes auch Masken, die die Form jedes erkannten Objekts abbilden.

YOLOv8 integriert eine Instanzsegmentierungskomponente, deren Kopf auf dem YOLACT-Konzept basiert und die zu den Detektionsausgängen zusätzliche Masken pro Objekt berechnet [27]. Durch diese Erweiterung kann ein Modell in einem einzigen Forward-Pass sowohl Objekte lokalisieren als auch deren pixelgenauen Umrisse bestimmen. Wie bei früheren YOLO-Versionen werden Varianten unterschiedlicher Größe bereitgestellt (Nano, Small, Medium, Large usw.), um den Kompromiss zwischen Geschwindigkeit und Genauigkeit zu steuern [27].

In dieser Arbeit wird eine YOLOv8-basierte Segmentierung für die Umfeldwahrnehmung genutzt. Das Modell ist auf vier Klassen trainiert – Wiese („meadow“), Hindernis („obstacle“), Hauptfahrbahn („street_main“) und Seitenfahrbahn („street_side“) [11] – und liefert pro Klasse ein Pixelmasken Overlay sowie die zugehörigen Bounding Boxes. Anschließend wird aus dem Segment „street_main“ der laterale Fehler bestimmt, der als Eingang für die Pfadverfolgung dient. Die

genauen Details zur Modellauswahl, Datensatzgenerierung und Implementierung werden in einem späteren Kapitel beschrieben. Hier soll lediglich hervorgehoben werden, dass die Kombination aus schneller YOLO-Detektion und Instanzsegmentierung eine effiziente und genaue Grundlage für die kamerabasierte Fahrspurerkennung bildet.



Figure 2: YOLO-Segmentierung

4 Webots-Simulationsumgebung

Hier wird die in Webots realisierte Simulationsumgebung vorgestellt, die als zentraler Versuchsstand für Entwurf, Parametrierung und Evaluation der kombinierten Regelung des selbstbalancierenden Fahrrads dient. Ziel ist es, ein reproduzierbares und sicheres Testbett zu schaffen, in dem das physikalische Fahrverhalten, die sensorgeführte Pfadverfolgung sowie die Kopplung von schneller Balance-Regelung und langsamer Vision-/Trajektorienplanung unter kontrollierten Bedingungen untersucht werden können. Webots bietet dafür eine deterministische Dynamiksimulation auf Basis von ODE, eine hierarchische Szenenbeschreibung und eine klare Trennung von Geometrie, Physik, Sensorik, Aktuatorik und Controllern – Eigenschaften, die systematische Parameterstudien, Logging und automatisierte Auswertung wesentlich erleichtern [4].

Das Kapitel ist wie folgt strukturiert: Zunächst wird ein Überblick über Webots und die genutzten Kernfunktionen gegeben. Anschließend wird die konkrete Szenenstruktur der Weltdatei erläutert, einschließlich globaler Simulationsparameter und der Organisation der Knoten. Darauf aufbauend folgt die Beschreibung des Fahrradmodells mit seinen Gelenken, Kontakt- und Masseneigenschaften sowie der eingesetzten Sensoren und Aktuatoren. Abschließend werden die Controller-Prozesse und deren Interaktion skizziert sowie die angebundene GUI bzw. Auswertepfade beschrieben. Diese Gliederung macht transparent, wie ausgehend von einem konsistenten physikalischen Modell die Regelungsalgorithmen iterativ entwickelt, validiert und für den späteren Transfer auf den Realprototyp vorbereitet werden.

4.1 Überblick

Webots ist ein frei verfügbares, quelloffenes und plattformunabhängiges Robotik Simulationsprogramm, das seit 1998 kontinuierlich gepflegt wird und in Forschung, Lehre und Industrie breite Anwendung findet [4, 26]. Es kombiniert eine moderne QtBenutzeroberfläche mit einer Mehrkörper-Dynamiksimulation auf Basis der *Open Dynamics Engine* (ODE) sowie einer OpenGLgestützten RenderingPipeline. Dadurch lassen sich fotorealistische 3DSzenen mit deterministischem Integrationschema und feingranularer Kontrolle der Simulationszeit schrittgenau ausführen [4].

Kern des WebotsModells ist die hierarchische Szenenrepräsentation in *WorldDateien* (*.wbt; VRML-basiert). Eine Welt kapselt globale Parameter (z. B. Zeitauflösung, Kontakt und Reibungsmodelle), Umgebungsobjekte (Boden, Licht, Hintergrund) sowie Roboter-knoten mit Sensorik, Aktuatorik und Masseneigenschaften. Geräte (z. B. Kamera, IMU, Lidar, *RotationalMotor*) sind als Nodes mit klar definierten Schnittstellen realisiert; ihre Parameter (Auflösung, Bildfrequenz, Dämpfung, Grenzwerte) sind reproduzierbar konfigurierbar. Controller laufen als getrennte Prozesse in C/C++, Python, Java oder MATLAB und kommunizieren zur Laufzeit über wohldefinierte APIs mit der Simulationswelt; mittels ROS 2Bridge ist zudem eine Einbettung in verteilte Robotik-Stacks möglich [4, 14]. Eine spezielle *SupervisorAPI* erlaubt höherwertige Eingriffe (Positionsreset, automatisiertes Logging, MetrikAuswertung) und bildet die Grundlage für reproduzierbare BenchmarkingPipelines [4].

Für regelungsorientierte Studien bietet Webots drei Eigenschaften, die in dieser Arbeit zentral sind: (i) *Determinismus* durch festen Zeitschritt und explizite Integrationsreihenfolge, was faire Parameterstudien ermöglicht; (ii) *Modularität* durch die Trennung von Geometrie, Physik, Sensorik/Aktuatorik und Controllern, wodurch Varianten (z. B. Reibungskoeffizienten, Trägheiten, Abtastraten) isoliert untersucht werden können; (iii) *Automatisierbarkeit* über den Supervisor, u. a. für Szenario-Resets, Störgrößeninjizierung, BatchRuns und standardisierte Ergebnisberichte [4].

Relevanz für *fahrdynamische Einspurfahrzeuge*: In jüngeren Arbeiten wird Webots gezielt zur Simulation von Fahrrädern und Motorrädern eingesetzt. Das OpenSourceAddon *WeBikes* erweitert Webots um ein konfigurierbares, physikalisch fundiertes Einspurfahrzeugmodell (Geometrie, Masseverteilung, Reifenkontakt, Steuerkinematik) und demonstriert, dass typische Phänomene wie Selbststabilisierung, Lenkdynamik und Kurvenfahrt reproduzierbar abgebildet werden können [23]. Damit wird ein freier, reproduzierbarer Vergleich zu Spezialwerkzeugen ermöglicht und eine Brücke zwischen Regelungsentwurf (z.B. Balanceregulung, MPCTrajektorienplanung) und virtuellen Fahrversuchen geschlagen. Über den gleichen Mechanismus nutzen weitere Studien Webots als DigitalTwinPlattform für komplexe, materialabhängige Systeme (z. B. weiche Greifer), was die Übertragbarkeit auf neuartige Robotertypen unterstreicht [15].

Für diese Arbeit ist Webots folglich ein geeignetes Testbett, um die Kopplung einer schnellen BalanceRegelung (PID, $\approx$ kHzSkala) mit einer langsameren vision bzw. modellprädiktiven Pfadführung (tens of Hz) systematisch zu entwerfen, zu parametrieren und zu evaluieren. Die klare Trennung der Subsysteme (Physik, Sensorik, Controller) erlaubt ablationartige Studien (z. B. Variation von Zeitschritt/Reibung, Sensorausfall, Verzögerungen), während die SupervisorSchnittstelle reproduzierbare Kampagnen mit konsistentem Logging unterstützt [4].

4.2 Struktur der Simulationsumgebung

Die verwendete Simulationsumgebung basiert auf Webots und damit auf einer hierarchisch organisierten Szenenbeschreibung im VRML-Format. Eine *.wbt*-Datei kapselt sämtliche Entitäten der virtuellen Welt – von globalen Parametern (Zeitauflösung, Kontaktmodelle) über Umgebungsobjekte (Boden, Hintergrund, Lichtquellen) bis hin zu Robotern mitsamt deren Sensorik, Aktuatorik und Controllern [4, 14]. Jeder Knoten (Node) kann wiederum Unterknoten besitzen, sodass sich komplexe Systeme modular aus standardisierten Bausteinen zusammensetzen lassen. Für die vorliegende Arbeit ist diese Hierarchie entscheidend, weil sie erlaubt, (i) das physikalische Verhalten des Fahrrads getrennt von (ii) der Szenengeometrie und (iii) den übergeordneten Reglerprozessen zu modellieren. Änderungen an einem Teilbereich – etwa an den Reibungsparametern der Reifen oder an der Position einer Kamera – bleiben somit lokal und beeinträchtigen die übrigen Komponenten nicht.

Ausgangspunkt der Welt ist eine rechteckige Bodenarena mit texturierter Fahrbahnoberfläche, die als Referenzstrecke für die visuelle Spurführung dient. Globale Kontakt- und Reibungsparameter sind hier definiert, um das Zusammenspiel zwischen Reifen und Untergrund konsistent zu halten. Ein fester Zeitschritt im Millisekundenbereich stellt sicher, dass sowohl die Balancebewegungen des Fahrrads als auch die Bildverarbeitungsschleifen deterministisch und in Echtzeit simuliert werden können [4]. Die Kameraperspektive des Betrachters (Viewpoint) folgt dem Fahrrad aus erhöhter Position, was die visuelle Analyse der Fahrmanöver während der Versuche erleichtert.

Das zentrale Element der Szene ist der Roboterknoten *BICYCLE*, der das selbstbalancierende Fahrrad repräsentiert. Sein Geometriemodell wurde aus CAD-Daten übernommen und mit vereinfachten Kollisionsvolumina (Boxen und Zylinder) hinterlegt, um die Physiksimulation effizient zu halten. Die Massen- und Trägheitsparameter der Baugruppen (Rahmen, Räder, Gabel) sind explizit angegeben, sodass die Dynamik – insbesondere Roll- und Nickbewegungen – realitätsnah nachgebildet wird. Bewegliche Komponenten sind über Hinge-Joints angebunden: Das Hinterrad wird über einen Rotationsmotor für den Vortrieb angesteuert, die Lenkbewegung des Vorderrads erfolgt über ein zweites Gelenk mit eigenem Servomotor. Ergänzend existiert ein Kurbel-/Pedal-Gelenk, das in erster Linie der geometrischen Vollständigkeit und nicht der Antriebserzeugung dient.

Zur Zustandserfassung sind Sensorschnittstellen integriert: Eine Inertialmesseinheit (IMU) liefert dem Balance-Regler fortlaufend Orientierung und Rollwinkel [2]; eine am Fahrrad montierte Kamera kann genutzt werden, um on-board visuelle Informationen auszuwerten. Ein Display-Modul steht optional für Debug- oder Visualisierungszwecke zur Verfügung. Die Kommunikation zwischen Fahrrad und übergeordnetem Regler erfolgt über ein Paar aus **Emitter** und **Receiver**, die einen leichten, paketbasierten Nachrichtenaustausch innerhalb von Webots ermöglichen.

Übergeordnet ist ein *Supervisor*-Knoten angelegt. Dieser spezielle Robotertyp besitzt in Webots weitreichende Eingriffsmöglichkeiten auf Welt-Ebene (z. B. Position-Reset, Logging, automatisierte Auswertung) und dient hier als externe Beobachter- und Steuerinstanz. Der Supervisor trägt eine statisch in der Szene platzierte Kamera, die die Fahrbahn aus einer globalen Perspektive erfasst. Auf Basis dieser Bilder führt der Python-basierte Vision-Controller eine Spurdetektion (YOLO) durch und berechnet über ein modellprädiktives Regelungsverfahren (MPC) Lenksollwerte bzw. Trajektorien. Diese Kommandos werden per Funkkanal (Emitter) an den Balance-Controller des Fahrrads übertragen, der sie in konkrete Motorbefehle umsetzt. Umgekehrt sendet das Fahrrad Statusdaten zurück, die der Supervisor für Evaluations- und Loggingzwecke speichert. Dieses Zusammenspiel zweier Controller-Prozesse – ein echtzeitnaher PID-Regler in C auf Roboterseite und ein bildverarbeitungsgetriebener MPC-Regler in Python auf Supervisorebene – bildet die Kernstruktur der kombinierten Regelung.

Die gewählte Struktur der Simulationsumgebung verfolgt zwei Ziele: Erstens ermöglicht sie reproduzierbare und sichere Experimente in verschiedensten Fahrszenarien (Geradeausfahrt mit Störimpuls, Slalomkurse, Kurvenfahrten mit Steigung), ohne das reale Hardware-Risiko des Umkippen oder der Beschädigung einzugehen. Zweitens gestattet die klare Trennung von physikalischem Modell, Sensorebene und Reglerlogik eine gezielte Variation einzelner Parameter (z. B. Reibung, Zeitschritt, Bildauflösung), um deren Einfluss auf die Gesamtpformance systematisch zu untersuchen. Webots fungiert damit als flexibles Testbett, in dem sowohl Balance- als auch Spurführungsalgorithmen iterativ entwickelt, validiert und optimiert werden können, bevor sie auf den realen Prototyp übertragen werden. 

4.3 Das Fahrradmodell

Das folgende Kapitel führt in das in Webots implementierte Fahrradmodell ein, wie es in der `.wbt`-Datei spezifiziert ist. Das Modell ist als kinematischer Baum aus starren Körpern (`Solid`) aufgebaut, die über eindimensionale Drehgelenke (`HingeJoint`) verbunden sind. Die sichtbare Geometrie (`CadShape`) ist vom physikrelevanten Kollisionsmodell (`boundingObject` mit einfachen Primitiven) getrennt; Masse, Schwerpunkt und Trägheit sind im `Physics`-Block definiert. An ausgewählten Gelenken wirken `RotationalMotor`-Aktuatoren, während Dämpfung und Haftreibung über `HingeJointParameters` festgelegt sind. In den nachfolgenden Unterabschnitten werden die Baugruppen des Fahrrads entlang der Knotenstruktur der Weltdatei beschrieben und deren Parameter hinsichtlich ihres Einflusses auf die Dynamik (Lenkung, Rad-Boden-Kontakt, Eigendämpfung) erläutert.



Figure 3: Das Fahrradmodell in Webots.

Der Knoten `rear_wheel` ist Teil des Gesamtfahrrads und definiert das angetriebene Hinterrad (`HingeJoint`). Die `HingeJointParameters` beschreiben das Lager des Hinterrads. Der `anchor 0 0.5312 0` gibt die Position der Radachse im Elternkoordinatensystem an. Die `dampingConstant 0.8` ist die viskose Dämpfung des Lagers: höher bedeutet mehr Beruhigung gegen Schwingungen, aber eine trägere Rotation. Die `staticFriction 0.1` ist die Haftreibung im Lager: sie verhindert kleines Flattern nahe Stillstand, erhöht jedoch das zum Anfahren nötige Drehmoment. Das `endPoint Solid` ist der vom Gelenk bewegte starre Körper auf der Kind-Seite des `HingeJoint`. Er rotiert um den am Gelenk definierten Anker/Achse und bildet damit das eigentliche Bauteil (hier: das Hinterrad), das durch die Bewegung betroffen ist. In der Hierarchie gilt: Elternteil → Gelenk → `endPoint Solid`; alles, was hier angehängt ist, bewegt sich mit. Das `boundingObject` definiert die kollisions- und kontaktrelevante Geometrie des Hinterrads, unabhängig vom sichtbaren Mesh. Es bestimmt, wo das Rad den Boden „berührt“, wo Kräfte eingeleitet werden und wie Kollisionen erkannt werden. Im Modell ist es ein einfacher Zylinder, der per `Pose`-Rotation (entspricht Drehung um die y -Achse um 90°) in die Radebene gedreht wird; so bildet er die Kontaktfläche des Reifens ab. Zusammen mit dem `contactMaterial` (z. B. `"wheel"`) entstehen daraus die Reib- und Rückpralleigenschaften im Kontakt mit dem Untergrund. Größe und Ausrichtung des `boundingObject` sollten zur realen Radgeometrie passen, da es die Kontaktlage und damit das Fahrverhalten maßgeblich beeinflusst.

Physik (Physics) Der `Physics`-Block definiert die Masseneigenschaften des Hinterrads. `density -1` bedeutet: Masse und Trägheit werden *nicht* aus dem `boundingObject` berechnet, sondern mit `mass`, `centerOfMass` und `inertiaMatrix` explizit vorgegeben. `mass 2` setzt die Gesamtmasse (kg), `centerOfMass [0 0 0]` positioniert den Schwerpunkt im Solid-Frame (hier an der Nabe), und `inertiaMatrix [0.122 0.122 0.245 0 0 0]` legt die Rotations-Trägheiten ($\text{kg} \cdot \text{m}^2$) fest; größere Werte bedeuten trägere Reaktion auf Dreh- bzw. Kippbewegungen. Die Off-Diagonale `0 0 0` zeigt an, dass Hauptachsen und Solid-Achsen ausgerichtet sind. Für automatische Berechnung könnte statt `density -1` eine positive Dichte verwendet werden (dann kommen Masse/Trägheit aus $\text{Volumen} \times \text{Dichte}$ des `boundingObject`).

Listing 1: Hinterrad—Kinematik, Kontakt und Physik (Auszug)

```

1  DEF rear_wheel HingeJoint {
2      jointParameters HingeJointParameters {
3          anchor 0 0.5312 0
4          dampingConstant 0.1
5          staticFriction 0.8
6      }
7      device [...] # RotationalMotor (siehe Kaptitel 4.5 Aktuatoren)
8      endPoint Solid {
9          children [
10             Transform {...} # visuelles Mesh (ohne Physik)
11         ]
12         contactMaterial "wheel"
13         boundingObject Pose {
14             rotation 0 1 0 1.5708
15             children [ Cylinder { height 0.05 radius 0.352 subdivision 72 } ]
16         }
17         physics Physics {
18             density -1, mass 2, centerOfMass [ 0 0 0 ],
19             inertiaMatrix [ 0.122 0.122 0.245 0 0 0 ]
20         }
21     }
22 }
23

```

4.4 Sensoren

Für die Stabilisierung und Spurführung des selbstbalancierenden Webots-Fahrrads kommen mehrere Sensorsysteme zum Einsatz. Zentral sind eine **Inertialmesseinheit** (IMU) am Fahrzeug zur Lageerfassung und eine **Kamera** zur optischen Fahrwegdetektion. Ergänzend liefern **Motorsensoren** (Encoder) an den Achsen Messgrößen wie Lenk- und Raddrehwinkel. Die IMU versorgt den inneren schnellen Regelkreis (Balance-PID) laufend mit dem aktuellen Rollwinkel, während die Kamera den äußeren, langsamer abtastenden Regelkreis (vision-basiertes MPC) mit Spurabweichungsinformationen speist. Beide Sensorströme werden anschließend in einfacher Weise fusioniert, um die Regelgrößen an die Aktuatoren (Lenkmotor und Antrieb) auszugeben [GitHubGitHub](#). Im Folgenden werden die einzelnen Sensoren hinsichtlich physikalischer Funktion, Modellierung in Webots, Rolle im Regelkreis und Integration im Quellcode beschrieben.

4.4.1 Inertialmesseinheit (IMU)

Physikalisches Prinzip: Eine Inertial Measurement Unit (IMU) erfasst Trägheitsgrößen des Fahrzeugs, typischerweise mit kombinierter **Beschleunigungsmessung** (Accelerometer) und **Drehratensensoren** (Gyroskope) auf drei Raumachsen [24]. Durch Fusion dieser Signale – oft ergänzt um einen Magnetometer-Kompass – lässt sich die Orientierung im Raum bestimmen. Im realen Prototyp kommt z.B. der IMU-Sensor **BNO055** zum Einsatz, der intern die Rohdaten filtert und direkt **Euler-Winkel** (Roll, Pitch, Yaw) bereitstellt [2]. Die Rollachse (Seitenneigung) ist beim Fahrrad die entscheidende Stabilitätsgröße. Reale IMUs liefern diese Winkel mit begrenzter Auflösung und unterliegen Rauschen sowie Drift, was meist durch Kalibrierung und Sensorfusion kompensiert werden muss. Die Ausgaberate moderner IMUs liegt im Bereich mehrerer hundert Hertz (der BNO055 etwa ~ 200 Hz [2]), was für eine schnelle Lagekontrolle ausreicht.

IMU-Modell in Webots: Webots stellt mit dem `InertialUnit`-Knoten ein abstraktes IMU-Modell bereit, das idealisierte Orientierungsdaten liefert. Konkret berechnet und aktualisiert das `InertialUnit`-Gerät in jeder Simulationsschleife den **Kipp- und Gierwinkel** des Roboters relativ zu einem globalen Bezugssystem [24]. Im Gegensatz zu einer realen IMU entfallen hier Kalibrierfehler und Rauschen – die Simulation liefert die Orientierungswinkel also praktisch **fehlerfrei und ohne Drift**. Das im Webots-Weltmodell definierte Gerät `imu` ist starr im Fahrradrahmen verbaut (an geeigneter Position, etwa nahe dem Schwerpunkt) und gibt standardmäßig Roll-, Pitch- und Yaw-Winkel in Radiant aus. Die Abtastung erfolgt synchron mit dem physikalischen Simulationsschritt. Im vorliegenden Modell ist der `basicTimeStep` der Welt auf 2 ms gesetzt [13], was einer Messfrequenz von 500 Hz entspricht. Dadurch können selbst schnelle Kippbewegungen des Fahrrads in Echtzeit erfasst werden [13].

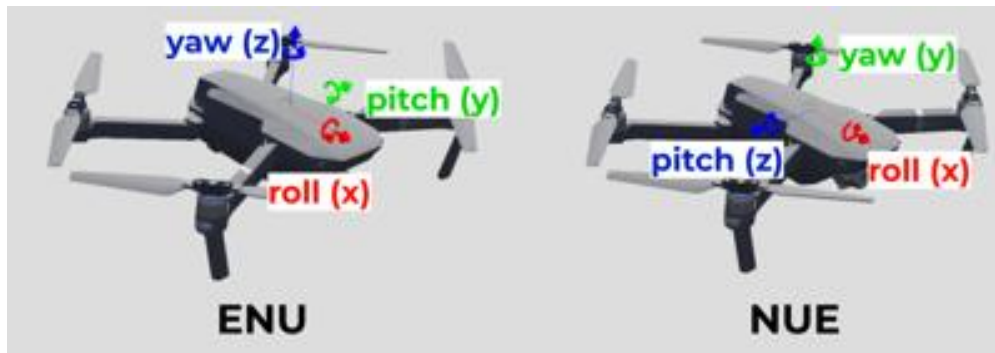


Figure 4: IMU-Knoten in Webots.

Rolle im Regelkreis: Der IMU-Sensor bildet die zentrale **Rückführung** für die **Balance-Regelung**. In jedem Simulationsschritt liest der PID-Regler den aktuellen Rollwinkel φ_{roll} aus der IMU und berechnet daraus die Stellgröße für den Lenkantrieb. Konkret fungiert der Rollwinkel als *Regelgröße* im inneren Regelkreis: weicht das Fahrrad nach links/rechts ab, soll durch einen entsprechenden Lenkeinschlag gegengesteuert werden, bis $\varphi_{\text{roll}} = 0$ (senkrechte Lage) erreicht ist. Die hohe Abtastrate der IMU erlaubt es, den **Regelkreis mit einer Zykluszeit von 2 ms** praktisch verzögerungsfrei zu schließen. Dadurch reagiert der PID-Regler sehr schnell auf Lageänderungen, was eine Grundvoraussetzung ist, um das instabile überhaupt beherrschen zu können. Im äußeren (vision-basierten) Regelkreis spielt die IMU direkt keine Rolle, allerdings fließen die Balancierinformationen indirekt ein, wenn die Vision-Steuerung, z. B. die Fahrgeschwindigkeit reduziert, sobald der Stabilitätsfaktor sinkt (starke Schräglage detektiert). Insgesamt sorgt die IMU dafür, dass die **Regelstrecke** mit hoher Güte beobachtet wird, und bildet so das Fundament der Stabilisierung.

Im C-Controller `balance_control.c` wird die IMU über die Webots-API initialisiert und ausgelesen. Listing 1 zeigt auszugsweise, wie das Gerät referenziert, aktiviert und genutzt wird. Zunächst wird der Device-Tag der IMU mit ihrem Namen `"imu"` angefordert und das Sensor-Sampling mit dem Simulationstimestep (2 ms) eingeschaltet. In der Hauptschleife ruft der Regler dann kontinuierlich die Funktion `wb_inertial_unit_get_roll_pitch_yaw()` auf, welche einen Vektor der Winkel zurückliefert. Der Rollwinkel (Index 0 des Arrays) wird extrahiert und in die Regelung eingespeist. Da die Webots-IMU per se idealisiert ist, entspricht dieser Winkel exakt der aktuellen Fahrzeugneigung ohne Messfehler. Im Code ist dennoch ein Fallback vorgesehen: Falls der Supervisor-Knoten verfügbar ist, wird alternativ dessen Orientierungsmatrix ausgewertet – dies diene experimentell der Verifikation der IMU-Daten und kann die gleiche Information liefern. Im Normalfall nutzt der

PID jedoch direkt die IMU-Daten. Der exemplarische Code zeigt, wie einfach die Orientierung in Webots abgefragt werden kann, verglichen mit einer realen IMU (wo man per I²C erst Rohdaten lesen und konvertieren müsste). Die geringe Latenz der `wb_inertial_unit_get_roll_pitch_yaw`-Funktion (nur ein Zeiger auf interne Simulationsdaten) trägt dazu bei, dass der Balance-Regelkreis praktisch ohne zusätzliche Verzögerung arbeitet.

Listing 2: IMU-Initialisierung und -Abfrage im C-Controller (Balance-Regelung)

```

1 // IMU-Geraet initialisieren (in init_devices)
2 imu_sensor = wb_robot_get_device("imu");
3 wb_inertial_unit_enable(imu_sensor, timestep); // Abtastrate = Welt-Zeitschritt (2 ms)
4
5 // ... (im Hauptprogramm der Balance-Regelung)
6
7 // Roll-, Pitch- und Gierwinkel vom IMU-Sensor abrufen
8 const double *rpy = wb_inertial_unit_get_roll_pitch_yaw(imu_sensor);
9 double roll_angle = rpy[0]; // aktueller Rollwinkel [Bogenmass]
10

```

4.4.2 Kamera (Videosensor)

Physikalisches Prinzip: Die Kamera dient als **optischer Sensor**, um die Strecke vor dem Fahrrad zu erkennen. Physikalisch basiert sie auf einem *digitalen Bildaufnehmer* (z.B. CMOS-Sensor), der durch eine Linse abgebildetes Licht in Pixelwerte umwandelt. In einem realen System würde eine am Fahrzeug montierte Weitwinkel-Kamera laufend **Videobilder** der Fahrbahn liefern. Wesentliche Parameter sind die **Auflösung** (Anzahl Pixel, bestimmt Detailgenauigkeit) und die **Bildrate** (Bilder pro Sekunde, bestimmt zeitliche Auflösung). Für eine stabile Spurhaltung in Echtzeitanwendungen genügen in der Regel zweistellige Bildfrequenzen ($\sim 10\text{--}30\text{ Hz}$), sofern das Fahrzeug nicht extrem schnell fährt. Die Kamera erfasst visuelle Merkmale der Umgebung – im vorliegenden Kontext insbesondere die **Straßenmarkierung** bzw. den Verlauf der Fahrspur – die dann von Bildverarbeitungsalgorithmen ausgewertet werden. Physikalisch können **Umgebungs-faktoren** wie Beleuchtung, Bewegungsunschärfe oder Linsenverzerrungen die Bildqualität beeinflussen, was eine robuste **Merkmalsextraktion** herausfordernd macht. Im Gegensatz dazu operiert eine Kamera jedoch **kontaktlos** und kann reichhaltige Umgebungsinformationen liefern, was sie ideal für die **Pfadführung** macht (im Vergleich zu z.B. lokalen Sensorsignalen eines Gyroskops).

Simulation in Webots: Webots simuliert Kameras über den **Camera-Node**, der virtuelle Renderings der 3D-Szene erzeugt. Im *Weltfile* ist für den Supervisor (die übergeordnete Instanz) eine Kamera mit definierter Position, Ausrichtung und Eigenschaften eingebettet. Diese sogenannte *Vision-Kamera* blickt aus erhöhter Perspektive nach vorne unten auf die Fahrbahn (vergleichbar mit einer Helm- oder Drohnenkamera, die dem Fahrrad folgt). Die in der Simulation gewählte Auflösung beträgt z.B. 640×360 Pixel bei einem Gesichtsfeld von ca. 114° (Field of View $\approx 2.0\text{ rad}$). Diese Einstellungen sind ein Kompromiss zwischen Sichtfeld und Bilddetail – ein breites Sichtfeld erfasst die S-Kurve vollständig, während die moderate Auflösung die Datenmenge und Rechenzeit begrenzt. Die Kamera liefert pro aktiviertem Zeitschritt ein Bild im **RGBA-Format** (Rot–Grün–Blau plus Alphakanal) als Array von Byte-Werten. In der Controller-Software muss die Kamera explizit aktiviert werden, wobei die Abtastrate frei wählbar ist. Der Vision-Controller nutzt eine Bildrate von rund 20 Hz, entsprechend einem Abtastintervall von 50 ms. Dies ist deutlich langsamer als der Balance-Regelkreis, spiegelt aber die realistische Anforderung wider, dass Bildverarbeitung

zeitaufwändiger ist und nicht in jedem Simulationsschritt erfolgen muss. Im Webots-Code wird die Kamera z. B. mit `camera.enable(50)` konfiguriert, sodass alle 50 ms ein neues Frame bereitgestellt wird. Jedes Frame kann vom Controller über `camera.getImage()` abgeholt und weiterverarbeitet werden. Zu beachten ist, dass Webots standardmäßig keine Kameraräusche oder Sensorschwächen simuliert – die gelieferten Bilder sind idealisiert (scharf, bei gleichbleibender Beleuchtung). Um Realismus zu erhöhen, könnten optional Rauschen oder Unschärfe simuliert werden; im gegebenen Modell wird jedoch auf eine idealisierte Spurenerkennung abgezielt.

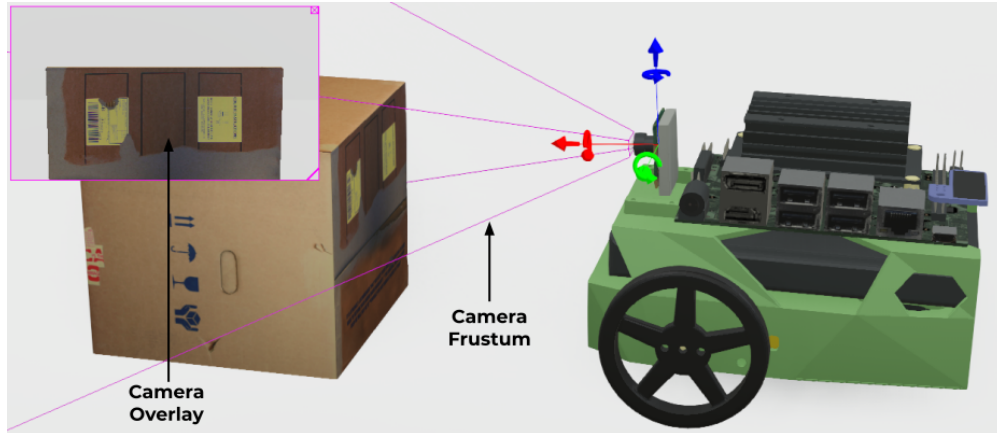


Figure 5: Kamera-Knoten in Webots.

Rolle im Regelkreis: Die Kamera ist der **Schlüsseingang** für den vision-basierten äußeren Regelkreis. Ihre Aufgabe besteht darin, dem **MPC-Fahrtrichtungsregler** Informationen über den Verlauf der Fahrspur relativ zum Fahrzeug zu liefern. Im konkreten Szenario der S-Kurve extrahiert der Vision-Controller aus jedem Kamerabild den lateralen Positionsfehler e der Straße: also wie weit und in welche Richtung das Fahrrad von der gewünschten Spurmitte abweicht.

Diese Bildverarbeitung geschieht in mehreren Schritten:

1. **Merkmalsextraktion:** Zunächst werden relevante Bildmerkmale berechnet, etwa mittels **Segmentierung** der Straße. Im Projekt wurde hierzu ein vortrainiertes YOLO-Modell verwendet, das die Fahrbahn in der Vogelperspektive segmentiert. Falls kein KI-Modell verfügbar ist, greift ein Fallback auf klassische Bildverarbeitung: Kantenerkennung und Auswertung eines definierten *Region-of-Interest* liefern ebenfalls einen Schätzwert für die Spurmittellage. Ergebnis dieses Schritts ist der Querfehler e (in normierter Form, z. B. $-0.5 \dots + 0.5$ relativ zur Bildmitte) sowie ggf. zusätzliche Qualitätskennzahlen (etwa die erkannte Kontur oder den Bedeckungsgrad der Fahrbahnmaske).
2. **Spurführungsregelung:** Auf Basis des visuellen Fehlers bestimmt der Controller einen passenden **Lenksollwert**. Im entwickelten System kommt hierfür vorzugsweise ein **modell-prädiktiver Regler (MPC)** zum Einsatz, der das Fahrrad als dynamisches Modell (Zustand $\mathbf{x} = [x, y, v, \psi]$) vorausberechnet. Der MPC optimiert die Steuergröße Lenkwinkel δ (und ggf. Beschleunigung) über einen kurzen Zeithorizont so, dass der Querfehler minimiert und Fahrdynamikgrenzen (Lenkwinkel- und -ratenbegrenzung, Komfort) eingehalten werden.

Falls der MPC (etwa mangels Rechenpower) nicht verfügbar ist, kann ersatzweise ein einfacherer PID-Ansatz verwendet werden („*Vision-PID*“), der den Fehler e in einen Lenkwinkel

übersetzt. In beiden Fällen resultiert ein Soll-Lenkswinkel δ_{sol} bzw. ein normierter Lenkbefehl $u_{\text{vis}} \in [-1, 1]$, der an den Balance-Controller gesendet wird.

Zusätzlich kann die Vision-Einheit eine **Geschwindigkeitsempfehlung** v_w berechnen – z. B. wird bei großem Kurvenfehler eine niedrigere Fahrgeschwindigkeit empfohlen, um die Stabilität zu wahren.



Figure 6: Querfehler e der Straße.

Die Kamera beeinflusst den Balance-Regler auch indirekt, indem deren Messdaten den **Kurs der Fahrt** vorgeben. Wichtig ist, dass die Vision-Daten mit deutlich geringerer Frequenz ankommen als die IMU-Daten. Der Balance-Controller hält während der 50 ms zwischen zwei Kamerabildern die zuletzt empfangene Soll-Lenkvorgabe bei und stabilisiert weiterhin autonom. Sobald ein neues Bild ausgewertet ist, wird der Lenksollwert ggf. korrigiert. Dieses **zweistufige Regelungskonzept** (innere IMU-Schleife, äußere Vision-Schleife) sorgt dafür, dass das Fahrrad einerseits aufrecht bleibt, andererseits aber auch aktiv Richtungskorrekturen vornimmt, um einer kurvigen Referenzstrecke zu folgen. Ohne die Kamera wäre nur ein statisches Balancieren oder eine Geradeausfahrt möglich – erst der optische Sensor ermöglicht die *spurtreue Autonomie* des Modells.

4.4.3 Motorsensoren (Inkrementalgeber)

Physikalisches Prinzip: Neben IMU und Kamera, die den *Zustand* des Fahrzeugs in Bezug auf Lage und Umwelt erfassen, spielen sogenannte **Motorsensoren** eine wichtige Rolle für die interne Zustandsmessung. Gemeint sind hier **Winkelgeber (Encoder)** an den drehbaren Komponenten – konkret am Lenkmotor und am Antriebsrad. Physikalisch handelt es sich meist um *inkrementelle Drehgeber*, die auf der Motorachse montiert sind und pro Umdrehung eine bestimmte Anzahl Impulse (Zählschritte) liefern. Durch Mitzählen der Impulse kann der Steuerrechner den zurückgelegten Winkel bestimmen; über die Impulsfrequenz lässt sich auch die Drehgeschwindigkeit ermitteln. In echten Systemen werden beispielsweise **optische Encoder** (eine Scheibe mit Strichmuster und Fotodetektor) oder **Hall-Sensoren** (Magnet und Hallsensor für Umdrehungszählung) eingesetzt. Im Kontext unseres selbstbalancierenden Fahrrads wird der Lenkwinkel-Encoder benötigt,

um den aktuellen **Lenkeinschlag** zu kennen – wichtig etwa, um Lenkwinkelgrenzen nicht zu überschreiten und für die protokollierte Ausgabe. Der Hinterrad-Encoder erfasst die **Raddrehstellung** bzw. via Differenz auch die momentane Raddrehzahl, was zur Abschätzung der Fahrgeschwindigkeit dient. Physikalisch sind solche Sensoren sehr präzise (hochaufgelöste Encoder bieten > 1000 Impulse/Umdrehung) und liefern in schneller Abfolge Signale, typischerweise synchron zum Motor (Erfassung erfolgt im kHz-Bereich, Latenzen sind vernachlässigbar). Allerdings muss die Ansteuerungselektronik die Impulse auswerten (Interrupts zählen etc.), was in der realen Implementierung zusätzlichen Programmieraufwand bedeutet.

Rolle im Regelkreis: Die Positionssensoren werden vor allem zur **Überwachung und für abgeleitete Regelungsgrößen** genutzt. Der **Lenkwinkelsensor** gibt dem Balance-Controller Feedback, welchen Lenkeinschlag der Motor aktuell realisiert – dies ist nützlich, um z. B. einen *Sättigungseffekt* zu erkennen: Wenn der Soll-Lenk Winkel die mechanische Grenze erreicht, kann der Regler keine weitere Korrektur mehr bewirken. Im Code wird aus dem Verhältnis von aktuellem Lenkausschlag zu Maximalwinkel ein **Stabilitätsfaktor** berechnet, der als Indikator dient. Außerdem fließt der gemessene Lenkwinkel in die Logging-Daten ein, um das Reglerverhalten nachträglich analysieren zu können. Im laufenden Regler steckt der Lenkwert implizit schon in der Dynamik: Da der Lenkmotor in Webots im Positionsmodus betrieben wird, sorgt das System automatisch dafür, dass der Ist-Winkel dem Soll-Winkel folgt (siehe Abschnitt *Aktuatoren*). Ein separater Positionsregelkreis ist im Code also nicht implementiert – im realen System würde hier ein unterlagerter Motor-PID am Encoder hängen, aber in der Simulation übernimmt dies der Motorendpunkt.

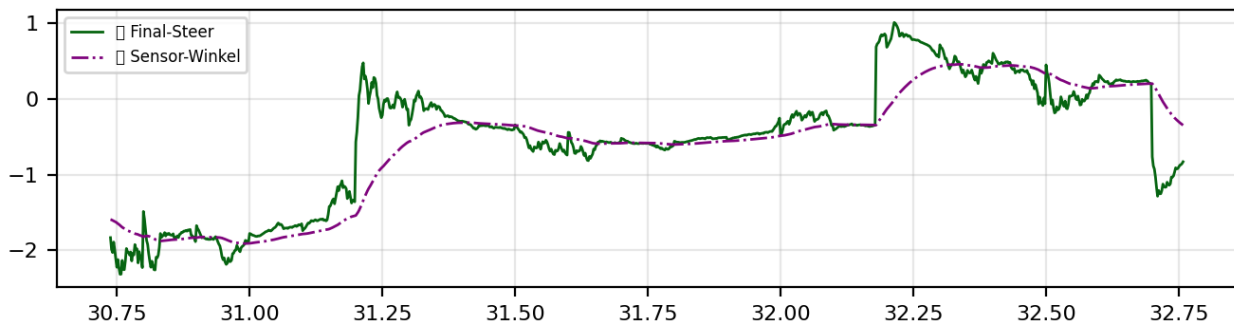


Figure 7: Sensor- und Aktuator-API in Webots.

Der **Raddrehwinkelsensor** am Hinterrad wird primär genutzt, um die **Fahrgeschwindigkeit** zu ermitteln. Im Controller-Code wird aus der zeitlichen Differenz des Radwinkels $\Delta\theta/\Delta t$ die Winkelgeschwindigkeit ω berechnet und in km/h umgerechnet. Diese Information kann für verschiedene Zwecke dienen: einerseits zur **Ausgabe** (Anzeige der aktuellen Geschwindigkeit), andererseits könnte sie in einen Geschwindigkeitsregler zurückwirken. Tatsächlich wird im Balance-Controller ein einfacher Speed-Controller benutzt, der allerdings nicht die Ist-Geschwindigkeit, sondern den Vision-Lenkbefehl als Stellgröße nutzt (d. h. bei starkem Lenken wird die Sollgeschwindigkeit reduziert). Eine echte geschlossene Geschwindigkeitsregelung (Fahrgeschwindigkeit konstant halten) wäre ebenfalls denkbar – hierzu würde der Rad-Encoder den Ist-Wert für einen Geschwindigkeits-PID liefern, ähnlich wie im Hardware-Design mit Hall-Sensor beschrieben. Im aktuellen Simulationsmodell beschränkt sich die Nutzung also darauf, **Zustandsgrößen** zu erfassen und die Regelung zu überwachen, weniger auf direkte Korrektur. Dennoch sind die Encoder essenziell, um konsistente Zustandsinformationen zu haben: z. B. ermöglicht der Radencoder dem System zu erkennen, ob das Fahrrad fährt oder fast steht, was für die Balance eine Rolle spielen kann (bei zu niedriger Geschwindigkeit nimmt die Selbststabilisierung ab).

4.5 Aktuatoren

Das selbstbalancierende Fahrradmodell verfügt über zwei zentrale **Aktuatoren**, die als Stellglieder in die Dynamik des Systems eingreifen: einen **Lenkantrieb** an der Vorderradgabel und einen **Antriebsmotor** für das Hinterrad. Beide sind in Webots als **RotationalMotor**-Geräte realisiert und jeweils an einem drehbaren HingeJoint angebracht. Physikalisch handelt es sich um **elektrische Rotationsmotoren** (Gleichstrommotoren mit Getriebe), die in der Lage sind, Drehmomente aufzubringen und so entweder einen bestimmten Winkel (Servomodus) oder eine bestimmte Drehgeschwindigkeit einzustellen. Im Folgenden werden zunächst die realen Grundlagen dieser Aktuatoren betrachtet, dann ihre Umsetzung in der Simulation und schließlich ihr konkreter Einsatz im Regelungssystem mitsamt Code-Beispielen erläutert.

Physikalisches Wirkprinzip:

Bei **bürstenbehafteten DC-Motoren**, wie sie in vielen Robotikanwendungen Verwendung finden, erzeugt ein stromdurchflossener Anker in einem Magnetfeld ein Drehmoment (Lorentz-Kraftwirkung auf die Leiter). Das resultierende **Motordrehmoment** ist annähernd proportional zum Ankerstrom, während die Leerlauf-Drehzahl von der angelegten Spannung bestimmt wird. Im offenen Regelkreis würde ein DC-Motor also schneller drehen bei höherer Spannung und stärker drehen (mehr Kraft) bei höherem Strom.

Um präzise **Stellbewegungen** auszuführen, werden Motoren oft mit **Getrieben** (zur Erhöhung von Drehmoment und Auflösung) und mit **Sensoren** rückgekoppelt (Encodern, s. o.). So entsteht ein **Servoantrieb**: ein Motor-Getriebe-System mit interner Positions- oder Drehzahlregelung. Im realen selbstbalancierenden Fahrrad gibt es zwei solcher Antriebe:

- **Der Lenkmotor** (z. B. ein Getriebemotor mit MD49-Motorcontroller) verstellt den Lenkerwinkel. Er arbeitet im Prinzip als **Positionsantrieb**: auf einen Sollwinkel hin geregelt, mit einem begrenzten Arbeitsbereich (hier ca. $\pm 30^\circ$ mechanisch). Der Motorcontroller nutzt den Encoder am Lenkgetriebe, um den Winkel exakt einzustellen (geschlossener Regelkreis). Physikalisch bedeutet dies, dass bei Abweichung zwischen Soll- und Ist-Winkel ein proportional-integral-derivativer Korrekturstrom ins Motortreiber-Modul gegeben wird, bis der Lenker die gewünschte Position erreicht. Der Lenkwinkelantrieb muss schnell und kräftig genug sein, um das Fahrrad innerhalb von Sekundenbruchteilen stabilisieren zu können – in Jonah Zanders Hardware-Aufbau wurde z. B. ein starker 12 V-Motor mit Untersetzung verwendet, der $\pm 150^\circ$ in kurzer Zeit durchfahren kann.
- **Der Antriebsmotor** am Hinterrad (z. B. ein Nabenmotor oder kettengetriebener DC-Motor) sorgt für den Vortrieb. Dieser wird im Unterschied zum Lenker meist als **Drehzahl- bzw. Drehmomentantrieb** betrieben. Im realen System regelt man hier oft die Geschwindigkeit: Ein separater PID hält die Raddrehzahl konstant, indem er den Motorstrom anhand des Hall-Encoder-Feedbacks justiert. Physikalisch verleiht das rotierende Hinterrad dem System einen **gyroskopischen Effekt**, der stabilisierend wirkt, weshalb eine gewisse Grundgeschwindigkeit hilfreich ist. Gleichzeitig muss der Motor aber auch schnell drosseln oder bremsen können, um Kurven sicher zu durchfahren, ohne umzukippen.

Webots-Umsetzung: Webots stellt mit dem `RotationalMotor`-Node einen Aktuator zur Verfügung, der an einen `HingeJoint` gekoppelt Drehmomente ausüben kann. Im Hintergrund berechnet der Physik-Kernel (ODE) die Wirkung des Motors auf das Gelenk unter Berücksichtigung von Maximalmoment und Zielvorgabe. Jeder Rotationsmotor kann in drei Modi betrieben werden:

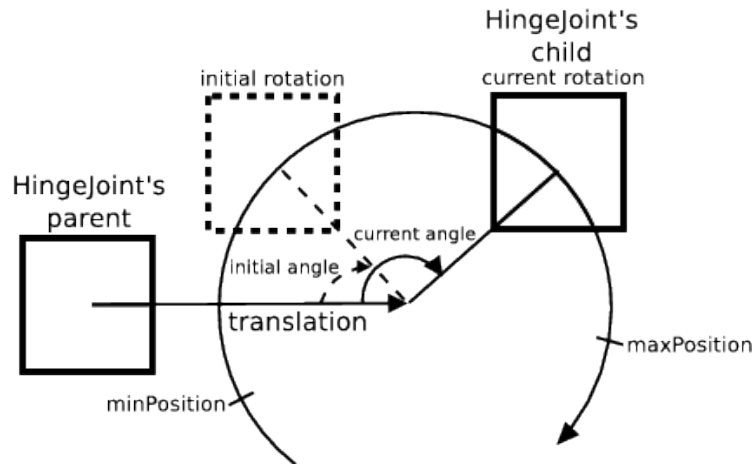


Figure 8: Aktuatoren im Fahrradmodell.

- **Positionsmodus:**

Der Motor strebt einen vorgegebenen Zielwinkel an (`setPosition(angle)`), wobei er innerhalb der konfigurierten Kraft- und Geschwindigkeitsgrenzen das nötige Drehmoment erzeugt, um diesen Winkel zu erreichen. Dieser Modus entspricht einem Servomechanismus. Im Webots-Modell ist der Lenkungsaktor in den Positionsmodus geschaltet. Dazu wird im Code einmalig `wb_motor_set_position(handlebars_motor, 0.0)` aufgerufen, um die Start-Sollposition (geradeaus) zu setzen. Anschließend erhält der Lenkmotor in jeder Regeliteration einen neuen Sollwinkel. Die Regelung der Position übernimmt Webots intern: der Motor wirkt wie eine Feder-Dämpfer-Kopplung ans Gelenk, die Differenz zwischen Ist- und Sollwinkel generiert ein entsprechendes Gegendrehmoment (virtueller P-Regler). Parameter wie `maxTorque` und `maxVelocity` im Weltfile bestimmen, wie schnell und kräftig das Soll nachgeführt wird. Im Modell sind z.B. für den Lenkmotor `maxTorque = 25Nm` festgelegt – ein Wert, der sicherstellt, dass genügend Kraft zum Aufrichten aus Schräglagen vorhanden ist, ohne das System zu unrealistisch zu machen.

- **Geschwindigkeitsmodus:**

Hier wird dem Motor eine Ziel-Angulargeschwindigkeit vorgegeben (`setVelocity(omega)`), während die Position auf unendlich gesetzt wird (`setPosition(INFINITY)`). In Webots bedeutet dies, dass der Motor keinen festen Endwinkel hat, sondern kontinuierlich dreht. Der Antriebsmotor des Hinterrads wird so betrieben: im Initialisierungscode wird `wb_motor_set_position(wheel_motor, INFINITY)` aufgerufen, was den Wechsel in den Geschwindigkeitsmodus bewirkt. Fortan interpretiert Webots den Befehl `wb_motor_set_velocity(wheel_motor, v)` als Soll-Drehzahl (in Rad/s) für das Rad. Intern wirkt wieder ein virtueller Regler, der das nötige Drehmoment aufbringt, um diese Geschwindigkeit zu erreichen/halten (bis zum max. Drehmoment). Im Weltmodell ist für den Hinterradmotor ein recht hohes maximales Drehmoment von 120 Nm erlaubt, um Beschleunigungsvorgaben schnell umsetzen zu können. Allerdings ist die erreichbare Drehzahl in der Praxis auch durch die Antriebsspannung bzw. Webots-Parameter

`maxVelocity` limitiert. In S-Kurven-Simulationen wurde typischerweise eine Grundgeschwindigkeit von ca. 3–5 m/s gefahren, was durch den entsprechenden `velocity`-Wert in Rad/s vorgegeben wird.

- **Kraft-/Drehmomentmodus:** Alternativ kann man in Webots Motoren auch direkt ein Drehmoment vorgeben (`setTorque(t)`), um z. B. eigene Regelalgorithmen für die Positions- und Geschwindigkeitssteuerung umzusetzen. In unserem Modell wird dieser Modus nicht explizit genutzt, da die beiden in Steuerungsmodellen gängigen Modi bereits ausreichen. Dennoch ist es wichtig zu wissen, dass Webots stets mit Kräften/Momenten rechnet: Die Position- und Velocity-Modelle setzen letztlich ein solches Drehmoment um, das aus der jeweiligen Referenzgröße berechnet wird.

Die beiden Motoren sind im Weltfile als Geräte `handlebars motor` und `motor::wheel` deklariert und jeweils dem passenden HingeJoint zugeordnet. Webots erlaubt es, mehrere Motoren pro Gelenk zu definieren, doch hier ist jeweils nur einer aktiv.

Die Namensgebung `motor::wheel` für den Hinterradantrieb ist ein Spezialfall: Das Präfix `::` dient dazu, den Motor vom gleichnamigen (per DEF) vorderen Rad zu unterscheiden, welches ebenfalls als `wheel` benannt ist – so können Kollisionen in der Namensauflösung vermieden werden. Im Controller-Code greift man dann genau mit diesem Namen zu.

Rolle im Regelkreis

Die Aktuatoren setzen die vom Regler berechneten Sollgrößen in physikalische Bewegung um und schließen damit die **Wirkkette des Regelkreises**. Konkret berechnet der Balance-Controller in jedem Zeitschritt zwei Stellwerte:

- den **Lenksollwinkel** δ_{sol} (aus dem PID-Regler + Vision-Überlagerung), und
- die **Antriebs-Sollgeschwindigkeit** v_{sol} .

Diese Sollwerte werden direkt an die Motoren übergeben. Der Lenkwinkel-Sollwert stammt aus der Kombination des inneren und äußeren Regelkreises: Wie in Abschnitt *Sensoren* erläutert, fusioniert der Controller den **Balance-Anteil** (zur Selbstausrichtung) mit dem **Vision-Anteil** (zur Kurvenfahrt) zu einer endgültigen Vorgabe `final_steer`. Dieses `final_steer` entspricht einem Winkel in Radian, der begrenzt auf den physikalisch möglichen Bereich (± 0.524 rad) an den Lenkmotor geschickt wird.

Der Hinterrad-Sollwert `target_speed` wird ebenfalls bestimmt – im aktuellen Konzept abhängig vom Lenkeinschlag (starker Lenkeinschlag $\rightarrow$ automatische Reduktion der Grundgeschwindigkeit). Damit fungiert der Balance-Controller gleichzeitig als einfacher **Geschwindigkeitsmanager**, der z. B. in Kurven abbremst, um die Balance nicht zu gefährden. Die Berechnung von `target_speed` erfolgt durch einen separaten Algorithmus (`speed_pid_compute`), der den Vision-Lenkbefehl in eine prozentuale Reduktion der Grundgeschwindigkeit umsetzt. Alternativ könnte hier ein echtes Geschwindigkeits-PID auf Basis der Encoder-Daten stehen – dies ist eine mögliche Erweiterung.

Sobald die Stellgrößen vorliegen, werden sie an die Motor-API übergeben. Damit findet die **Regler-Ausgabe** statt: Die Software schreibt die Sollwerte in die Motoren, und die Physiksimulation reagiert, indem sie die entsprechenden Kräfte auf das Modell einleitet.

Der **Lenkmotor** dreht die Vorderradgabel proportional zum Regelfehler – wenn das Fahrrad z. B. nach rechts kippt (positiver Rollwinkel), verlangt der PID einen Lenkeinschlag nach rechts, um einen stabilisierenden Lenkimpuls zu erzeugen (das Rad wird unter den Schwerpunkt gesteuert, ähnlich wie man einen fallenden Besen durch Vorziehen stabilisiert). Gleichzeitig berücksichtigt der Vision-Anteil, ob ggf. bewusst in eine Kurve gelenkt werden soll. Die resultierende Motorbewegung ist somit eine Überlagerung aus Stabilisieren und Navigieren.

Der **Hinterradmotor** liefert währenddessen den Vortrieb: Eine Grundgeschwindigkeit hält das Fahrrad in Bewegung (was das Balancieren erleichtert, da dynamische Stabilisierungskräfte wirken), und bei Bedarf wird verlangsamt. Im Geradeausfall (Vision inaktiv) regelt der Balance-Controller sogar nur die Balance und reduziert Geschwindigkeit auf ein Mindestmaß, falls keine Steuerkommandos kommen.

Insgesamt sorgen die Aktuatoren dafür, dass die vom Regler vorgegebenen Stellgrößen (Lenkwinkel, Antriebskraft) tatsächlich umgesetzt werden – sie sind somit die finalen Ausführungselemente des Reglers. Ohne ausreichende Aktuatorleistung könnte das beste Regelgesetz das Fahrrad nicht stabilisieren; in der Simulation sind sie jedoch so gewählt, dass die Anforderungen erfüllt werden.

Implementierung und Code-Integration: Die Ansteuerung der Motoren erfolgt im Balance-Controller (C-Code) nach Berechnung der Sollwerte. Bereits während der Initialisierung werden die Motoren konfiguriert: Listing 4 zeigt, wie der Lenkmotor und der Radmotor als Devices geholt und in die gewünschten Modi versetzt werden. Für den Lenkantrieb wird die Position initial auf 0.0 gestellt (*Lenker gerade*). Für den Hinterradantrieb wird `wb_motor_set_position(wheel_motor, INFINITY)` gesetzt, was – wie oben beschrieben – den Geschwindigkeitsmodus aktiviert. Beide Motoren würden standardmäßig mit `velocity = 0` starten; der Hinterradmotor erhält später im laufenden Betrieb eine Startgeschwindigkeit (Basiswert).

Im Regelprozess selbst (Hauptschleife) werden dann die aktuellen Stellbefehle übergeben:

Listing 3: Motorsteuerung im Regelkreis

```
wb_motor_set_position(handlebars_motor, -final_steer);  
wb_motor_set_velocity(wheel_motor, target_speed);
```

Dabei ist wichtig zu erwähnen, dass der Lenkwinkel mit negativem Vorzeichen gesetzt wird. Dies rührt von der Achsdefinition her – je nach Koordinatensystem ist eine positive Rotation der Lenkachse evtl. als Linkseinschlag definiert, während der Regler positive Werte als Rechtsdrehung interpretiert (oder umgekehrt). Durch das Vorzeichen wird sichergestellt, dass z. B. ein positiver Reglerausgang (Korrektur nach rechts) tatsächlich den Lenker nach rechts steuert. Solche Vorzeichenkonventionen sind in der Modellierung zu beachten.

Der Wert `target_speed` wird in Rad/Sekunde an das Hinterrad übergeben. Um einen Eindruck zu geben: Eine Zielgeschwindigkeit von z. B. 4 m/s entspricht (bei ~ 0.7 m Raddurchmesser) ungefähr 11.4 rad/s Winkelgeschwindigkeit.

Das Logging im Code zeigt typischerweise Werte um 10–15 rad/s für die Grundgeschwindigkeit. Nachdem die Sollwerte gesetzt sind, greift die Webots-Physik die neuen Sollwerte im nächsten Zeitschritt auf und beschleunigt/verstellt die Motoren entsprechend. Im Code werden nach dem Setzen der Motoren noch Statuspakete versendet und Logging betrieben, was für die Aktuatorwirkung selbst aber keine Rolle mehr spielt.

Insgesamt ist die Motoransteuerung in Webots sehr direkt – ein Funktionsaufruf genügt, um das physikalische Stellglied zu beeinflussen. Dies erlaubt es, die Regelalgorithmen unkompliziert mit der

Simulationswelt zu koppeln. Listing 4 fasst die wichtigsten Schritte zusammen. Hier sind sowohl die Initialisierung (Modussetzen) als auch die Laufzeitschleifen der Motoren zu sehen, mit deutschen Kommentaren erläutert. Man erkennt, wie die Aktuator-Schnittstelle an die zuvor beschriebenen Regelungsgrößen gebunden ist: `final_steer` (in rad) geht an den Lenkmotor, `target_speed` (in rad/s) an den Radmotor. Diese Werte stammen direkt aus der Regelung (siehe vorherigen Abschnitt) – es findet keine weitere Transformation mehr statt, da die Größeneinheiten konsistent mit dem Webots-Modell gewählt wurden.

```

1 // Motoren-Devices abrufen
2 handlebars_motor = wb_robot_get_device("motor::handlebars");
3 wheel_motor      = wb_robot_get_device("motor::wheel");
4 if (handlebars_motor == 0 || wheel_motor == 0) {
5     exit(1);
6 }
7 // Initialisierung der Motoren
8 // Lenkmotor: Positionsmodus, Startwinkel = 0 rad
9 wb_motor_set_position(handlebars_motor, 0.0);
10 // Hinterradmotor: Geschwindigkeitsmodus aktivieren
11 wb_motor_set_position(wheel_motor, INFINITY);
12 // (Velocity bleibt zunächst 0, wird später gesetzt)
13
14 //...(im Reglerloop, nach PID/MPC-Berechnung von final_steer/target_speed)
15
16 // Lenkantrieb: Soll-Lenkwinkel setzen
17 wb_motor_set_position(handlebars_motor, -final_steer);
18 // Radantrieb: Soll-Drehgeschwindigkeit setzen
19 wb_motor_set_velocity(wheel_motor, target_speed);

```

Listing 4: IMU-Initialisierung und -Abfrage im C-Controller (Balance-Regelung)

Durch diese Implementierung wird das Zusammenwirken aller Komponenten deutlich: **Sensoren** (IMU, Kamera, Encoder) liefern den aktuellen Zustand, der **Regler** (PID & MPC) berechnet daraus Stellgrößen, und die **Aktuatoren** (Motoren) führen diese Stellgrößen am physikalischen Modell aus. Die Webots-Simulation ermöglicht es, all dies in einem deterministischen Umfeld zu testen – etwaige Fehlanpassungen (wie zu schwache Motoren oder unzureichende Sensorfrequenzen) würden sofort im Verhalten sichtbar und könnten iterativ justiert werden, bevor der Algorithmus auf einen realen Prototyp übertragen wird.

4.6 Controller

Ein Webots-Controller ist ein separates Programm (Prozess), das einen Roboter in der Simulation steuert und über eine definierte API mit der Simulationswelt kommuniziert. Für jeden Roboter kann ein Controller in C/C++, Python, Java oder MATLAB laufen, wobei Webots den Datenaustausch (Sensorwerte, Aktuatorbefehle) bei jedem Simulationsschritt synchronisiert. Typischerweise durchläuft ein Controller eine Endlosschleife, in der er Sensoren liest, Berechnungen durchführt und Aktuatoren ansteuert, wobei am Ende jeder Iteration die Funktion `wb_robot_step()` aufgerufen werden muss. Dieser Aufruf übergibt die gewünschte Control-Step-Dauer (in Millisekunden) an den Simulator und bewirkt, dass Webots die Simulation um genau diesen Zeitschritt fortsetzt und dabei alle bis dahin vom Controller gesetzten Befehle anwendet. Danach erhält der Controller die aktualisierten Sensorwerte des neuen Simulationszustands zurück. Abbildung 3.6 veranschaulicht diese Synchronisation [5].

Alle Anweisungen vor dem nächsten Zeitschritt (`wb_robot_step`-Aufruf) werden zunächst gesammelt; mit dem Step-Aufruf rechnet der Simulator den nächsten Physikschrift und aktualisiert die Sensoren, bevor der Controller weiterläuft. Auf diese Weise laufen Simulation und Controller *synchron* in gleichmäßigen Zeitschritten. Standardmäßig wartet Webots in diesem synchronen Modus darauf, dass jeder Controller seinen `wb_robot_step` ausführt, bevor der nächste Simulationsschritt erfolgt. Dies garantiert Konsistenz, erlaubt aber auch differierende Abtastraten: Der Kontrollschritt muss ein ganzzahliges Vielfaches des Simulations-Basisschritts sein.

Beispielsweise kann bei einer Weltzeit-schrittauflösung von 2 ms ein Controller mit 2 ms, 10 ms oder 50 ms Schrittweite betrieben werden — Webots führt dann pro Controller-schritt entsprechend 1, 5 bzw. 25 Simulationsschritte aus und wartet am Ende ggf. auf den Controller. Diese Entkopplung nutzt Webots, um auch langsamere Controller (z. B. in Python) in eine schnelle Physiksimulation einzubetten, ohne die Synchronisation zu verlieren.

Um auf Sensoren und Aktuatoren zuzugreifen, stellt Webots gerätespezifische API-Funktionen bereit. Im C-Controller werden z. B. Geräte über `wb_robot_get_device("name")` referenziert und Sensoren mit `wb_sensor_enable()` aktiviert. Entsprechende Methoden existieren in Python (`robot.getDevice()`, `device.enable()`). Ein Sensor liefert erst ab dem nächsten `wb_robot_step()` gültige Messwerte, da die Simulationszeit dann fortgeschritten ist.

Aktuatorbefehle (z. B. `wb_motor_set_position()`

oder `motor.setPosition()` in Python) werden zunächst lokal gepuffert—die tatsächliche Ausführung in der Physik-Engine erfolgt erst am nächsten Step-Synchronisationspunkt. Daher ist es sinnlos, in derselben Schleifeniteration zwei unterschiedliche Positionen für denselben Motor zu setzen: Nur der zuletzt vor `wb_robot_step()` gesetzte Wert wird im kommenden Schritt ausgeführt. Ebenso bleiben zwei unmittelbar nacheinander gelesene Sensorsamples identisch, solange kein Simulationsschritt dazwischen liegt. Dieses deterministische Schritt-für-Schritt- Ausführungsmodell erleichtert die Reproduzierbarkeit und verhindert *race conditions* zwischen Physik und Reglercode.

Für das selbstbalancierende Fahrrad wurden zwei Controller-Prozesse implementiert, die in Webots zeitgleich laufen und über die oben beschriebene Mechanik gekoppelt sind: Einerseits der *Balance-Controller* auf dem Fahrrad (in C, hoher Takt), andererseits ein *Vision-Controller* auf

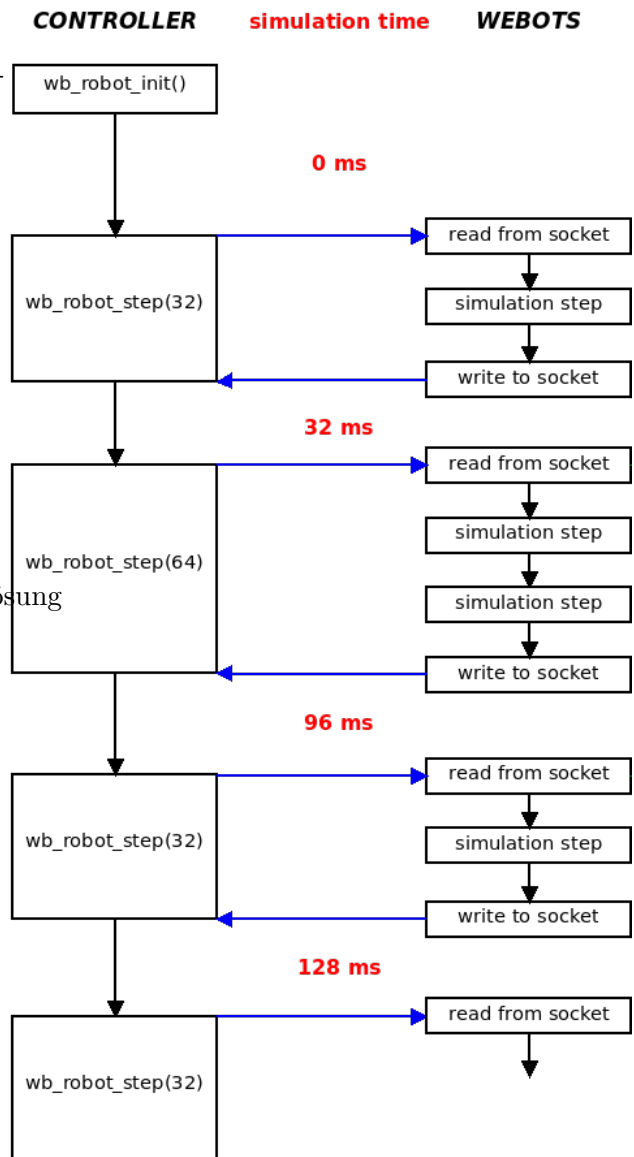


Figure 9: Synchronisation zwischen Controller und Simulation.

einem Supervisor-Knoten (in Python, langsamer Takt). Die Welt ist so konfiguriert, dass das Fahrrad einen **Receiver** und einen **Emitter** trägt, während der Supervisor das komplementäre Gegenstück besitzt; damit entsteht ein bidirektionaler Nachrichtenkanal innerhalb der Simulation:

```

1  Robot {
2      Controller "balance_control_c"
3      Receiver { name "command_rx" channel 1 bufferSize 64 }
4      Emitter { name "status_tx" channel 2 bufferSize 64 }
5  }
6  Supervisor {
7      Controller "vision_control_py"
8      Emitter { name "command_tx" channel 1 bufferSize 64 }
9      Receiver { name "status_rx" channel 2 bufferSize 64 }
10 }
```

Listing 5: Nachrichtenkanal zwischen Controller und Simulation

Über Channel 1 sendet der Vision-Controller Steuerkommandos an das Fahrrad, und über Channel 2 übermittelt der Balance-Controller seinerseits Statusdaten zurück (z.B. aktueller Rollwinkel, Lenkwinkel, Geschwindigkeit). Beide Controller laufen im *synchronen Modus*, d. h. Webots hält die Physik so lange an, bis sowohl der schnelle C-Controller (alle 2 ms) als auch der langsamere Python-Controller ihren Schritt abgearbeitet haben [6] [7]. Praktisch bedeutet dies: Der Balance-Regler berechnet bei jedem Simulationsschritt neue Motorbefehle, während der Vision-Algorithmus nur alle 25 Schritte (50 ms) ein neues Lenkkommando liefert. Die folgenden Punkte fassen Aufgaben und Interaktion zusammen:

- *Balance-Controller (C, 500 Hz)*: Liest hochfrequent IMU- (Rollwinkel) und Radsensoren aus, regelt den Balancierwinkel mittels PID und steuert den Lenkmotor sowie den Antriebsmotor kontinuierlich an. Verarbeitet eingehende Sollkommandos der Vision-Ebene (z.B. Lenkwinkel) und kombiniert sie mit dem internen Regleroutput. Sendet zyklisch seinen Status (Stabilitätsmaß, aktuelle Geschwindigkeit etc.) über den Emitter aus. Bei Ausbleiben externer Kommandos hält er das Fahrzeug selbständig im Gleichgewicht (Fallback auf reinen Balance-Modus).
- *Vision-Controller (Python, ~20 Hz)*: Läuft als Supervisor-Controller mit Zugriff auf globale Informationen. Nutzt die am Fahrzeug montierte Kamera (Bildrate ca. 20 Hz) zur Fahrbahnerkennung (z.B. YOLO-Segmentierung) und berechnet auf dieser Basis einen Trajektorien- bzw. Lenksollwert mittels modellprädiktiver Regelung (MPC) oder wahlweise eines PID. Dieses Sollkommando wird paketbasiert an den Balance-Controller gesendet (**Emitter** auf Kanal 1). Zudem überwacht der Vision-Controller den empfangenen Fahrzeugzustand (**Receiver** auf Kanal 2), um die Regelgüte zu beurteilen oder Logging durchzuführen. Gegebenenfalls passt er seine Fahrstrategie an (etwa Geschwindigkeitsanpassung bei großen Balance-Fehlern).

Durch diese Aufteilung konnte eine zweistufige Regelungsarchitektur realisiert werden: Der innere C-Regler stabilisiert das Fahrrad auf kurzen Zeitskalen autonom, während der äußere Python-Regler in größeren Abständen Kurskorrekturen vorgibt. Webots gewährleistet die Synchronisierung beider Teilprozesse, sodass Kommandos verlustfrei und deterministisch am Step-Synchronisationspunkt ausgetauscht werden. Im Ergebnis reagiert das Gesamtsystem echtzeitnah auf Störungen (Balance) und plant zugleich vorausschauend seine Fahrtrajektorie; die Kopplung konnte in der Simulation umfassend getestet und für den späteren Hardware-Einsatz vorbereitet werden.

4.7 Angebundene GUI

Die Simulationsumgebung verfügt über ein tkinter-basiertes GUI-Kontrollzentrum, mit dem sich die Regelungsparameter des selbstbalancierenden Fahrrads in Echtzeit anpassen und die Fahrdaten live überwachen lassen. Dieses GUI bietet eine benutzerfreundliche Oberfläche, um während der laufenden Webots-Simulation Einstellungen zu ändern, ohne die Simulation unterbrechen zu müssen. Es gliedert sich in mehrere Tabs, von denen insbesondere “Parameter” und “Monitoring” für die Laufzeitanpassung und -beobachtung relevant sind. Im Folgenden wird erläutert, wie diese Tabs funktionieren und wie die Laufzeitanpassung der Parameter sowie das Live-Monitoring umgesetzt sind.

Parameter-Tab - Laufzeitfähige Parameteranpassung: Der Parameter-Tab ermöglicht die direkte Einstellung der Regelungsparameter während der Simulation. Über Slider und Eingabefelder können z. B. PID-Reglergrößen (K_p , K_i , K_d) und andere Steuerungsgrößen komfortabel angepasst werden. Dabei werden die aktuellen Werte in Echtzeit im GUI angezeigt und validiert, während der Nutzer sie ändert. Einzelne Parametergruppen (etwa Winkel-PID, Geschwindigkeitsregelung, mechanische Limits) sind übersichtlich gruppiert, sodass man nicht auf die Details jedes Parameters eingehen muss, aber einen strukturierten Überblick behält.

Wesentlich ist, dass Änderungen unmittelbar wirksam werden, ohne die Simulation neu zu starten. Technisch geschieht dies durch ein SON-basiertes Konfigurationssystem im Hintergrund. Das GUI lädt zu Beginn die aktuellen Parameter aus einer Konfigurationsdatei (z. B. `balance_config.json`) und schreibt geänderte Werte bei Bedarf zurück. Wird ein Wert verändert, kann der Nutzer die neue Konfiguration speichern und anwenden (Schaltflächen “Speichern” bzw. “Anwenden” im GUI). Die laufende Regelungssoftware erkennt diese Aktualisierung dank eines Hot-Reload-Mechanismus: Der Balance-Controller lädt die JSON-Config in kurzen Intervallen oder auf Befehl neu, sodass geänderte Parameter sofort in den Regler einfließen.

Monitoring-Tab – Live-Datenvisualisierung und CSV-Logging: Im *Monitoring*-Tab werden Fahr- und Regeldaten *live* visualisiert, um das Verhalten des Fahrrads während der Simulation nachvollziehen zu können. Abbildung 9 zeigt den Monitoring-Tab mit Echtzeitdiagrammen. Der Tab bietet interaktive Plots, die zentrale Größen über der Zeit darstellen — etwa den Rollwinkel, den Lenkausgang des Reglers, die aktuelle und Ziel-Geschwindigkeit sowie die PID-Regleranteile (P, I, D). Über Auswahlkästchen kann gezielt gesteuert werden, welche Kurven angezeigt werden, um den Überblick zu behalten; beispielsweise lassen sich nur die Balance-Controller-Daten oder nur die PID-Terme ein- oder ausblenden. Zusätzlich stehen Zoom-Regler zur Verfügung, um den dargestellten Zeitraum einzugrenzen und Details zu analysieren.

Die Datengrundlage bilden CSV-Protokolldateien, die vom Balance-Controller während jeder Fahrt automatisch erzeugt werden. Jede Simulation (bzw. Fahrt) wird lückenlos in einer Log-Datei festgehalten — mit Zeitstempel und den relevanten Mess- und Steuergrößen (u. a. Winkel, Geschwindigkeiten, Reglerausgänge, Fehlerwerte). Der Monitoring-Tab listet diese Aufzeichnungen und ermöglicht sowohl das nachträgliche Laden einer aufgezeichneten CSV-Datei als auch den Live-Stream der jeweils neuesten Datei. Im Live-Modus wählt das System automatisch die aktuell von der Simulation beschriebene Log-Datei aus und aktualisiert die Plots fortlaufend; alternativ kann der Benutzer eine frühere Aufzeichnung vollständig laden und visualisieren. In beiden Fällen normalisiert das GUI die eingelesenen CSV-Daten (z. B. Umrechnung von Radmaß in Grad oder m/s in km/h) und zeichnet zweigeteilt: typischerweise erscheinen Winkel- und Lenkdaten im oberen Diagramm, die PID-Terme sowie Geschwindigkeiten im unteren, jeweils mit gemeinsamer Zeitachse.

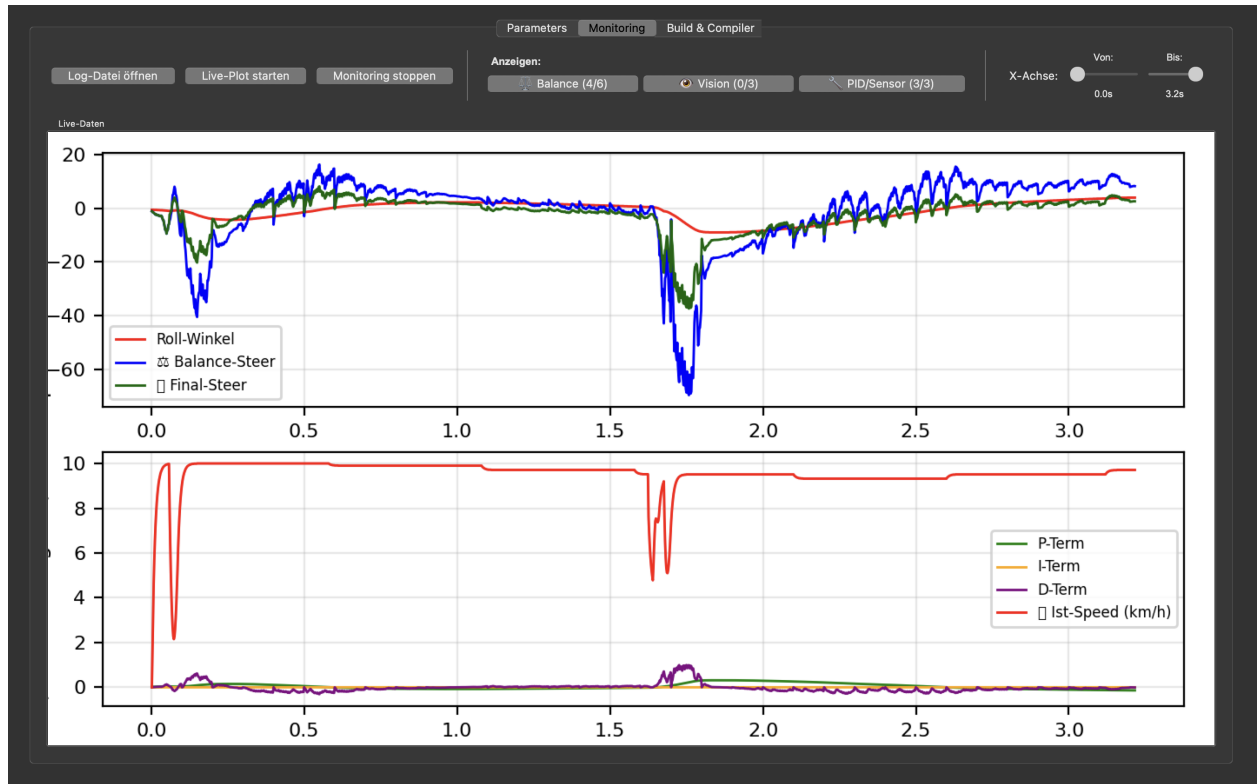


Figure 10: Monitoring-Tab mit Echtzeitdiagrammen.

Ergänzend zur grafischen Darstellung bietet der Monitoring-Tab Statusanzeigen, die u. a. den aktuell geladenen Logfile-Namen, den Zeitpunkt der letzten Aktualisierung und den Laufzustand des Controllers ausweisen. So bleibt stets transparent, ob die Verbindung zur Simulation aktiv ist und welche Datenquelle visualisiert wird. Zusammengenommen stellt der Monitoring-Tab ein leistungsfähiges Werkzeug dar, um die Auswirkungen von Parameteränderungen sofort sichtbar zu machen und das Systemverhalten mittels aufgezeichneter Daten zu analysieren. Die Kombination aus laufzeitfähiger Parameteranpassung und Live-Monitoring ermöglicht ein effizientes Trial-and-Error-Tuning des selbstbalancierenden Fahrrads, unterstützt durch intuitive GUI-Interaktion und automatisierte Datenaufzeichnung.

5 Kombinierte Regelung

5.1 Portierung des Balance-PID (Zander)

Jonah Zanders Bachelorarbeit (2023) behandelt die Entwicklung einer Regelung für ein selbst-balancierendes Fahrrad [35]. ein Ansatz stützte sich auf eine mehrstufige Kaskadenregelung, um die Balance des Fahrrads trotz realer Sensor- und Aktorbeschränkungen sicherzustellen. In der Original-Hardwarearchitektur erfassen ein *BNO055*-IMU-Sensor und ein *BeagleBone-Black*-Einplatinencomputer den Neigungswinkel (Rollwinkel) des Fahrrads. Diese Information wird in einer *primären* PID-Schleife — dem “Angle-to-Steering”-Regler — verarbeitet, der aus dem Rollwinkelabweichungssignal einen Ziel-Lenkswinkel berechnet. Zander wählte dafür relativ hohe proportionale ($K_p \approx 10$) und differentielle Verstärkungen ($K_d \approx 2,2$) und verzichtete auf einen Integralanteil ($K_i = 0$). Diese Parametrierung reagiert schnell und ausreichend gedämpft auf Neigungsabweichungen, ohne driftbedingtes Aufintegrieren — ein wichtiger Aspekt, um Aufschaukeln im realen System zu vermeiden. Der resultierende Lenkwinkel-Sollwert wird anschließend an eine *sekundäre* PID-Schleife übergeben. Diese zweite Regelstufe regelt den elektrischen *Lenkmotor* (über einen MD49-Motorcontroller) so, dass der Lenker den vorgegebenen Sollwinkel tatsächlich einnimmt. Aufgrund von Reibung, Trägheit und Totzeit des Lenkantriebs sind hier wesentlich höhere Reglerverstärkungen erforderlich (im Original z.B. $K_p \approx 850$, $K_i \approx 300$) sowie eine Ausgangsbegrenzung, um das physische System nicht zu übersteuern. Als *dritte* Stufe implementierte Zander eine einfache Geschwindigkeitsregelung für den Antrieb des Fahrrads. Diese war in seiner Umsetzung stark vereinfacht: Über eine Fernbedienung konnte eine konstante Vorwärtsgeschwindigkeit (ca. 3 m/s) ein- oder ausgeschaltet werden. Damit wurde gewährleistet, dass das Fahrrad eine minimale Fahrt erreicht — notwendig, da ein selbstbalancierendes Fahrrad bei Nullgeschwindigkeit instabil ist.

Zusätzlich integrierte Zander sicherheitskritische Mechanismen wie einen Watchdog-Thread und eine mehrsträngige Programmausführung, um unterschiedliche Abtast- und Regelpfade in Echtzeit ablaufen zu lassen (z.B. IMU-Abtastung mit 200 Hz, Lenkregelung mit 1000 Hz). Diese Maßnahmen kompensieren hardwarebedingte Latenzen und mögliche Sensorausfälle und stellten die Balance in der Praxis robust sicher.

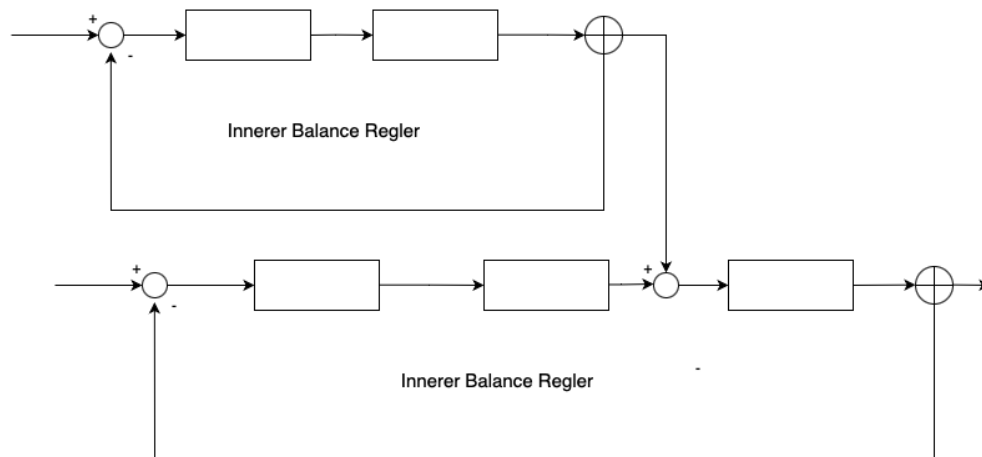


Figure 11: Kombimierter Regler.

Im Rahmen dieser Arbeit wurde Zanders Balance-PID-Regler in einer Webots-Simulation nachge-

bildet — jedoch in vereinfachter Form, da die Simulationsumgebung viele reale Komplexitäten abfängt. Die Sensorik in Webots liefert z.B. den Rollwinkel ohne nennenswertes Rauschen oder Verzögerung, wodurch aufwändige Filterung oder Sensorfusion nicht notwendig sind; ein einfacher gleitender Mittelwert über wenige Schritte genügt zur Glättung. Ebenso erlaubt die Simulations-Aktuatorik direkten Zugriff auf den Lenkmotor: Anstatt wie im realen System einen separaten PID-Regler für die Lenkposition und einen Motorcontroller anzusteuern, kann der simulierte Lenkwinkel direkt gesetzt werden. Konkret wird in jedem Simulations-Timestep zunächst der aktuelle Rollwinkel berechnet und dann der Balance-PID (mit denselben Parametern K_p, K_i, K_d wie bei Zander) ausgeführt. Dessen Ausgang — anstelle eines Encoder-Sollwerts — wird unmittelbar als Stellwert an den virtuellen Lenkmotor übergeben, der den Lenker entsprechend positioniert. Die zweite Regelstufe (Steering-Position-PID) entfällt somit, da Webots intern den Motor auf die Position eigenständig hält. Auch die Thread-Struktur kann stark vereinfacht werden: Webots arbeitet mit einer einzigen synchronen Hauptschleife, die Sensoraktualisierung und Stellgrößenberechnung in festen Zeitschritten kombiniert. Das eliminiert die Notwendigkeit paralleler Threads und spezieller Watchdogs — Timing und Stabilität werden durch die Simulationsengine garantiert.

Trotz dieser Vereinfachungen wurden die Kernprinzipien von Zanders Regelung beibehalten. Insbesondere stellt der Rollwinkel-PID weiterhin das zentrale Element dar, das die Balance regelt. Die gleiche Abstimmung ($K_p = 10, K_i = 0, K_d = 2,2$) hat sich auch in der Simulation als geeignet erwiesen, um ein schnelles Ausregeln mit moderater Dämpfung zu erreichen — analog zu den realen Tests von Zander [35]. Unterschiede ergeben sich vor allem durch die Randbedingungen: Zanders physisches System war verschiedenen Umwelteinflüssen (Bodenunebenheiten, Windstöße, Reibungsschwankungen) ausgesetzt, während die Webots-Simulation idealisierte Bedingungen bietet.

Ein weiterer Unterschied liegt in der Geschwindigkeitssteuerung. Während im realen Aufbau eine konstante Mindestgeschwindigkeit manuell vorgegeben wurde, nutzt die Simulation eine adaptive Geschwindigkeitsanpassung: Die Antriebsdrehzahl wird dynamisch reduziert, wenn starke Lenkeingriffe (große Stellgrößen des Balance-PID) auftreten. Diese direkte Kontrolle im Simulator erhöht die Stabilität, indem das „Fahrrad“ bei drohender Instabilität automatisch abbremst. Zander musste hingegen einen Sicherheitsabstand einplanen: Sein System durfte nicht zu langsam werden, da ein reales Fahrrad unter $\approx 3 \text{ m/s}$ kaum stabil bleibt. Die Simulation erlaubt hier flexible Anpassungen, da keine Sturzgefahr für ein echtes Gerät besteht und Parameter gefahrlos variiert werden können.

Die Portierung des Zander’schen PID-Reglers in die Webots-Umgebung zeigt, dass ein vereinfachtes Regeldesign genügt, wenn Sensorik und Aktorik idealisiert sind. Viele Komplexitäten der realen Implementierung (zusätzliche Regelkreise, Echtzeit-Threading, Hardware-Toleranzen) mussten in der Simulation nicht nachgebildet werden, was die Implementierung schlanker machte. Dennoch basiert die Regelung auf dem gleichen Prinzip wie bei Zander: der stabilisierenden Rückkopplung des Rollwinkels auf den Lenkwinkel. Die erfolgreiche Balance in der Simulation bestätigt die Wirksamkeit von Zanders Ansatz unter erleichterten Bedingungen und bildet die Grundlage für weiterführende Regelungs- und Steuerungsstrategien in dieser Arbeit.

5.2 Portierung des Fahrtrichtungs-MPC (Giray)

Yasin Giray entwickelte in seiner Masterarbeit einen vision-basierten Fahrtrichtungsregler auf Basis von Model Predictive Control (MPC), um das selbstbalancierende Fahrrad autonom der Fahrspur folgen zu lassen. Dieser Regler modelliert die Fahrzeugdynamik mittels Einspurmodell und nutzt

zwei Steuergrößen — aktuelle Geschwindigkeit v und Lenkwinkel ϕ — als Eingaben für die Prädiktion. Der MPC berechnet also auf Grundlage von v und Lenkeinschlag die Fahrzeugbewegung voraus. Für den prädiktiven Regler wird ein Zustandsvektor $x = [x, y, v, \text{yaw}]$ (Position, Geschwindigkeit, Gierwinkel) und ein Eingangsvektor $u = [\text{accel}, \text{steer}]$ definiert. Giray wählte einen Zeithorizont von etwa 2 s und formulierte ein Optimierungsproblem, das die Abweichung von einer Soll-Trajektorie minimiert sowie Steuerungsaufwand und -änderungen penalisiert. Die Kostenfunktion des MPC addiert quadrierte Abweichungen der Zustände und Eingriffe über den Horizont sowie eine Endzustands-Strafe, z. B. $J = \sum (x^T Q x + u^T R u + \Delta u^T R_d \Delta u) + x_{\text{final}}^T Q_f x_{\text{final}}$. Neben den Kosten werden physikalische Nebenbedingungen wie maximaler Lenkwinkel, Lenkwinkelrate und Geschwindigkeit im Optimierungsproblem berücksichtigt, um realistische Lösungen zu gewährleisten.

Wesentliche Voraussetzung für den Fahrtrichtungs-MPC ist eine geeignete Referenztrajektorie der Straße. Giray implementierte hierfür eine Computer-Vision-Pipeline auf Basis von YOLOv8-Segmentierung, welche im Kamerabild die befahrbare Straße und deren Verlauf erkennt. Aus den segmentierten Bilddaten wird eine Reihe von Stützpunkten der Fahrbahn extrahiert (z. B. die Mittellinie oder Fahrbahnrande). Anschließend wird mittels Spline-Interpolation eine glatte Kurve durch diese Punkte gelegt, die als Referenzkurve für den MPC dient. Der MPC nutzt diese prognostizierte Straßenspur, um vor auszuplanen: Er simuliert auf Basis des Einspurmodells die Fahrzeugbewegung und berechnet die optimalen Steuerbefehle, damit das Fahrrad der Kurve bestmöglich folgt. In Girays Umsetzung wird der Regler zyklisch mit ca. 10 Hz ausgeführt (Tickrate 100 ms). Pro Zyklus initialisiert der Controller den aktuellen Fahrzeugzustand (Position, Geschwindigkeit, Orientierung) und berechnet aus der ersten Sollkurve einen Vorhersagepfad. Die Optimierung (z. B. gelöst mit `cvxpy`) liefert dann eine Stellgrößen-Sequenz für Beschleunigung und Lenkwinkel. Tatsächlich wird nur der erste Lenkwinkel-Impuls umgesetzt (Receding Horizon Control), welcher als neuer Lenkbefehl an das Fahrzeug geht. Im nächsten Zyklus wird der MPC erneut mit aktualisiertem Zustand und ggf. angepasster Referenz gestartet. Giray konnte in Simulationen zeigen, dass der MPC-Regler das Fahrrad effektiv der vorgegebenen Spur folgen lässt: Die geplante Spline-Trajektorie (rot) und die vom MPC gefahrene Ist-Trajektorie (blaue Punkte) stimmen eng überein, was die Güte der prädiktiven Regelung belegt.

In der vorliegenden Arbeit wird Girays Fahrtrichtungs-MPC nahtlos in die Webots-Simulationsumgebung portiert und mit dem Balance-Regler gekoppelt. Die Umsetzung erfolgt durch einen Vision-Controller in Python, der analog zum Ansatz von Giray einen modellprädiktiven Lenkwinkel-Regler realisiert. Auch hier kommt ein vereinfachtes Bicycle-Modell ($x = [x, y, v, \text{yaw}]$, $u = [\text{accel}, \text{steer}]$) zum Einsatz, und die prädiktive Optimierung berücksichtigt ähnliche Stellgrößen- und Zustandskosten sowie Begrenzungen (Lenkwinkel, Lenkrate, Geschwindigkeit). Einige Anpassungen waren jedoch nötig: Anstatt einer komplexen Kurvenextraktion aus dem Kamerabild nutzt der Vision-Controller einen direkten Querfehler zur Fahrspur als Eingang. Die laterale Abweichung des Fahrrads von der Fahrbahnmitte (z. B. normiert zwischen ± 0.5) wird laufend aus der Bildsegmentierung oder einem Fallback-Verfahren ermittelt. Aus diesem Fehler wird eine Zielführung abgeleitet: der Soll-Gierwinkel des Fahrrads wird proportional zum Querfehler angepasst, sodass das Fahrzeug zurück zur Spurmitte dreht. Konkret wird ein `target_yaw` berechnet, der vom aktuellen `yaw`-Winkel um einen Korrekturterm (proportional zum Fehler) abweicht. Zwischen aktuellem `yaw` und diesem Ziel-`yaw` erzeugt das System eine kurze Referenztrajektorie über den Horizont T : das Fahrrad soll in T Schritten gleichmäßig von der aktuellen Ausrichtung zur Zielausrichtung drehen. Dazu werden über $T = 3$ diskrete Zeitschritte (mit $\Delta t = 0,1$ s) Zwischenausrichtungen interpoliert und die entsprechenden vorwärts projizierten Positionen berechnet. Die resultierende Trajektorie ist im Wesentlichen eine sanfte Kurve zurück zur Fahrbahnmitte. Dieses

Verfahren vereinfacht die Pfadplanung erheblich und ist ausreichend für die Spurhaltung, ohne im Rahmen dieses Kapitels tief in die Bildverarbeitung einsteigen zu müssen (die Details der Fahrbahnerkennung folgen in einem späteren Kapitel).

Auf Basis der generierten Referenz führt der Vision-MPC in Webots die Optimierung durch. Der Prädiktionshorizont ist mit 3 Schritten ($\approx 0,3\text{s}$) bewusst kurz gehalten, um die Rechenzeit gering zu halten. Innerhalb dieses Horizonts minimiert der MPC Abweichungen zwischen Fahrzeug und Referenzpfad sowie plötzliche Änderungen der Steuerbefehle. Es gelten harte Constraints: Der Lenkwinkel ist z. B. auf $\pm 30^\circ$ begrenzt, die Lenkwinkeländerung pro Zeitschritt auf ca. $20^\circ/\text{s}$, die Geschwindigkeit auf ca. 8 m/s (bzw. wird nicht unter $\sim 0,5\text{ m/s}$ fallen). Diese Grenzen orientieren sich an den physikalischen Fähigkeiten des simulierten Fahrrads und entsprechen teils den in Girays Arbeit genutzten Werten. Der Optimierer (`cvxpy`) berechnet dann die Eingangssequenz $u_{0..T-1}$; wenn ein optimales Ergebnis gefunden wird, entnimmt der Controller den ersten Steuerbefehl (Beschleunigung und Lenkausschlag) als Sollwert. Falls das Optimierungsproblem nicht zu lösen ist oder zu lange dauert, greift ein Fallback-Regler: In diesem Fall wird etwa ein einfacher P-Regler genutzt, der den Lenkwinkel proportional zum aktuellen Querfehler setzt. In der Praxis zeigte sich jedoch, dass der MPC mit den reduzierten Dimensionen echtzeitfähig läuft (die `cvxpy`-Lösung erfolgt in wenigen Millisekunden).

Die vom Vision-MPC berechneten Sollgrößen — kontinuierlicher Lenkwinkel (`steer_rad`) und gewünschte Beschleunigung bzw. Geschwindigkeit (`accel`) — werden anschließend für die Ausführung skaliert und geglättet. Insbesondere wird der Lenkwinkel auf den normierten Bereich $[-1, 1]$ abgebildet, der dem Stellbereich des Lenkservos entspricht. Zudem begrenzt ein Rate-Limiter plötzliche Änderungen (z. B. maximal $\pm 0,005$ pro Zeitschritt in der normierten Größe), um ruckfreie Lenkbewegungen zu gewährleisten. Auch die aus `accel` abgeleitete Zielgeschwindigkeit wird auf 0 bis 1 normiert. Dieser Wert dient als grober Speed-Befehl, obgleich die tatsächliche Geschwindigkeitsregelung in unserem System primär auf der C-Seite erfolgt (siehe unten).

Die eigentliche Integration des portierten MPC in die Simulationsregelung geschieht durch eine Kaskadierung mit dem Balance-Controller. Wie in Abbildung 4.2 (Signalflussdiagramm) skizziert, arbeitet der Vision-Controller (Python) auf niedriger Frequenz ($\sim 20\text{ Hz}$) und sendet periodisch seine berechneten Steuerbefehle an den Balance-Controller (C). Der Balance-Controller läuft mit dem Webots-Basis-Zyklus (2–10 ms Schritte, also $> 100\text{ Hz}$) und stabilisiert das Fahrzeug über einen PID-Regler auf den Rollwinkel. Konkret regelt er den Lenkwinkel so, dass das Fahrrad aufrecht bleibt (Rollabweichung $\rightarrow$ Lenkimpuls). Die vom Vision-MPC kommende Lenkvorgabe wird nun mit der Balance-Korrektur überlagert. In der Implementierung wird eine feste Gewichtung von 50:50 verwendet: der finale Lenksollwert an den Motor ist der Mittelwert aus Vision-Output und Balance-PID-Output. Damit wird verhindert, dass die Fahrtrichtungsregelung das Stabilisieren stört — das Fahrrad bleibt primär durch den PID aufrecht, während der MPC langsam die Richtung vorgibt. Zusätzlich beeinflusst der Vision-Befehl die Geschwindigkeit: Der Balance-Controller enthält einen unterlagerten Geschwindigkeitsregler, der bei großen Lenkausschlägen die Vortriebsleistung reduziert (ähnlich einer Kurvenverzögerung). Ist also der visionäre Lenkwunsch stark (scharfe Kurve), bremst der Balance-Controller das Hinterrad etwas ab, um Kippgefahr zu minimieren.

Die Kommunikation zwischen den beiden Reglern erfolgt über einen IPC-Kanal innerhalb von Webots. Der Vision-Controller sendet seine Sollwerte als strukturiertes Paket (Lenkkommando, Geschwindigkeitskommando, Gültigkeits-Flag und einige Diagnosewerte) an den Balance-Controller. Dieser liest das Paket (`Device command_rx`) in jedem Zyklus aus und nutzt die enthaltenen Sollgrößen in seiner Regelung. Umgekehrt schickt der Balance-Controller in festen Intervallen Status-

daten zurück – insbesondere den aktuellen Rollwinkel, den resultierenden Lenkbefehl und Informationen zur Stabilität – damit der Vision-Controller z. B. im Overlay Feedback anzeigen oder bei Kontrollverlust eingreifen kann. Insgesamt ermöglicht diese Architektur eine echtzeitfähige Portierung von Girays Fahrtrichtungs-MPC in die virtuelle Umgebung: Der prädiktive Lenkwinkel-Regler arbeitet mit dem Kamerabild und gibt Sollwerte vor, während der vorhandene Balance-Regler die kurzfristige Stabilisierung übernimmt.

5.3 Strategien der Spurführung

5.3.1 YOLO-Segmentierung (YOLOv8) + MPC

In dieser Strategie kommt ein auf YOLOv8 basierendes Segmentierungsmodell zum Einsatz, um die Fahrspur im Kamerabild zu erkennen. YOLOv8 (Ultralytics) ermöglicht die Echtzeit-Objekterkennung mit Segmentierung – hier wird es genutzt, um den Straßenbereich im Bild als Klasse “Straße” zu identifizieren. Ziel der YOLO-Strategie ist es, den lateralen Spurabweichungsfehler robust und präzise zu bestimmen, indem ein vortrainiertes neuronales Netz die Fahrbahnmarkierungen und -fläche erkennt. Der daraus resultierende Fehlerwert wird anschließend an einen Model Predictive Controller (MPC) übergeben, der optimale Lenk- und Beschleunigungsbefehle berechnet, um das Fahrrad auf Kurs zu halten.

Initialisierung: Beim Start des Vision-Controllers (`vision_control_py.py`) wird das YOLO-Modell einmalig geladen. Die Methode `_init_yolo()` versucht, eine trainierte Gewichtsdatei (z. B. `best.pt`) aus dem Projektverzeichnis zu laden. Ist die `ultralytics`-Bibliothek verfügbar (`YOLO_AVAILABLE=True`) und die Datei vorhanden, wird ein YOLOv8-Segmentierungsmodell in `self.yolo_model` geladen und auf das vorhandene Rechenggerät (GPU oder CPU) übertragen. Der folgende Codeausschnitt zeigt diese Initialisierung – es werden mehrere Pfade nach der Gewichtsdatei durchsucht, das Modell geladen und bei Erfolg aktiviert. Falls kein Modell gefunden wird, deaktiviert das System YOLO und wechselt automatisch in den Fallback-Modus:

```

1  if YOLO_AVAILABLE:
2      possible_paths = [
3          "../balance_control_c/yolo_vision/runs/segment/train/weights/best.pt",
4          "yolo_weights/best.pt",
5          "best.pt"
6      ]
7      for path in possible_paths:
8          if os.path.exists(path):
9              try:
10                 device = "mps" if torch.backends.mps.is_available() else \
11                     ("cuda" if torch.cuda.is_available() else "cpu")
12                 self.yolo_model = YOLO(path).to(device)
13                 print(f"[OK] YOLO-Modell geladen: {path} (Device: {device})")
14                 break
15             except Exception as e:
16                 print(f"[WARN] Fehler beim Laden von {path}: {e}")
17                 continue
18  if not self.yolo_model:
19      print("[WARN] Kein YOLO-Modell gefunden - Fallback-Vision aktiviert")
20      YOLO_AVAILABLE = False

```

Listing 6: YOLO-Modell Initialisierung

Im Echtzeitbetrieb (ca. 20 Hz) wird jedes Kamerabild durch YOLO analysiert. Die Funktion `get_vision_error_yolo()` führt eine Inferenz auf dem aktuellen Frame aus und filtert dabei nur Objekte der Klasse “Straße” (Klassen-ID 2) heraus. YOLO liefert für erkannte Straßenbereiche *Bounding Boxes* und zugehörige Segmentierungsmasken. Aus allen erkannten Bounding Boxes der Klasse 2 werden die Mittelpunkte berechnet und gemittelt, um einen durchschnittlichen Straßenmittelpunkt zu erhalten. Die laterale Abweichung (**error**) ergibt sich dann aus der horizontalen Differenz zwischen diesem Straßenmittelpunkt und der Bildmitte, normiert durch die Bildbreite. Der folgende Codeausschnitt illustriert die Berechnung des Fehlers **error**: Alle Erkennungen mit `cls == 2` (“Straße”) werden gesammelt, ihre x-Mitteltwerte berechnet und daraus die Abweichung vom Bildzentrum bestimmt. Ein positiver Fehlerwert bedeutet, dass die Straße im Bild links der Fahrzeugmitte liegt (das Fahrrad muss nach links gegensteuern), ein negativer Wert entsprechend, dass die erkannte Spur rechts liegt. Typischerweise liegt **error** im Bereich von etwa -0.5 bis $+0.5$ (dimensionslos):

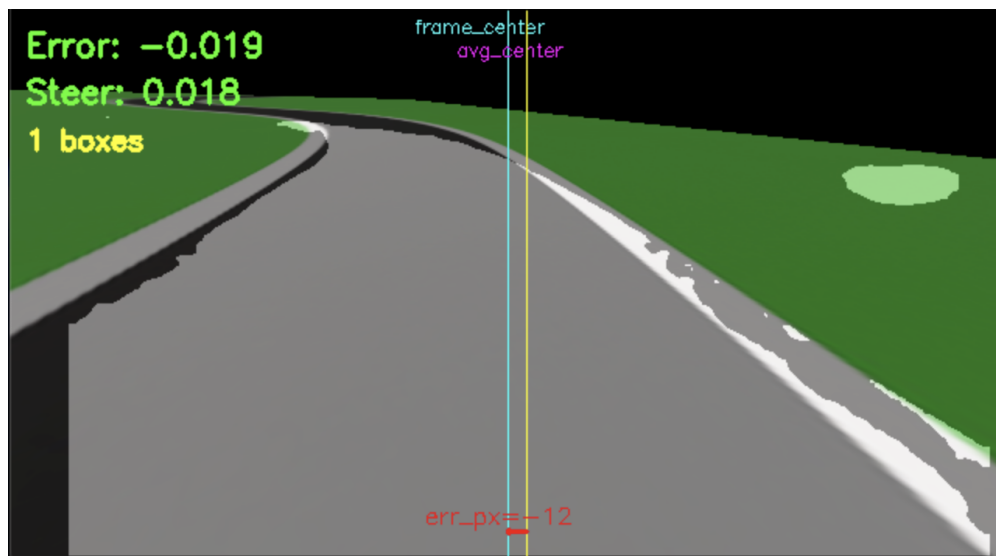


Figure 12: YOLOv8-Segmentierung.

```

1  street_indices = [idx for idx, cls in enumerate(r.bboxes.cls) if cls == 2]
2  if street_indices:
3      x_centers = []
4      for idx in street_indices:
5          xxyy = r.bboxes.xxyy[idx] # Koordinaten der Bounding Box
6          x_center = float((xxyy[0] + xxyy[2]) / 2.0)
7          x_centers.append(x_center)
8      avg_x_center = sum(x_centers) / len(x_centers)
9      frame_center = frame.shape[1] / 2
10     error = (frame_center - avg_x_center) / frame.shape[1] # Normiert
11     self._store_valid_values(error, mask, self.vision_mask_coverage)
12     return error, mask

```

Listing 7: YOLO-Modell Error Berechnung

Zusätzlich zur Fehlerberechnung wird eine Binärmaske der erkannten Straße erzeugt (**mask**). Alle Segmentierungspixel der Klasse Straße werden dazu vereinigt, und es wird die prozentuale Maskenabdeckung im Bild (**vision_mask_coverage**) berechnet. Diese Informationen dienen vor allem der

Visualisierung und Diagnose. Für die Regelung wird ausschließlich der Fehlerwert verwendet, nicht die gesamte Maske.

Um bei Aussetzern der Detektion stabil zu bleiben, implementiert das System eine Ausfallüberbrückung: Der zuletzt gültige Fehlerwert (`last_valid_error`) und die Maske werden zwischengespeichert (`_store_valid_values`). Bleibt eine YOLO-Detektion in einem Zyklus aus (z. B. temporäres Nichterkennen der Spur), greift der Controller auf die zuletzt gültigen Werte zurück. Bei längerer Detektionspause wird der gespeicherte Fehler schrittweise mit einem Faktor < 1 multipliziert (Decay), sodass sein Betrag allmählich abnimmt. Nach einer definierten Anzahl von Ausbleib-Zyklen fällt der Fehler auf 0 zurück, um ein Ausbrechen des Fahrrads zu verhindern. Durch diese Maßnahme werden abrupte Änderungen vermieden und der MPC-Regler bleibt auch bei kurzzeitig fehlender Bildinformation stabil.

MPC-Regelung. Der mittels YOLO ermittelte Spurfehler wird an den Vision-MPC-Controller übergeben, der daraufhin einen Lenksollwert und optional einen Geschwindigkeitssollwert berechnet. Intern nutzt der MPC ein einfaches Bicycle-Modell des Fahrzeugs und einen kurzen Prädiktionshorizont, um die Trajektorie zu optimieren. Ist die Optimierungs-Bibliothek verfügbar, wird ein Quadratikprogramm (`CVXPY`) gelöst, das den Querfehler sowie Änderungen des Lenkwinkels in die Kostenfunktion einbezieht; andernfalls fällt der Controller auf einen einfachen P-Regler zurück, der den Fehler proportional in einen Lenkwinkel umsetzt. Das Ergebnis der MPC-Berechnung ist ein Lenkwinkel (in Rad) sowie eine Beschleunigung bzw. Zielgeschwindigkeit. Diese werden im Python-Controller noch normalisiert und geglättet – der Lenkwert `steer_rad` wird durch den maximalen Lenkwinkel dividiert und zu `steer_cmd` $\in [-1, 1]$ skaliert, anschließend durch einen Rate-Limiter nur begrenzt verändert (max. Änderung ± 0.005 pro Schritt, Begrenzung auf ± 0.08). Der normierte Lenkbefehl sowie ein entsprechender Geschwindigkeitsbefehl werden schließlich als Vision Command an den Balance-Controller (C-Prozess) gesendet. Dort wird dieser Soll-Lenkwert mit dem stabilisierenden Lenkoutput des Balance-PID-Reglers kombiniert (standardmäßig 50/50-Gewichtung).

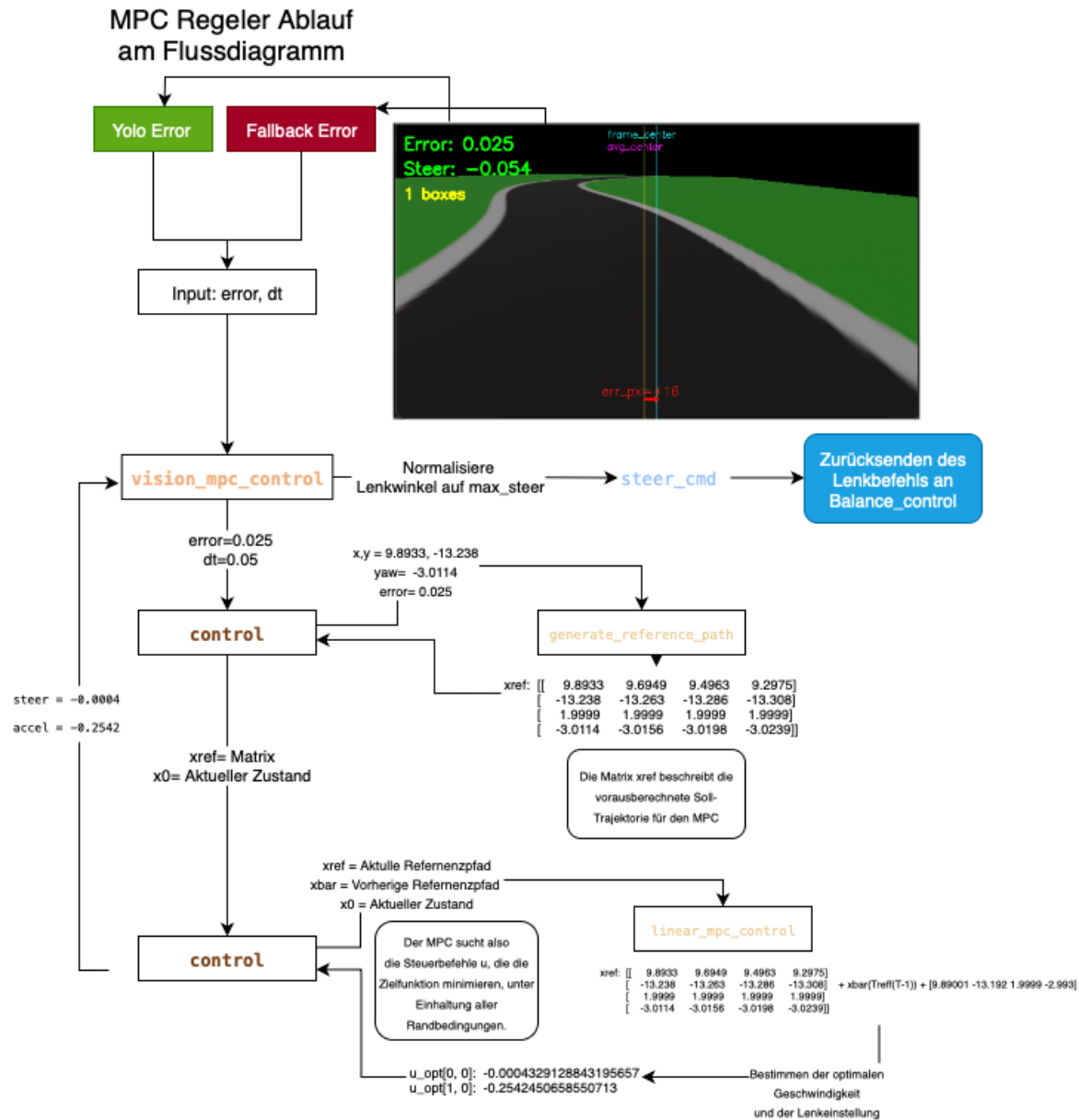


Figure 13: MPC-Strategie.

Limitierungen. Die YOLO-basierte Spurführung hängt stark von einem geeigneten vortrainierten Modell und ausreichender Rechenleistung ab. In Szenarien, die stark von den Trainingsdaten abweichen, kann es zu Fehl- oder Nichterkennungen kommen – in solchen Fällen muss der oben beschriebene Abkling-Mechanismus greifen. Zudem wird aus der Segmentierung derzeit lediglich ein einzelner Querfehlerwert extrahiert; Informationen über den Straßenverlauf (z. B. Krümmung) bleiben ungenutzt, obwohl die Segmentierungsmaske solche Details prinzipiell liefert. Zukünftige Erweiterungen könnten diese zusätzlichen Informationen verwenden, um vorausschauendere Lenkentscheidungen zu ermöglichen. Insgesamt bietet die YOLO+MPC-Strategie eine präzise Spurhaltung, ist aber auf eine funktionierende KI-Erkennung angewiesen.

5.3.2 Fallback-Strategie (Kanten/ROI) + MPC

Falls das YOLO-Modell nicht verfügbar oder die Erkennung unsicher ist, greift eine Fallback-Strategie auf klassische Bildverarbeitung zurück. Diese Methode nutzt Kanten- und Bereichserkennung innerhalb einer ROI (Region of Interest), um weiterhin einen Spurhaltefehler zu bestimmen. Ziel der Fallback-Strategie ist es, eine grundlegende Spurführung auch ohne trainiertes Modell zu gewährleisten – auf Kosten der Robustheit und Genauigkeit. Statt einer neuronalen Segmentierung werden einfache Farb- und Konturfilter verwendet, um die “Straßenkante” bzw. Fahrbahn im Kamerabild zu erkennen. Dadurch kann das System im Notfall den lateralen Fehler abschätzen und den MPC weiterhin mit sinnvollen Sollwerten versorgen.

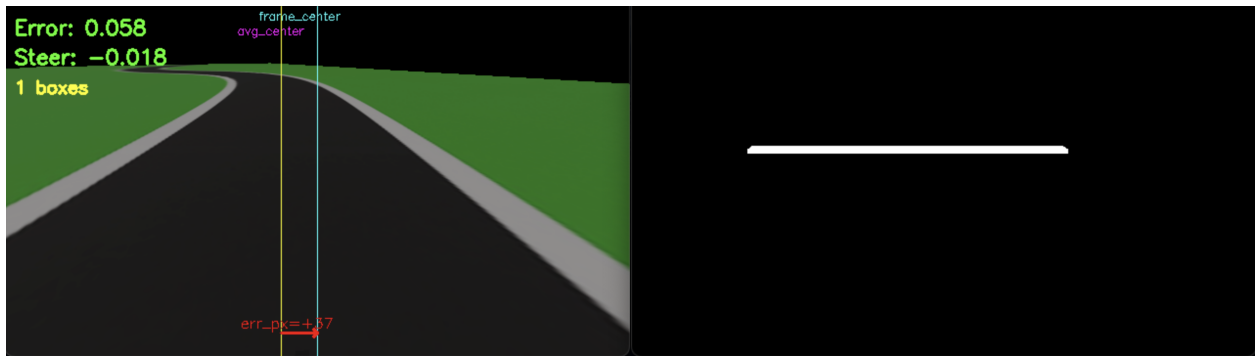


Figure 14: Fallback-Strategie.

Vorgehen. Zu Beginn setzt der Vision-Controller die Fallback-Methode ein, wenn kein YOLO-Modell geladen wurde (oder falls sie in der Konfiguration ausdrücklich gewählt ist). Der Algorithmus definiert zunächst eine feste ROI im Kamerabild, in der die Straße zu vermuten ist. Diese ROI ist trapezförmig: im oberen Bildbereich schmaler und nach unten breiter, um der Perspektive einer Straße nach vorne gerecht zu werden. Standardmäßig beginnt die ROI bei ca. 40% der Bildhöhe und erstreckt sich über $\sim 2\%$ der Bildhöhe als horizontaler Streifen. Sie liegt mittig im Bild und erfasst den Bereich einige Meter vor dem Fahrrad, in dem sich die Fahrspur voraussichtlich befindet. Innerhalb dieser ROI werden die Bilddaten in den HSV-Farbraum konvertiert, um Farbschwellen für die Straßenerkennung anzuwenden. Der Fallback-Algorithmus segmentiert gezielt dunkelgraue Farbtöne (typisch für Asphalt) sowie weiße Bereiche (Markierungslinien): Ein Pixel gilt als “Straße”, wenn es entweder geringe Sättigung und mittlere Helligkeit (Grau) oder sehr geringe Sättigung und hohe Helligkeit (Weiß) aufweist. Nach dieser Farbfilterung werden morphologische Operationen (Schließen und Öffnen) durchgeführt, um Rauschen zu entfernen und verbleibende Bereiche zu verbinden. Anschließend erfolgt eine Konturenerkennung in der bereinigten Binärmaske. Der folgende Codeauszug veranschaulicht diesen Ablauf – es werden zwei Binärmasken für Straße und Markierungen erstellt, vereinigt und daraus Konturen extrahiert. Die größte gefundene Kontur wird als Straßenbereich angenommen, und aus ihr wird der Querfehler bestimmt.

```

1  mask_road = cv2.inRange(hsv, road_lower, road_upper)      # Graue Strasse
    filtern
2  mask_white = cv2.inRange(hsv, white_lower, white_upper)   # Weisse Markierungen
    filtern
3  mask0 = cv2.bitwise_or(mask_road, mask_white)
4  mask0 = cv2.morphologyEx(mask0, cv2.MORPH_CLOSE, kernel)
5  mask0 = cv2.morphologyEx(mask0, cv2.MORPH_OPEN, kernel)
6  contours, _ = cv2.findContours(mask0, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
7  if contours:
8      largest_contour = max(contours, key=cv2.contourArea)
9      x, y, w, h = cv2.boundingRect(largest_contour)
10     center_x = int(x + w / 2)
11     error = (x_set - center_x) / float(max(1, width))      # Fehler normiert an
        Bildbreite
12     mask = np.zeros((height, width), dtype=np.uint8)
13     cv2.drawContours(mask, [largest_contour], -1, 255, -1)
14     ... # (Berechnung der Maskenabdeckung)
15     self._store_valid_values(error, mask, self.vision_mask_coverage)
16     return error, mask

```

Listing 8: Fallback-Modell Error Berechnung

Wie oben ersichtlich, wird die größte Kontur in der Maske ermittelt (vermutlich entspricht diese der Straße im Sichtbereich). Aus der umschließenden Bounding Box dieser Kontur berechnet der Code den Mittelpunkt `center_x`. Die Differenz zwischen diesem Konturmittelpunkt und der Bildmitte (`x_set`, entspricht hier der halben Bildbreite) ergibt den lateralen Fehler `error`, welcher – analog zur YOLO-Methode – durch die Bildbreite normiert wird. Auch in der Fallback-Strategie ist `error > 0` definiert als Spur links der Mitte, `error < 0` als Spur rechts der Mitte. Der berechnete Fehler und die zugehörige Konturmaske werden als neue gültige Werte gespeichert (`_store_valid_values`), sodass sie bei der nächsten Iteration zur Verfügung stehen. Wird keine Kontur gefunden (etwa wenn die Filter keine zusammenhängende Fläche ergeben), greift das System auch hier auf die zuletzt bekannten Werte zurück. In diesem Fall setzt die gleiche Decay-Logik ein wie bei der YOLO-Strategie: Der alte Fehlerwert wird weiterverwendet und schrittweise abgeschwächt, um ruckartige Lenkbewegungen zu vermeiden. Damit überbrückt der Fallback-Ansatz kurze Erkennungsverluste ähnlich robust.

MPC und Steuerung: Der so ermittelte Fehlerwert fließt wiederum in den MPC-Regler ein, der – wie bereits in Abschnitt 4.3.1 beschrieben – einen Lenksollwert berechnet. Die weiteren Schritte (Normierung, Glättung und Übergabe an den Balance-Controller) sind identisch zur YOLO-Strategie. Der Fallback unterscheidet sich also hauptsächlich in der Vision-Fehlerermittlung, während der Regelteil (MPC) unverändert bleibt.

Limitierungen: Die Fallback-Strategie ist einfacher, aber auch anfälliger für Störungen. Sie setzt voraus, dass sich die Straßenfarbe und -markierungen ausreichend deutlich vom Hintergrund abheben. Variierende Lichtverhältnisse, Schatten oder unkonventionelle Straßenbeläge können die fest eingestellten Schwellenwerte unbrauchbar machen. Zudem erfasst die schmale ROI nur einen begrenzten Bereich voraus – plötzliche Krümmungen oder Hindernisse außerhalb dieses Ausschnitts würden nicht rechtzeitig erkannt. Insgesamt bietet der Fallback-Ansatz eine geringere Präzision und Robustheit als die YOLO-Lösung. Er stellt jedoch sicher, dass das Fahrrad notfalls weiter stabil in der Spur gehalten werden kann, selbst wenn die KI-basierte Erkennung temporär ausfällt. In der

Praxis dient diese Strategie somit als Notfallmodus, bis die vollwertige Vision-Erkennung (YOLO) wieder greift.

5.4 Gesamtregelung

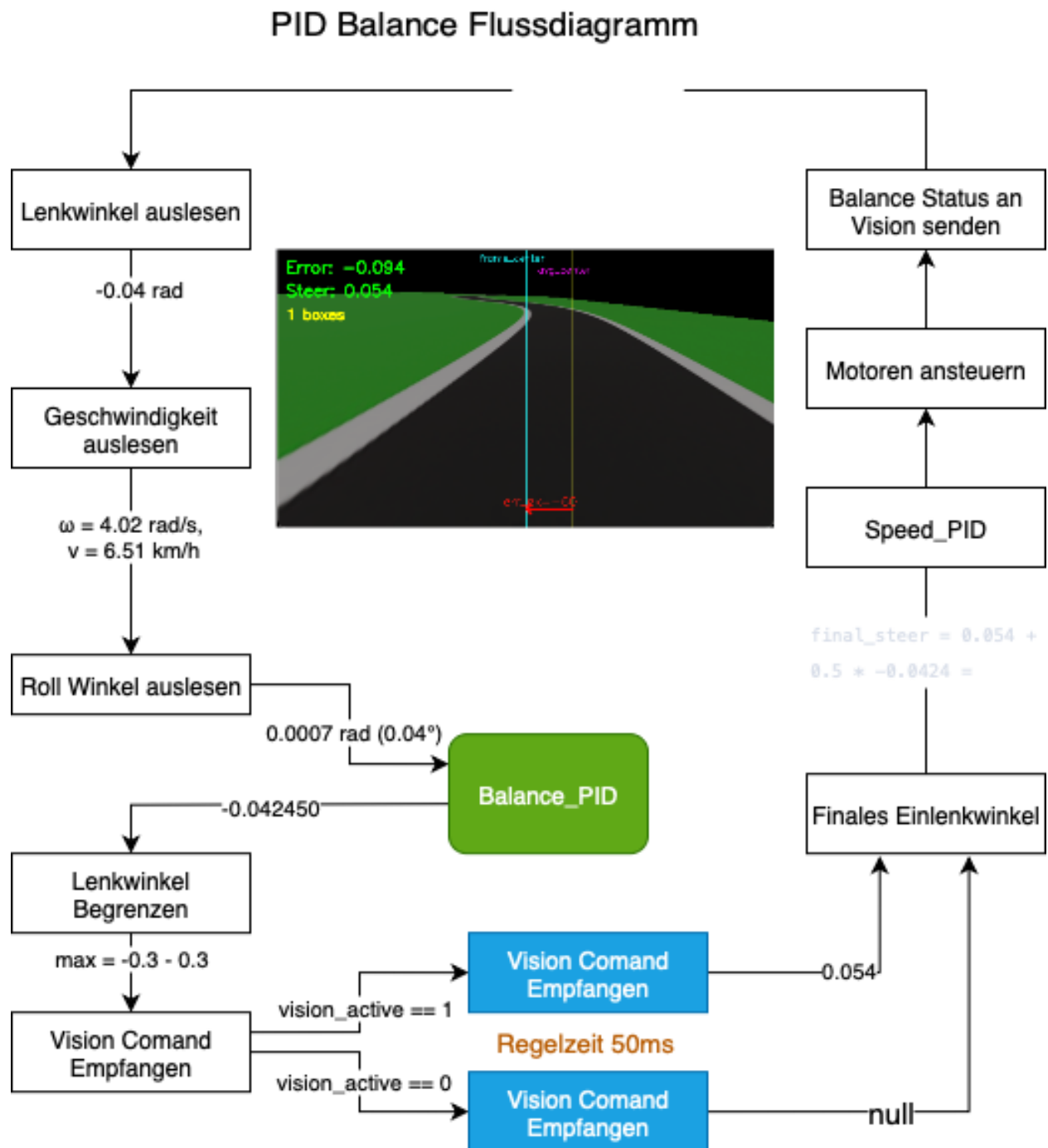


Figure 15: Gesamtregelung.

6 Ergebnisse

6.1 Fahrrad Datenblatt

Die folgenden Abschnitte beschreiben eine in Webots aufgebaute Simulation als digitalen Zwilling des realen Testfahrrads. Ziel ist, Masse-, Trägheits- und Reibungsparameter sowie Sensor-/Aktorschnittstellen so realitätsnah zu spiegeln, dass das Fahrverhalten und die Reglerkopplung mit dem Prototyp übereinstimmen. Auf Basis der ODE-Physik in Webots und der Controller-API (C/Python) wird die in der Hardware verwendete Architektur nachgebildet: eine schnelle innere Balance-Schleife und eine langsamere, vision-gestützte MPC-Spurführung. Eine Supervisor-Instanz ermöglicht reproduzierbare Versuchsreihen, Resets und Logging, sodass Parameterstudien und Kalibrierungen unter identischen Bedingungen erfolgen. Damit dient die Simulation als sichere Vorstufe für reale Fahrversuche und als Referenz für die im Anschluss dokumentierten physikalischen Eigenschaften und Regelungsparameter.

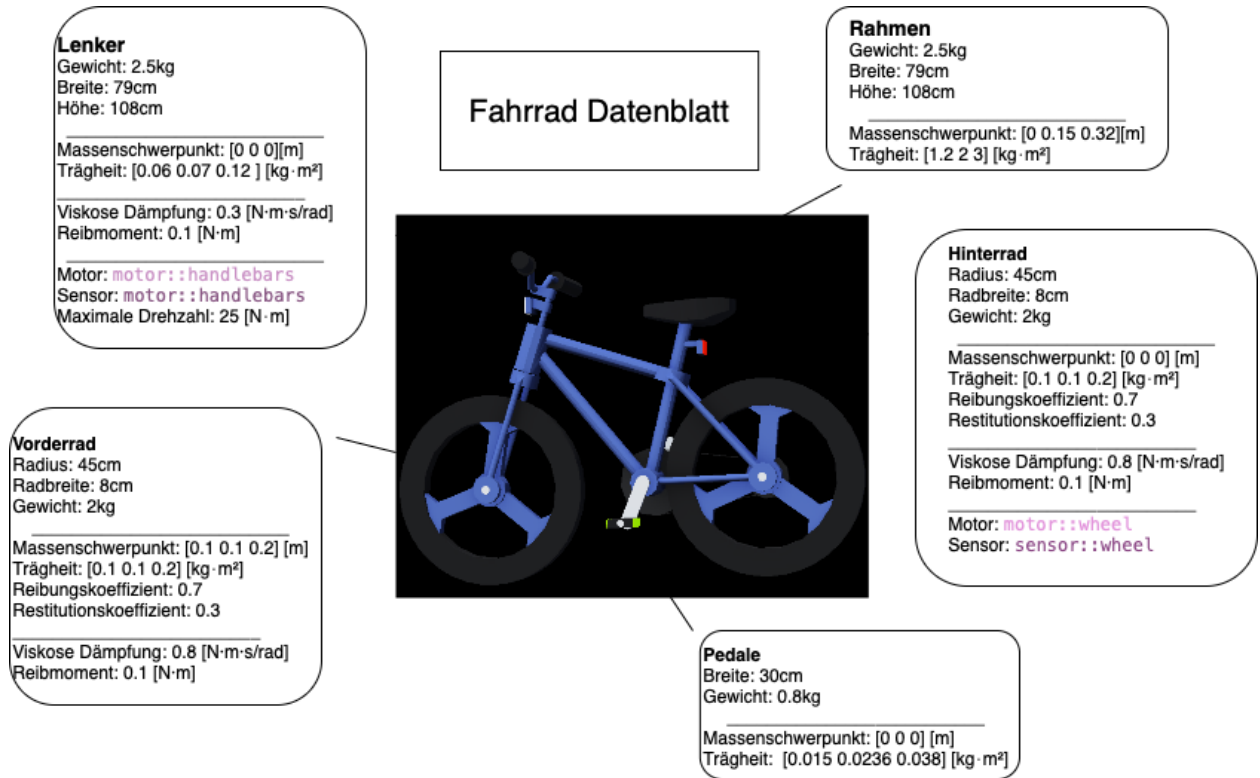


Figure 16: Datenblatt

- **Masse und Schwerpunkt:** Der Rahmen des simulierten Fahrrads wiegt 3,5 kg; zusammen mit den Rädern beträgt die Gesamtmasse ca. 5,1 kg. Der Schwerpunkt liegt etwa 5 cm über dem Boden, was ein stabiles Balanceverhalten begünstigt.
- **Trägheitssensor:** Im Webots-Modell ist der Trägheitssensor als Array in der Form

$$[I_{xx} \ I_{xy} \ I_{xz} \ I_{xy} \ I_{yy} \ I_{yz} \ I_{xz} \ I_{yz} \ I_{zz}]$$

angegeben. Für den Fahrradrahmen ergibt sich ein annähernd diagonal dominanter Trägheitstensor mit $I_{xx} \approx I_{yy} = 0,1$ und $I_{zz} = 0,05 \text{ kg m}^2$. Der kleinere Wert um die Hochachse

(Yaw-Trägheit) erleichtert schnelle Lenkbewegungen; die Off-Diagonaleinträge sind Null, was bedeutet, dass die Hauptträgheitsachsen mit den Achsen des Modells ausgerichtet sind.

- **Räder:** Die beiden Räder haben einen Radius von 5,5 cm und jeweils etwa 0,8 kg Masse. Durch die rotierende Masse entsteht ein kleiner gyroskopischer Stabilisierungs-Effekt (Trägheitsmoment pro Rad ca. $1 \times 10^{-3} \text{ kg m}^2$), der zur Gesamtstabilität beiträgt, aber aufgrund des geringen Werts nur begrenzten Einfluss hat.
- **Reibungsparameter:** Die Radnaben besitzen eine viskose Lagerdämpfung (`dampingConstant` = 0,1) sowie Haftreibung (`staticFriction` = 0,1). Diese Parameter simulieren Lagerverluste und verhindern z. B. ein freies Schwingen oder Durchrutschen der Räder im Stillstand. Zwischen Reifen und Untergrund sorgt ein Coulomb-Reibungskoeffizient von etwa 0,8 für gute Traktion; ein geringer Rückprallkoeffizient von 0,1 begrenzt die Sprungelastizität des Reifenkontakts.

6.2 Teststrecken

6.2.1 Kurvenfahrt

Die Fahrbahn ist als texturierte Ebene von 64m x 64m realisiert; die effektive Spurbreite beträgt an der Referenzlinie etwa 3.6 m. Die Strecke kombiniert eine lange Linkskurve (großer Radius) mit einem engen Rechtsknick und eignet sich damit zur Bewertung von Spurtreue und Balance unter gekrümmter Führung. Sofern nicht anders angegeben, sind für die Balance der *PID*-Satz $K_p=2.0$, $K_i=0.2$, $K_d=0.5$, der zulässige Lenkwinkelbereich $\delta \in [-0.3, 0.3]$ rad sowie $v_{\text{base}}=7 \text{ km/h}$ (Grenzen 5 km/h bis 7 km/h) aktiv. Mechanische Plausibilitätsgrenzen sind mit $\delta_{\text{phys,max}}=1.5 \text{ rad}$ und $\varphi_{\text{roll,max}}=45^\circ$ gesetzt. Für die visuelle Spurführung kann wahlweise YOLO (Konfidenz 0.5) oder eine robuste ROI-Fallback-Erkennung verwendet werden (`roi_top_frac` = 0.5, `roi_height_frac` = 0.06); Lenkbefehle werden durch `max_delta` = 0.001 und `max_steer_cmd` = 0.06 geglättet und begrenzt. Ziel ist die Fahrt von der ersten zur zweiten roten Markierung bei möglichst mittiger Spurhaltung und stabiler Balance.

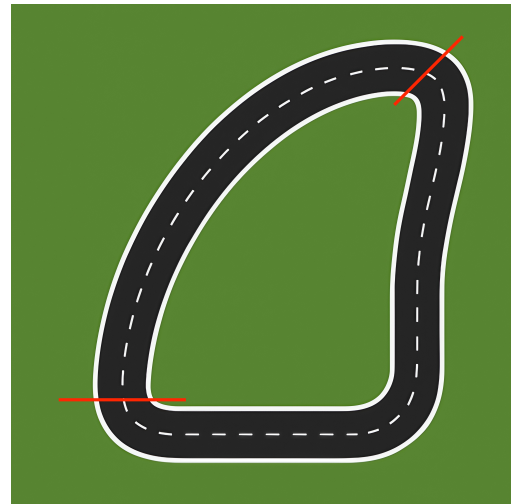


Figure 17: Kurvenfahrt

Einsatz eines vereinfachten Balance-P-Reglers Für eine Baseline wurde der Balance-Regler auf einen reinen P-Anteil gesetzt ($K_p=2.0$, $K_i=0.0$, $K_d=0.0$). Figure 18 zeigt die gemessenen Größen der Fahrt, Figure 19 den zugehörigen ROI-Querfehler und Figure 20 die Trajektorie auf der Strecke.

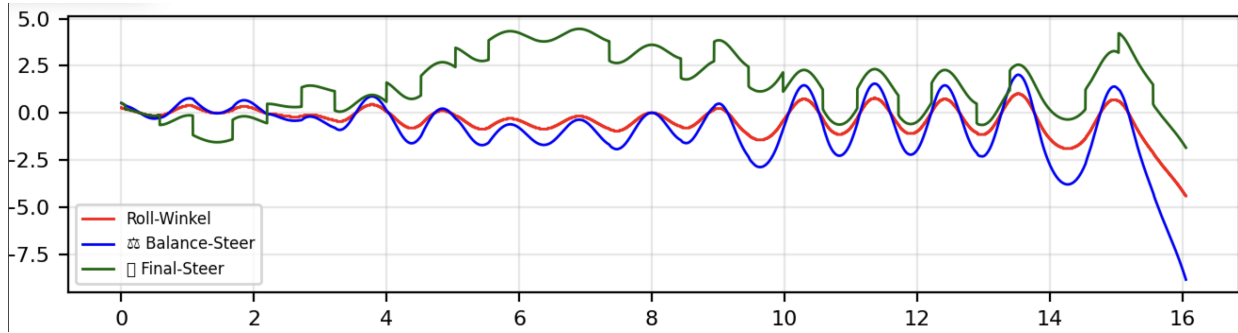


Figure 18: Kurvenfahrt mit reinem P-Regler (Zeitreihen)

Bereits nach wenigen Sekunden setzt ein wachsendes Aufschaukeln ein: Der durch den Rollwinkel induzierte Stellbefehl (`final_steer`) treibt den Lenkeinschlag alternierend nach links/rechts, bis die Phasenreserve erschöpft ist. Infolge der reinen Proportionalrückführung, der Aktuatorbegrenzungen und der unvermeidlichen Latenzen entsteht ein unterdämpftes Verhalten; das Fahrrad kippt nach rund 15 s nach rechts.

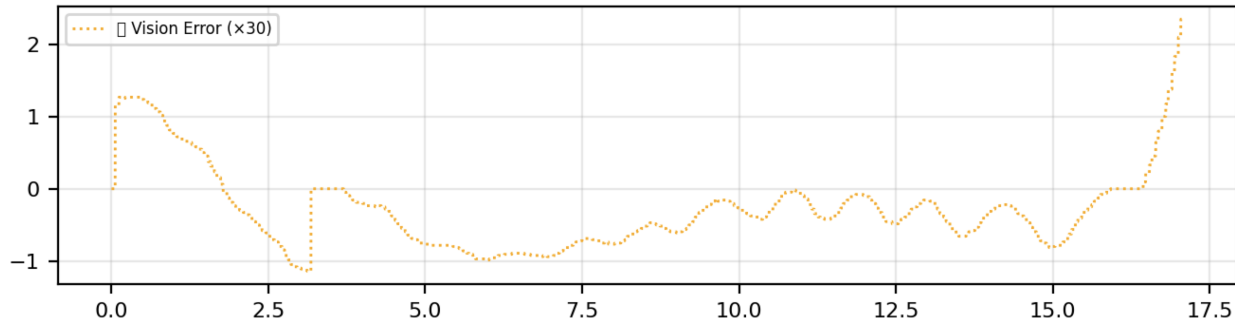


Figure 19: ROI-Querfehler während der P-Regler-Fahrt

Der ROI-Querfehler zeigt eine ausgeprägte Kopplung zum Rollwinkel: Das Pendeln der Balance überträgt sich phasenverschoben auf die seitliche Abweichung. Zur Stabilisierung ist daher ein kleiner Integralanteil (Elimination stationärer Abweichung mit Anti-Windup) und insbesondere ein Derivatanteil (Phasenführung/Dämpfung) erforderlich; in Kombination mit `Rate(max_delta)` und Amplitudenbegrenzung (`max_steer_cmd`) wird das Überschwingen reduziert und ein Kippen verhindert.

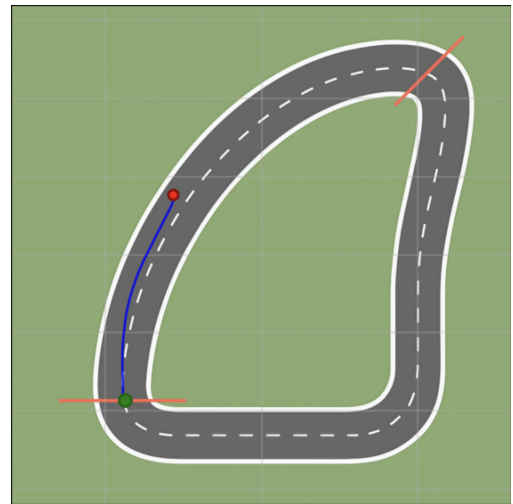


Figure 20: Gefahrene Trajektorie (P-Regler)

Einsatz des erweiterten Balance-PID-Reglers Für diese Versuchsfahrt wurden die erweiterten PID-Parameter $K_p = 2.0$, $K_i = 0.2$ und $K_d = 0.5$ eingesetzt. Die Spurführung erfolgte mittels YOLO-basierter Bildverarbeitung. Während die Parameterwahl eine Verbesserung der Stabilität gegenüber reinen P-Reglern bewirkte, zeigte sich nach etwa 35 s dennoch ein Kippen des Fahrrads, ausgelöst durch abrupte Richtungsänderungen in der vom Vision-Modell vorgegebenen Trajektorie.

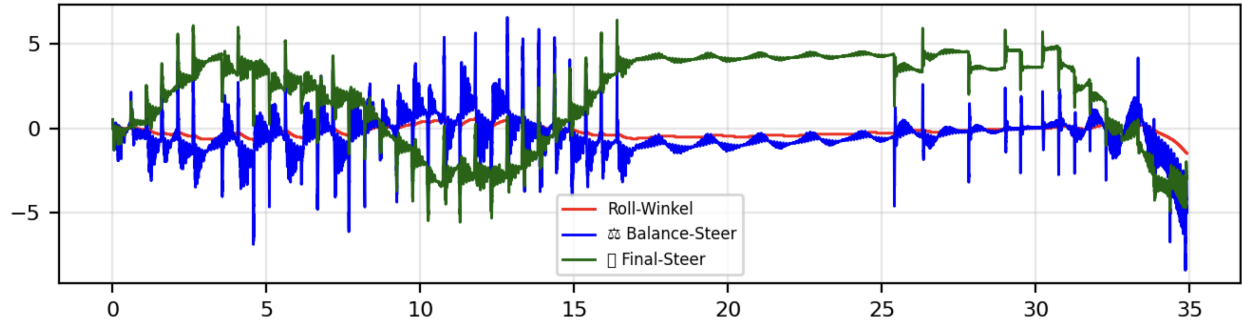


Figure 21: Kurvenfahrt mit PID-Regler und YOLO-Spurführung

Die Fehleranalyse bestätigt, dass die Fahrbahnerkennung zum Zeitpunkt der Versuche noch unzureichend trainiert war. Ab Sekunde 31 steigt der Querfehler abrupt an und fällt kurz darauf wieder ab (Sekunde 34). Dieses unstete Verhalten überträgt sich auf den Rollwinkel, dessen zunehmendes Pendeln schließlich eskaliert und zum Umkippen des Fahrrads nach rechts führt.

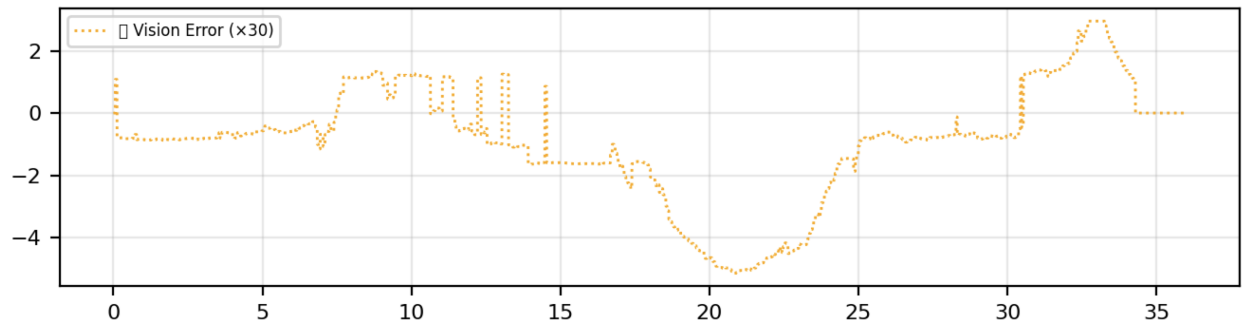


Figure 22: ROI-Querfehler während der PID-Regler-Fahrt (YOLO)

Trotz dieser Limitierungen lässt sich durch die Kombination von Integral- und Differentialanteil eine verbesserte Spurhaltung mit reduzierten Querabweichungen beobachten. Insbesondere werden stationäre Fehler kompensiert und Überschwinger wirksam gedämpft, sodass das Fahrrad über weite Teile der Strecke stabil bleibt – auch bei eingeschränkter Fahrbahnerkennung. Kurz vor dem Ziel versagt die Kompensation jedoch, und das System verliert seine Stabilität. Dies verdeutlicht, dass die Reglererweiterung eine deutliche Verbesserung bewirkt, die Robustheit aber maßgeblich von der Qualität der visuellen Erkennung abhängt.

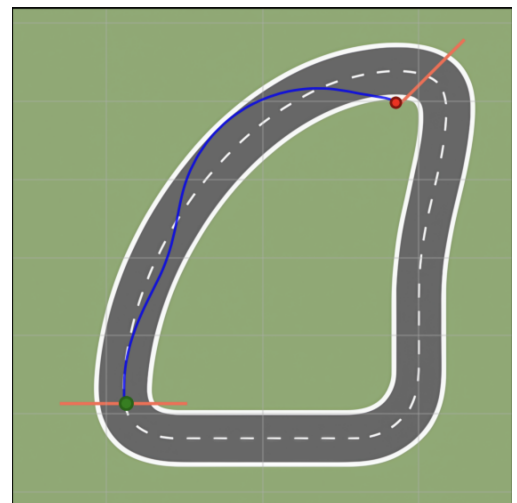


Figure 23: Gefahrene Trajektorie (PID-)

Die Abbildungen 24a und 24b verdeutlichen Limitierungen der *YOLO*-Segmentierung: Abschnitte der Fahrbahn werden nur teilweise bzw. falsch klassifiziert; insbesondere die Trennung zwischen Fahrbahn und Nebenflächen (z.B. Rand-/Grünstreifen) ist inkonsistent. Links (a) ist die Spur weitgehend korrekt maskiert, rechts (b) werden Randbereiche fälschlich als Straße erkannt. Diese Fehlklassifikationen verschieben die geschätzte Spurmitte, erzeugen sprunghafte Querfehler und können instabile Lenkbefehle auslösen.

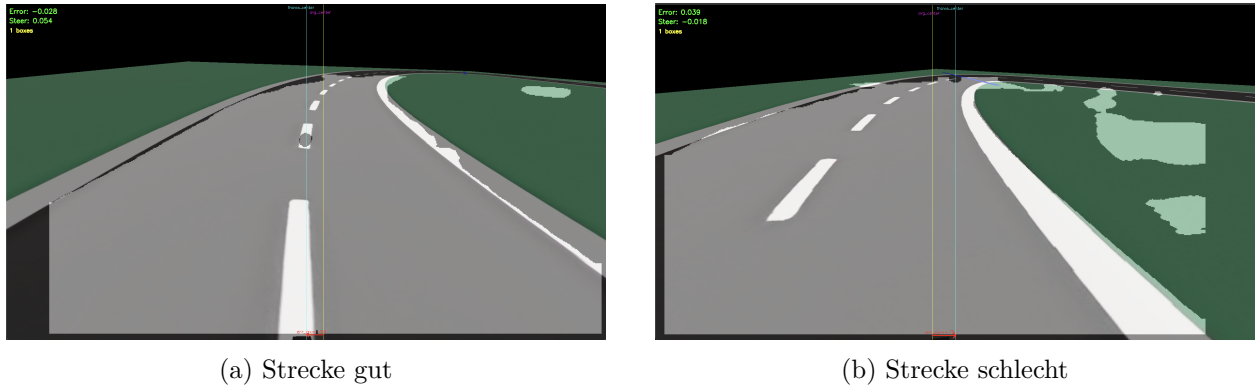


Figure 24: Strecke gut und schlecht nebeneinander.

Einsatz der ROI-Fallback-Erkennung In dieser Versuchsfahrt wurde die ROI-Fallback-Erkennung eingesetzt, um die in Abschnitt 4.3.1 beschriebenen Schwächen der *YOLO*-Segmentierung zu umgehen. Die Pipeline nutzt klassische Bildverarbeitung innerhalb eines definierten Bildausschnitts (ROI): farbbasierte Segmentierung der Fahrbahn und Kantenfilter werden kombiniert, die Spurmittellinie wird aus der ROI-Maske abgeleitet und zeitlich geglättet. Sämtliche Reglerparameter (*PID*) sowie die übrigen Versuchsbedingungen entsprechen exakt denen der *YOLO*-Konfiguration; die beobachteten Unterschiede sind daher allein auf die Wahrnehmung (Erkennung) zurückzuführen.

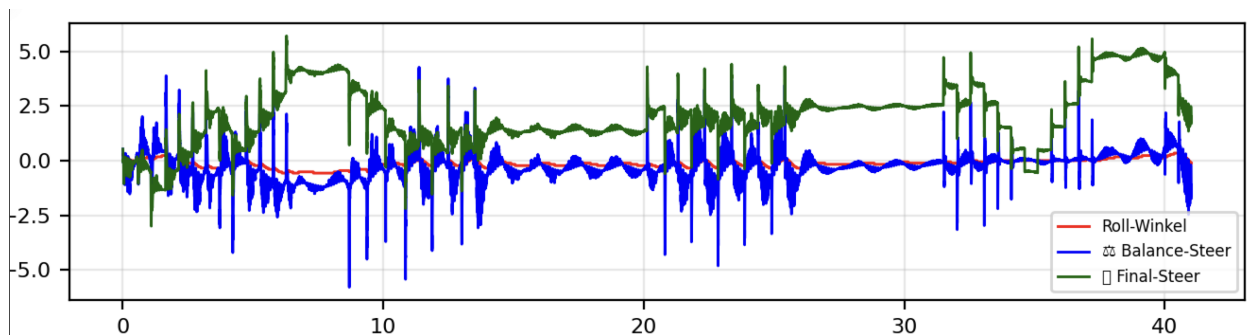


Figure 25: Kurvenfahrt mit PID-Regler und ROI-Fallback-Erkennung

Die Ergebnisse in Figure 25 zeigen einen deutlichen Qualitätsgewinn gegenüber der *YOLO*-Erkennung: sprunghafte Querfehler treten praktisch nicht mehr auf, und das Fahrrad fährt über weite Strecken stabil. Kurzzeitige Anregungen (z. B. zwischen Sekunde 20 und 25) führen zwar zu einer temporären Erhöhung der Stellgrößen, werden jedoch vom PID-Regler sauber kompensiert. Der anfängliche Anstieg des *Final-Steer* ist auf das Einfädeln in die Spur zurückzuführen; nach kurzer Transiente regelt sich das System auf einen kleinen stationären Fehler ein.

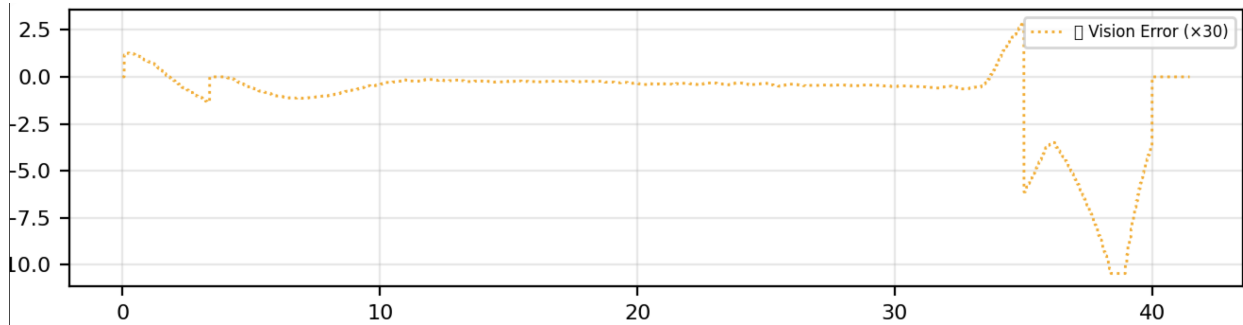


Figure 26: ROI-Querfehler während der PID-Regler-Fahrt (ROI)

Im Fehlerverlauf (Figure 26) ist außerdem ersichtlich, dass ab etwa Sekunde 35 die (rote) Schwelle zur scharfen Kurve überschritten wird und der Querfehler ansteigt. Dieses Verhalten ist konsistent mit den in der Regelkette gesetzten Lenkwinkel- und Ratenbegrenzungen (`max_steer_cmd`, `max_delta`): Bei hoher Streckenkrümmung gerät der Regler kurzzeitig in Sättigung, bevor der Fehler wieder abgebaut wird. Die zugehörige Trajektorie (Figure 27) belegt dennoch eine insgesamt mittige und ruhige Spurführung; Abweichungen konzentrieren sich auf den Scheitel der Kurve und bleiben begrenzt.

Fazit. Die ROI-Fallback-Erkennung erhöht die Robustheit der Spurführung, indem sie Ausreißer in der visuellen Messung reduziert und so die Kopplung zum Balance-Regelkreis entlastet. Ihre Wirksamkeit ist allerdings an Szenen mit gut trennbaren Fahrbahn-/Nebenflächen gebunden; für variable Beleuchtung, Texturen oder komplexe Hintergründe ist eine verbesserte (oder zusätzlich trainierte) lernbasierte Segmentierung weiterhin wünschenswert.

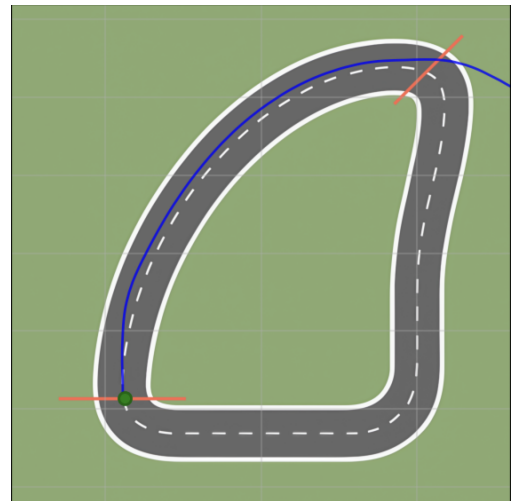


Figure 27: Gefahrene Trajektorie (PID-Regler, ROI)

6.2.2 S-Kurve

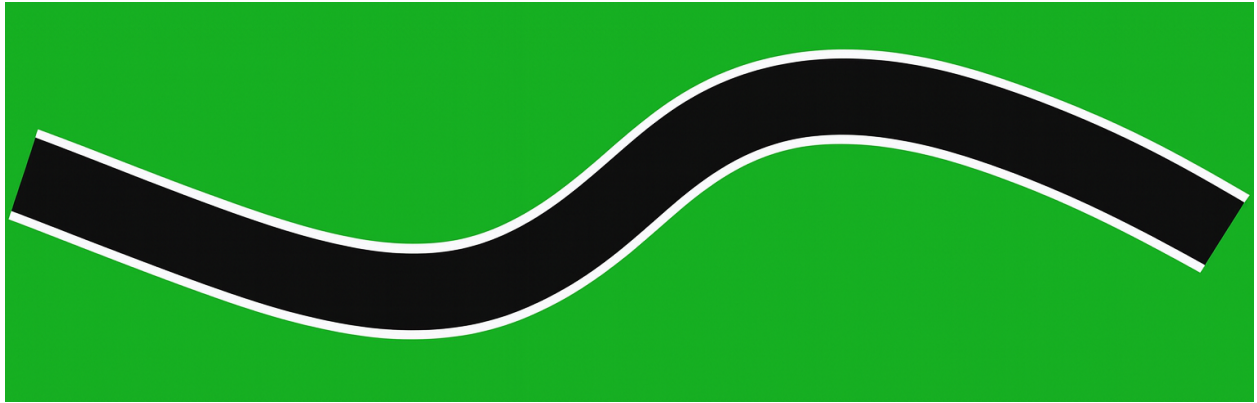


Figure 28: S-Kurve

Die Versuchsstrecke besteht aus einer etwa 100 m langen S-Kurve mit einer Fahrbahnbreite von rund 5,5 m. Das Szenario dient der Bewertung der Kopplung aus innerer Balance-Regelung (PID) und äußerer Spurführungsregelung (visionbasierter MPC) unter schnellem Krümmungswechsel. Für den Balance-PID wurden in diesem Versuch $K_p = 2.0$, $K_i = 0.05$ und $K_d = 0.2$ verwendet; im Vergleich zu anderen Strecken erwies sich hier eine reduzierte Gewichtung von Integral- und Derivatanteil als ausreichend, da die Balance an den Kurveneingängen sonst zu stark angeregt wird. Die innere Schleife arbeitet mit 2 ms Abtastzeit, die Vision-/MPC-Schleife mit 50 ms (20 Hz).

Einsatz des erweiterten Balance-PID mit YOLO-basierter Spurführung Wie bereits in den vorhergehenden Abschnitten gezeigt, ist die YOLOv8-Segmentierung die zentrale Wahrnehmungskomponente der äußeren Schleife. In der S-Kurve zeigt sich jedoch, dass die Segmentierung beim ersten starken Krümmungswechsel zeitweise unzuverlässig ist. Figure 29 illustriert, dass das System die Fahrbahn bereits nach der ersten Kurve partiell verliert; der MPC erhält damit nur intermittierend valide Referenzen. Formal lässt sich die Lenkvorgabe als

$$u_{\text{steer}}(t) = u_{\text{bal}}(t) + g_{\text{vis}}(t) u_{\text{mpc}}(t)$$

schreiben, wobei u_{bal} der schnelle Balance-Anteil, u_{mpc} die Vision-/MPC-Vorgabe und $g_{\text{vis}} \in [0, 1]$ ein Qualitätsgewicht ist. Fällt die Segmentierung aus, wird effektiv $g_{\text{vis}} \approx 0$ und die Fahrtrichtung wird nur noch durch den Balance-Regler gehalten.

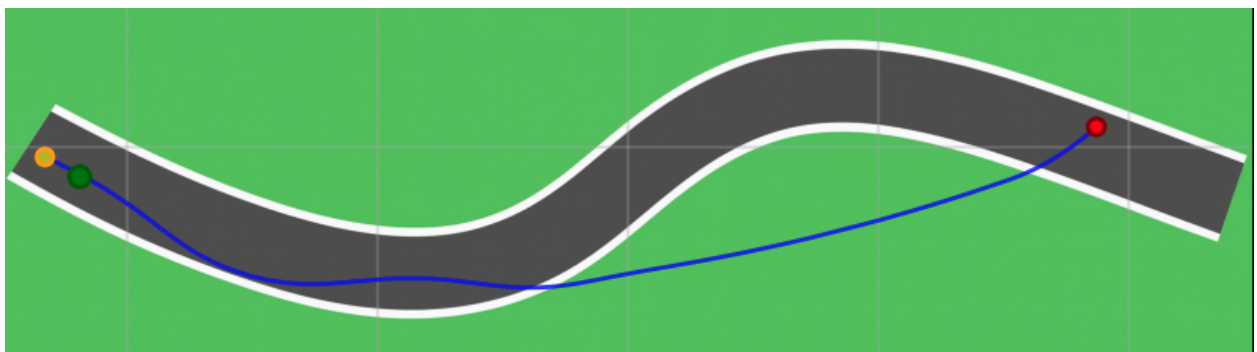


Figure 29: S-Kurve mit YOLO-Regler

Die Zeitverläufe in Figure 30 zeigen zu Versuchsbeginn eine unnötig starke Rechtsabbiegung mit anschließender Gegenlenkung nach links (um ca. 15 s). Diese Sequenz ist kein Folgeeffekt der Balance, sondern ein Artefakt der unstetigen Spurreferenz: das Segmentierungsmodell liefert kurzzeitig inkonsistente Mittellinien, die der MPC korrektiv verfolgt. Zwischen etwa 25 s und 40 s fällt die Spurdetektion stellenweise aus; ab ~ 30 s ist u_{mpc} mehrfach Null, sodass im **final_steer** praktisch nur u_{bal} wirksam bleibt. Erst ab ca. 45 s wird die Spur wieder sicher akquiriert und der MPC fordert erneut ein Rechtsabbiegen an.

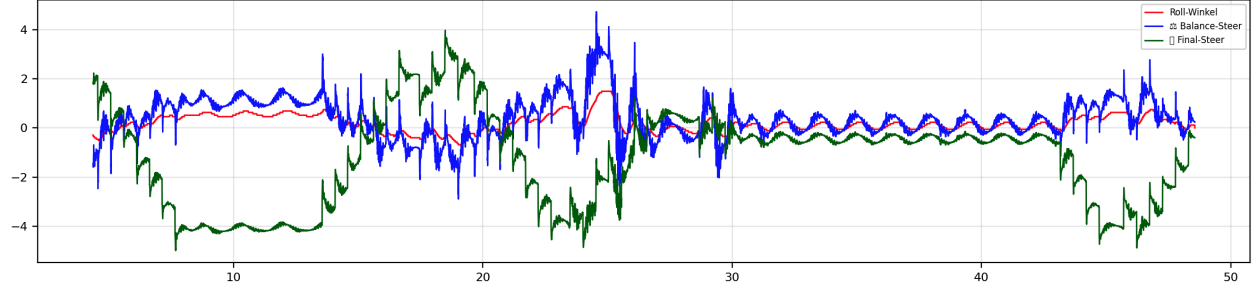


Figure 30: S-Kurve: ausgewählte Zeitreihen (Rollwinkel, Balance-Soll, **final_steer**)

Der Fehlerplot in Figure 31 bestätigt diese Diagnose: Der visionbasierte Querfehler $\tilde{e}_y$ ist im Intervall 25 s–40 s „nahe Null“ *nicht* aufgrund guter Spurhaltung, sondern weil kein verwertbares Signal vorliegt (Maskenfragmentierung bzw. Konfidenz < Schwellwert). Außerhalb der Aussetzer ist die Spurhaltung erkennbar hochfrequent und weist große Spitzen auf. Diese schnellen Lenkänderungen regen den Balance-Regler an (Kopplung zwischen Lenkdynamik und Rollwinkel), was die Rolldämpfung belastet und die Reserven gegenüber Störungen reduziert.

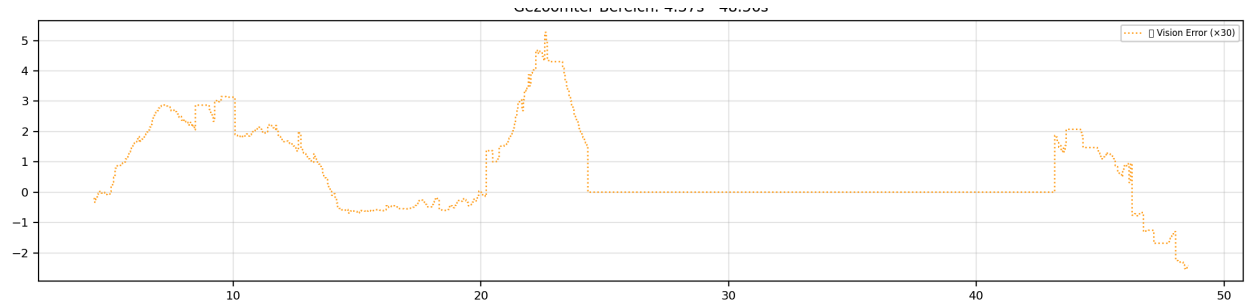


Figure 31: S-Kurve: visionbasierter Querfehler $\tilde{e}_y$

Fazit: Über beide Strecken hinweg (S-Kurve, Kurvenfahrt) zeigt die YOLO-basierte Spurführung die gleiche Schwäche: In stark gekrümmten Abschnitten liefert die Segmentierung intermittierend oder verspätet valide Spurmessungen. In der S-Kurve äußert sich dies als unnötige Anfangskorrektur (Rechts→Links, ~ 15 s) sowie als scheinbar „guter“ Querfehler nahe Null zwischen 25–40 s, der tatsächlich auf Aussetzer der Spurerkennung zurückgeht (vgl. Figure 29, Figure 30, Figure 31). In der Kurvenfahrt tritt das gleiche Muster ausgeprägter auf: sprunghafter Querfehleranstieg ab ~ 31 s und nachfolgendes Kippen trotz PID-Dämpfung, ausgelöst durch inkonsistente bzw. fehlerhafte Trajektorienvorgaben (Figure 21, Figure 22, Figure 23).

Regelungstechnisch führt die unstete Spurreferenz zu hochfrequenten Lenksignalen, die den Balancekreis anregen und die Rolldämpfung reduzieren. Demgegenüber zeigt der ROI-Fallback in

beiden Szenarien eine deutlich ruhigere Dynamik: Die Trajektorien bleiben mittig, Aussetzer entfallen, und der Querfehler weist die erwartete S-Kurven-Signatur ohne impulsartige Ausreißer auf (Figure 32, Figure 33, Figure 34).

Einsatz des erweiterten Balance-PID-Reglers mit ROI-Fallback-Erkennung

Figure 32 zeigt gegenüber der YOLO-Variante eine deutlich stabilere Spurhaltung über die gesamte S-Kurve. Die Trajektorie bleibt näher an der Spurmitte, unnötige großamplitudige Korrekturen entfallen weitgehend. Lediglich zu Beginn ist – analog zur YOLO-Fahrt – ein kurzer Rechts → Links-Korrekturzug zu erkennen, der als Einschwingvorgang der äußeren Regelung zu werten ist. Danach verläuft die Fahrt konsistent und ohne Abreißen der Spurinformation.

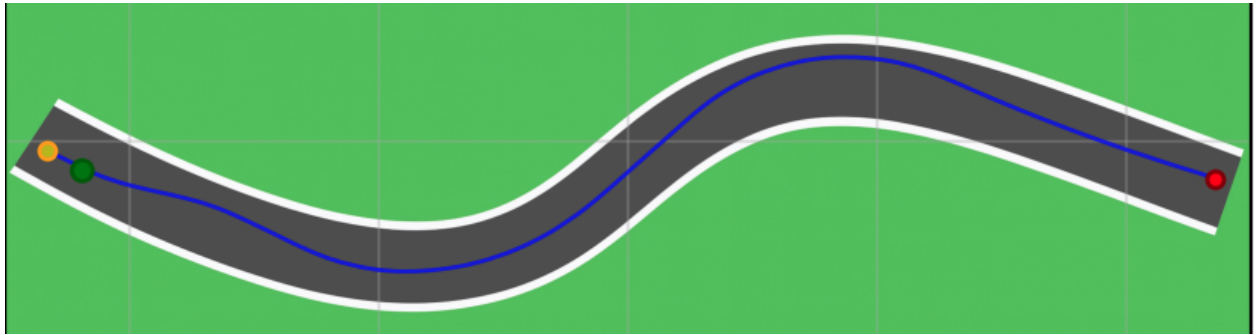


Figure 32: S-Kurve mit ROI-Regler

Die Zeitreihen in Figure 33 belegen dies während des Richtungswechsels zwischen ca. 25–35 s: Der Lenkbedarf steigt erwartungsgemäß am Krümmungsmaximum an, zugleich bleibt der `final_steer` frei von hochfrequenten Spitzen. Um etwa 35 s ist ein temporär größeres Pendeln des Rollwinkels sichtbar, das jedoch begrenzt bleibt und nach Passieren des zweiten Kurvenbogens rasch abklingt (wiederhergestellte Dämpfung, keine Aufschaukelung). Insgesamt resultiert eine klarere Kopplung zwischen Lenkbefehl und Fahrzeuglage ohne die bei YOLO beobachteten Artefakte.

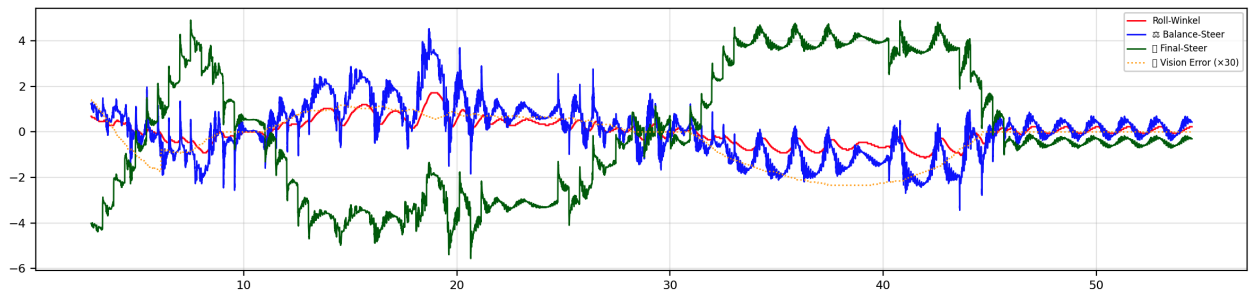


Figure 33: S-Kurve mit ROI-Regler Daten

Der Querfehlerverlauf in Figure 34 ist deutlich glatter und zeigt die erwartete S-Kurven-Signatur mit wohldefiniertem Vorzeichenwechsel in den beiden Bögen. Im Gegensatz zur YOLO-Fahrt treten keine Null-Plateaus durch Signalaussetzer und kaum impulsartige Ausreißer auf; die Varianz des Fehlers ist reduziert, und das Anfangsbias entspricht der kurzfristigen Suche nach der Spurmitte. Das ruhigere Fehlersignal führt zu geringerer Anregung des Balancekreises und damit zu einer robusteren Gesamtfahrt.

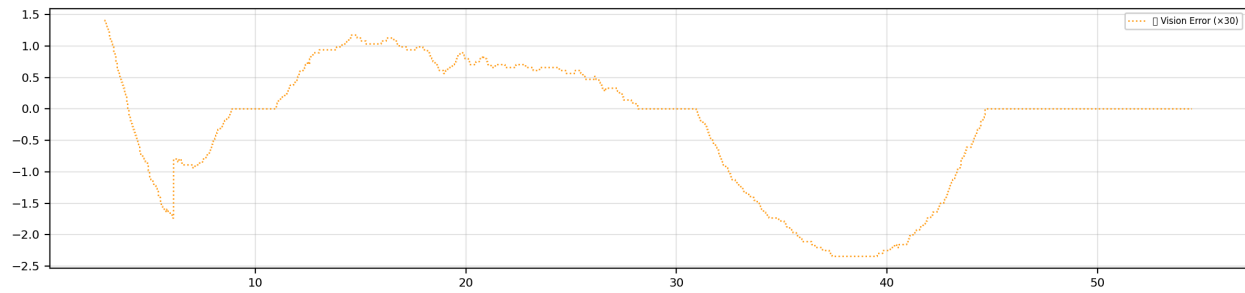


Figure 34: S-Kurve mit ROI-Regler Error

Fazit: Mit ROI-Fallback verbessert sich die Spurtreue über die gesamte Strecke, die Zahl und Amplitude unnötiger Korrekturen sinkt, Aussetzer der Spurinformaton werden vermieden und die Balance bleibt trotz Krümmungswechseln gut beherrschbar. Die Resultate deuten auf eine höhere Robustheit der Regelkette in kurvigen Abschnitten hin.

6.2.3 Gerade Strecke mit Seitenwind

Die Versuchsumgebung ist eine $64\text{ m} \times 64\text{ m}$ große Karte mit einer $9,3\text{ m}$ breiten, geraden Fahrbahn. Entlang der Geraden sind drei etwa 20 m lange Segmente definiert (vgl. Figure 35), die als Windkanäle mit seitlichem Anströmfeld modelliert sind. Die Windrichtung wechselt zwischen den Segmenten (rechts $\rightarrow$ links $\rightarrow$ rechts). Die Fahrgeschwindigkeit wird konstant auf 7 km/h geregelt, die Reifenbreite beträgt 2 cm (im Gegensatz zu 6 cm bei der kombinierten Regelung). Die Kopplung zum Strömungsfeld erfolgt über die in *Webots* aktivierten aerodynamischen Drag-/Lift-Modelle; die Seitenwinde werden durch drei **Fluid-Volumina** mit Luftparametern $\rho = 1,225\text{ kg m}^{-3}$ und $\mu = 1,81 \times 10^{-5}\text{ Pas}$ realisiert, jeweils mit einer achsenparallelen **boundingObject**-Box von $64 \times 21 \times 20\text{ m}$ und den Translationen $(0, 21, 0)$, $(0, 1,86, 0)$ und $(0, -19,33, 0)$. Das Anströmfeld ist stationär und homogen; pro Segment wird **streamVelocity** als $(-v_w, 0, 0)$, $(+v_w, 0, 0)$ bzw. $(-v_w, 0, 0)$ gesetzt, wobei in den Auswertungen $v_w \in \{0,5, 1,0\}\text{ m/s}$ verwendet wird (Stresstests bis 2 m/s sind möglich). Geregelt wird in der inneren Schleife die Fahrzeugbalance (PID, Abtastzeit $\approx 2\text{ ms}$); die äußere Spurführung (YOLO/ROI, $\approx 50\text{ ms}$) hält die Mittellinie und wirkt hier primär als langsame Vorsteuerung. Sofern nicht anders angegeben, verwenden wir die Balance-PID-Parameter $K_p = 2,0$, $K_i = 0,05$ und $K_d = 0,2$ (Baseline).

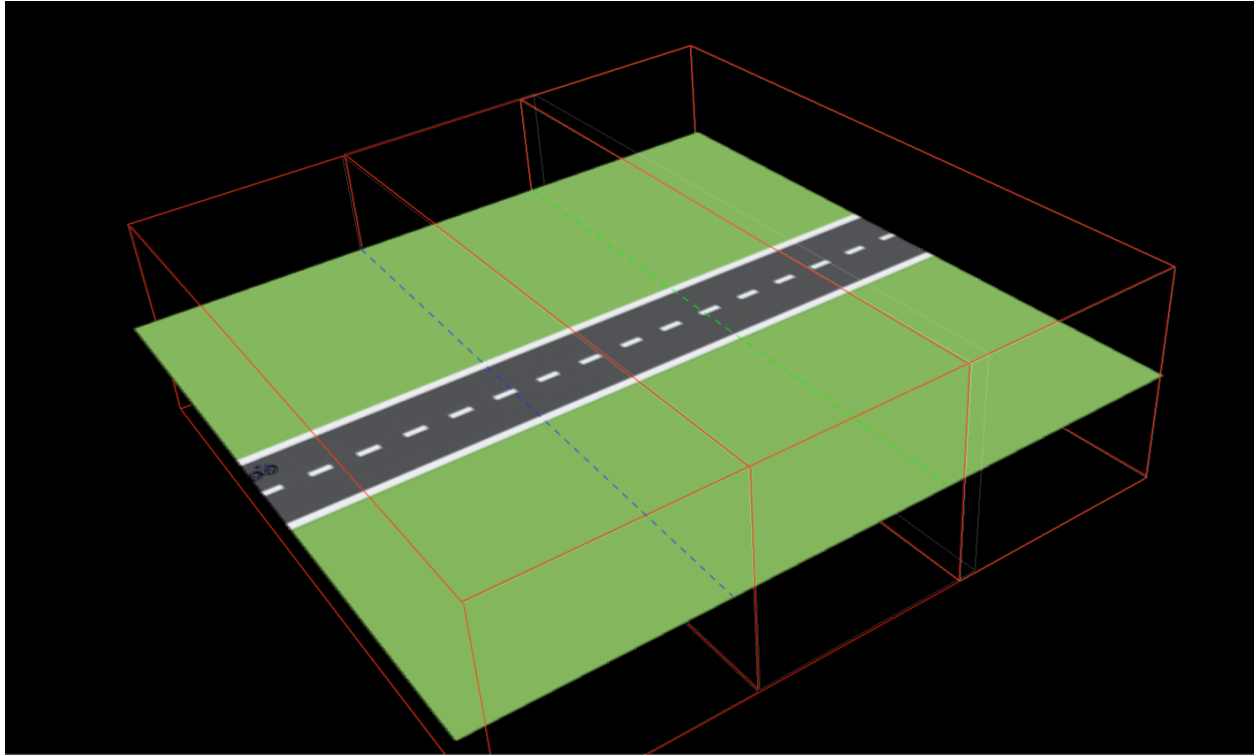


Figure 35: Versuchsaufbau: Gerade Strecke mit drei seitlichen Windkanälen (rote Drahtgitterboxen). Die Fahrbahnmitte dient als Referenz für die Spurführung.

Fall A: Seitenwind $0,5\text{ m/s}$ (Baseline-Regler) Bei moderatem Seitenwind ($0,5\text{ m/s}$) zeigt die Trajektorie in Figure 36 eine seitliche Auslenkung in Windrichtung und anschließend eine Rückkehr zur Ausgangsspur, nachdem das Fahrzeug den dritten Windkanal passiert hat. Die maximale laterale Abweichung beträgt dabei bis zu etwa 3 m , bleibt jedoch ohne Instabilität.



Figure 36: Trajektorie bei 0,5 m/s Seitenwind: Blaue Linie = Fahrzeugspur; Start (grün) und Endpunkt (rot). Die Auslenkung in den ersten beiden Windsegmenten wird nach dem dritten Segment wieder abgebaut.

Die Zeitreihen in Figure 37 verdeutlichen den Zusammenhang zwischen Rollwinkel und Lenkkommandos: Zu Beginn ist eine deutliche Rechtsneigung (positiver Rollwinkel) erkennbar, die der Regler mit gegenläufigen Balance- und Lenkimpulsen kompensiert. Die Signale bleiben beschränkt, der stationäre Versatz wird durch den Integralanteil weitgehend eliminiert.

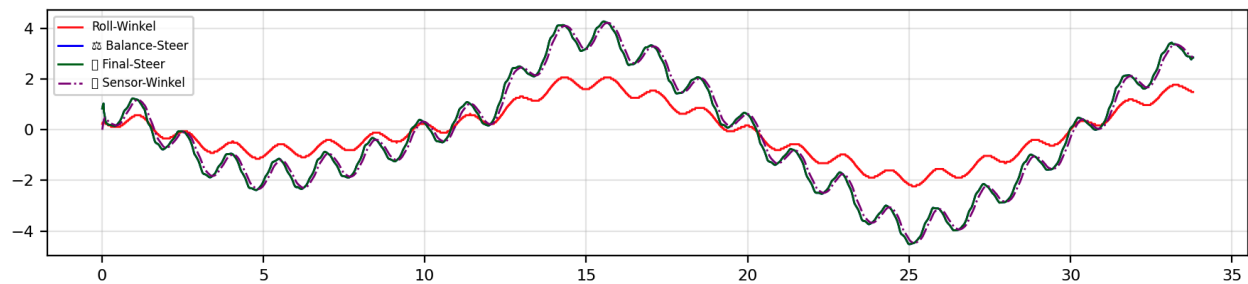


Figure 37: Zeitreihen bei 0,5 m/s Seitenwind: Rollwinkel, Balance-Steer und finales Lenkkommando. Die Richtung der Seitenböe spiegelt sich in der Vorzeichenfolge des Rollwinkels wider; der D-Anteil dämpft die schnelle Nick-/Roll-Dynamik.

Fall B: Seitenwind 1,0 m/s (Baseline-Regler) Erhöhen wir die Windstärke auf 1,0 m/s, vergrößert sich die laterale Abweichung deutlich (Figure 38). Die Zeitreihen in Figure 39 zeigen, dass der Rollwinkel nach etwa 7 s divergiert; das Fahrzeug kippt um. Interpretation: Die Störmomente aus Seitenwind übersteigen mit den Baseline-Gains die stellbare Gegenwirkung, der Regler läuft in Sättigungen (Lenkraten- und Amplitudenlimit), während der I-Anteil die anhaltende Störung nicht schnell genug kompensiert.

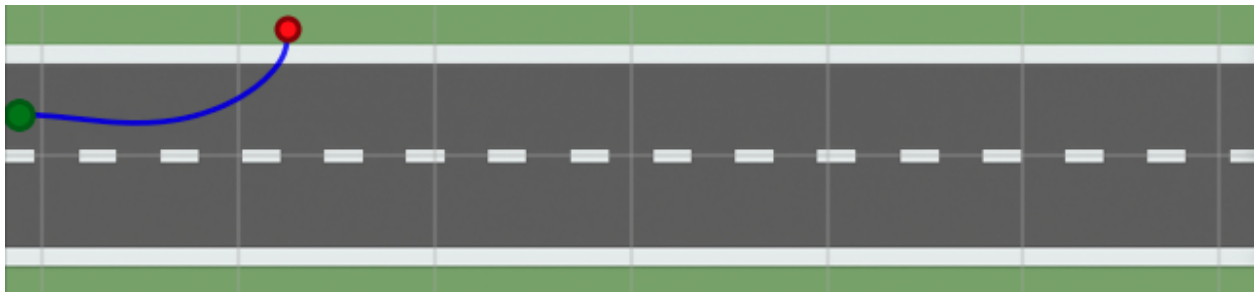


Figure 38: Trajektorie bei 1,0 m/s Seitenwind (Baseline-Gains): starke Abdrift nach rechts mit vorzeitigem Abbruch infolge Umkippens.

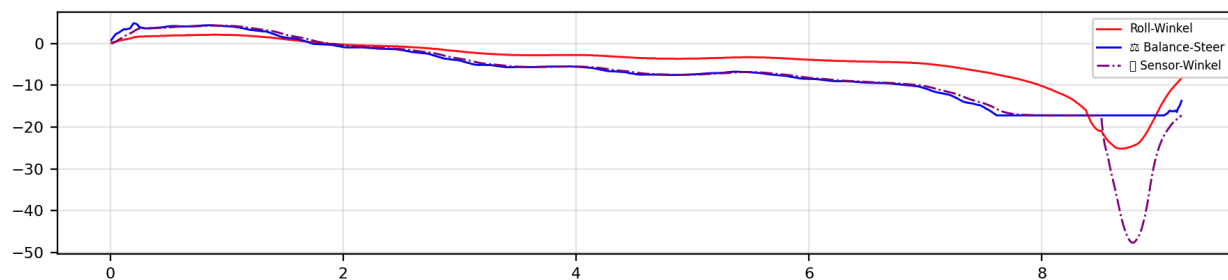


Figure 39: Zeitverlauf bei 1,0 m/s (Baseline-Gains): ansteigende Rollauslenkung und instabiles Verhalten nach ≈ 7 s.

Fall C: Gain-Erhöhung für die Balance-Regelung Zur Verbesserung der Robustheit gegenüber konstanten Seitenkräften erhöhen wir die Balance-Gains auf $K_p = 5,0$, $K_i = 0,2$ und $K_d = 0,4$. Der größere P-Anteil erhöht die Querstabilität (“Steifigkeit” gegenüber Winkelfehlern), der höhere D-Anteil liefert zusätzliche Dämpfung und verhindert Phasennachlauf, und der verstärkte I-Anteil reduziert den stationären Versatz durch die windinduzierte Momentenbilanz.

Die Trajektorie in Figure 40 bleibt nun über die gesamte Strecke beherrscht; die laterale Abweichung wächst nicht mehr unbeschränkt an. Gleichzeitig zeigen die Zeitreihen in Figure 41 ein stärkeres, aber kontrolliertes Schwingen des finalen Lenkkommandos: Die erhöhte Verstärkung erkaufte Stabilität durch höhere Stellaktivität, bleibt aber innerhalb der Aktuatorgrenzen.

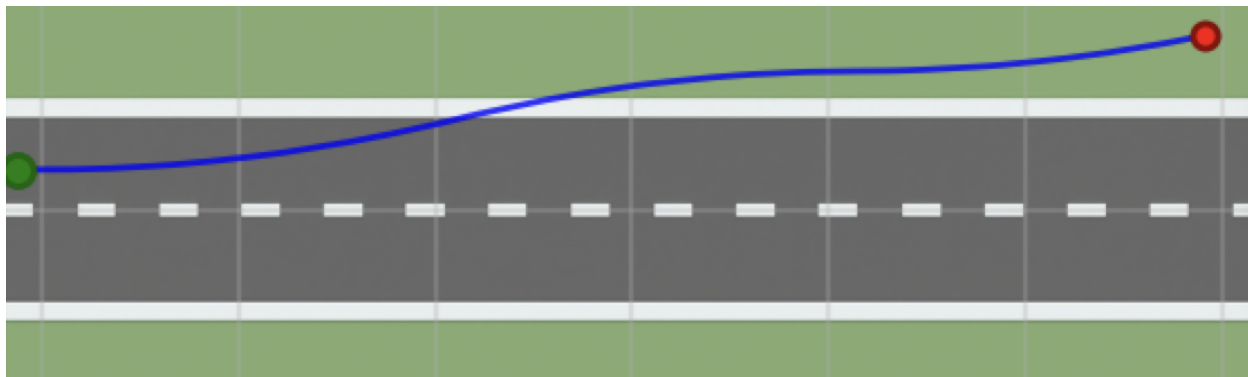


Figure 40: Trajektorie bei 1,0 m/s Seitenwind mit erhöhten Balance-Gains: stabilisierte Fahrt ohne Umkippen; die Abdrift wird begrenzt und anschließend abgebaut.

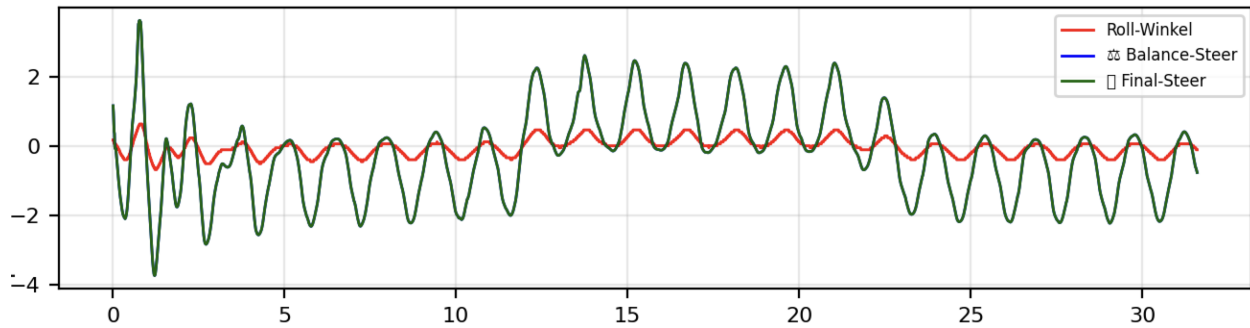


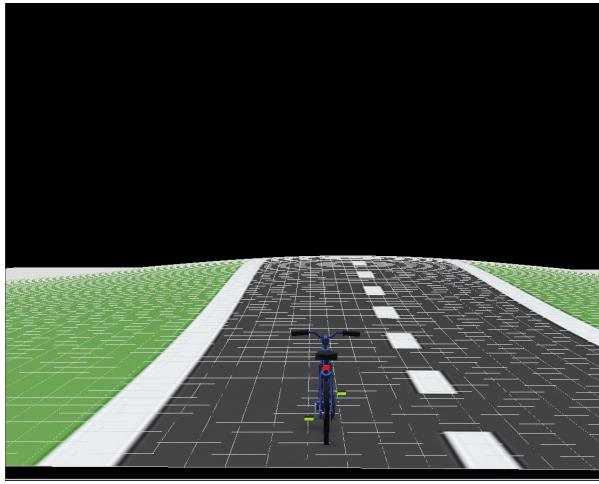
Figure 41: Zeitreihen bei 1,0 m/s mit erhöhten Gains: gedämpfter Rollwinkelverlauf; höhere, aber tolerierbare Lenkaktivität (keine Sättigung, keine Divergenz).

Einordnung und Grenzen Die Ergebnisse bestätigen die in dieser Arbeit gewählte Zwei-Ebenen-Architektur: Die schnelle Balance-Regelung muss Seitenwindstörungen primär aufnehmen; die langsamere Spurführung korrigiert den verbleibenden Versatz. Für moderate Böen ($\approx 0,5 \text{ m/s}$) genügt die Baseline-Parametrierung; bei stärkeren Böen ($\approx 1,0 \text{ m/s}$) ist eine Erhöhung von K_p , K_i und K_d erforderlich, um Umkippen zu vermeiden. Allerdings steigen dabei Stellaufwand und Oszillationstendenzen.

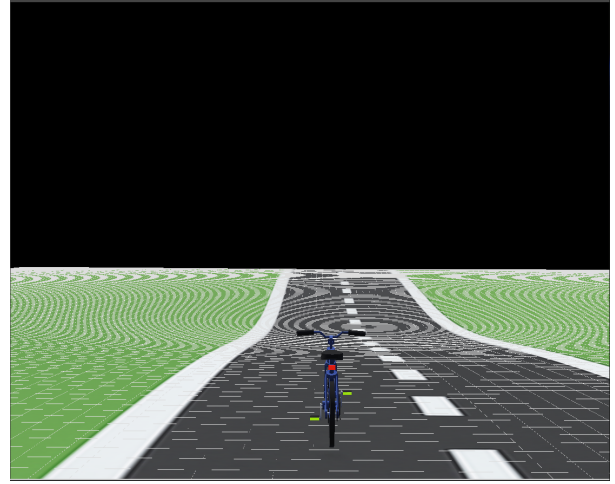
Ab einer gewissen Windstärke liefern reine PID-Anpassungen abnehmenden Zusatznutzen. Sinnvolle Erweiterungen sind (i) Anti-Windup und Lenkratenbegrenzung mit prädiktiver Phasenreserve, (ii) eine Störgrößen-Vorsteuerung (z. B. auf Basis eines Windschätzers oder der Querbeschleunigung), (iii) Gain-Scheduling über Geschwindigkeit und geschätzte Windlast sowie (iv) physikalische Anpassungen (größere Reifenbreite, tieferer Schwerpunkt, höhere Rotationsenergie der Räder). Diese Maßnahmen adressieren die beobachtete Kopplung von Roll- und Querfehler und erhöhen die Robustheit über das reine Tuning hinaus.

6.2.4 Gerade Strecke mit Höhenunterschied

In diesem Abschnitt bewerten wir die Stabilität der Balanceregung auf einer geraden Teststrecke mit definiertem Höhenprofil. Untersucht werden zwei archetypische Szenarien: das Überfahren eines Hügels sowie das Durchfahren einer Senke („Tunnel“), vgl. Figure 42. Soweit nicht anders angegeben, verwenden wir die Baseline-Parameter der Balance-PID-Regelung aus Kapitel 5.2.3. Die Höhendifferenz M wird schrittweise variiert (0,5 m, 1 m, 2 m), um den Einfluss auf Roll- und Spurverhalten systematisch zu untersuchen.



(a) Hügel



(b) Tunnel

Figure 42: Höhenprofile der Teststrecke: Hügel (a) und Senke/Tunnel (b).

Fall A: Senke, $M = 1,0\text{ m}$ Die Trajektorie in Figure 43 zeigt über weite Teile eine nahezu ideale Spurhaltung; geringe laterale Abweichungen treten erst beim Ausfahren aus der Senke auf. Die Zeitreihen in Figure 44 bestätigen dieses Bild: Der Rollwinkel bleibt klein und wird vom Balance-Regler zuverlässig kompensiert. Bei $t \approx 2,5\text{ s}$ ist eine kurzzeitige Amplitudenerhöhung sichtbar, die rasch ausklingt. Da die tiefste Stelle des Profils mittig liegt und die Ränder auf der Referenzhöhe (Nullhöhe) verlaufen, wirken die seitlichen Anteile der Hangabtriebskraft symmetrisch, was die beobachtete Stabilität der Talfahrt erklärt.



Figure 43: Trajektorie durch die Senke (Fall A, $M = 1,0\text{ m}$).

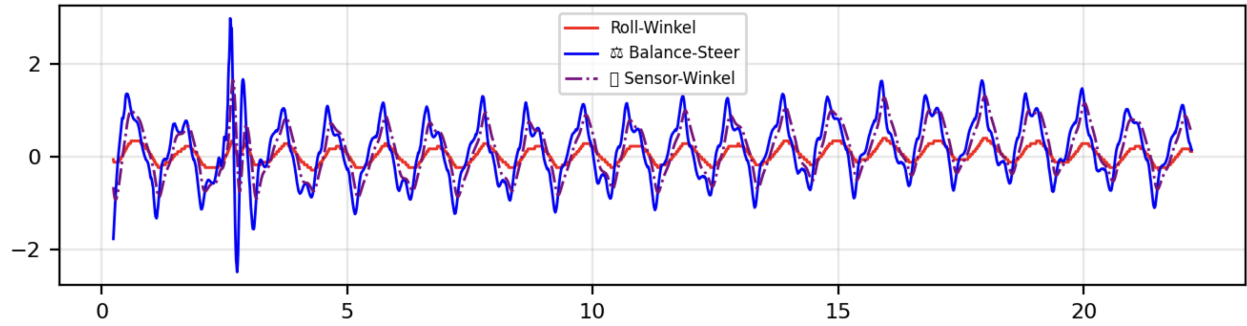


Figure 44: Zeitverläufe Rollwinkel/Balancelenkung (Fall A, $M = 1,0$ m).

Fall B: Senke, $M = 2,0$ m (Baseline-Gewichte) Wird die Höhendifferenz auf $M = 2,0$ m erhöht, verschlechtert sich das Verhalten mit den Baseline-Gewichten deutlich. Figure 45 dokumentiert eine wachsende Querabweichung im letzten Drittel der Strecke. In den Zeitreihen (Figure 46) steigt die Schwingungsamplitude des Rollwinkels progressiv an—ein Muster, das auf Unterdämpfung bei stärkerer Störmomentenerregung durch Längsneigung und Krümmungswechsel hinweist.

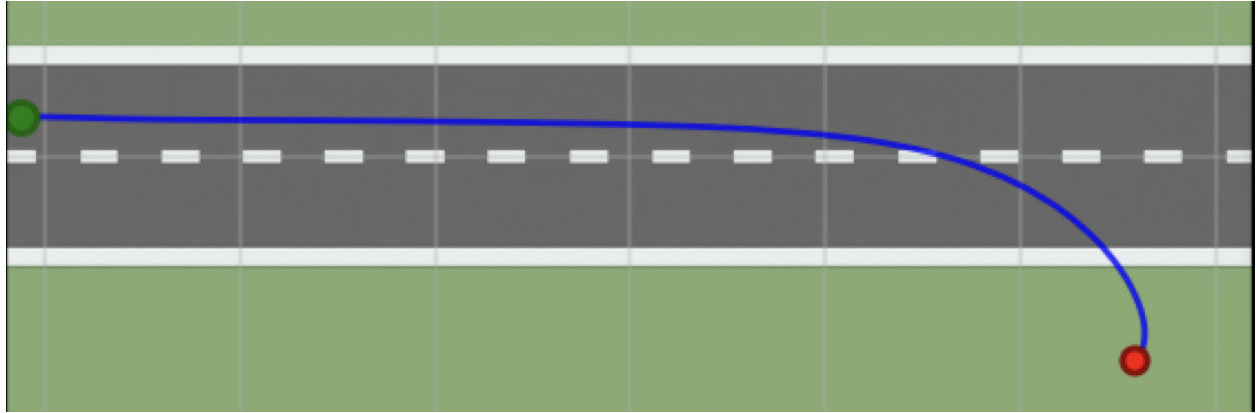


Figure 45: Trajektorie durch die Senke (Fall B, $M = 2,0$ m, Baseline-Gewichte).

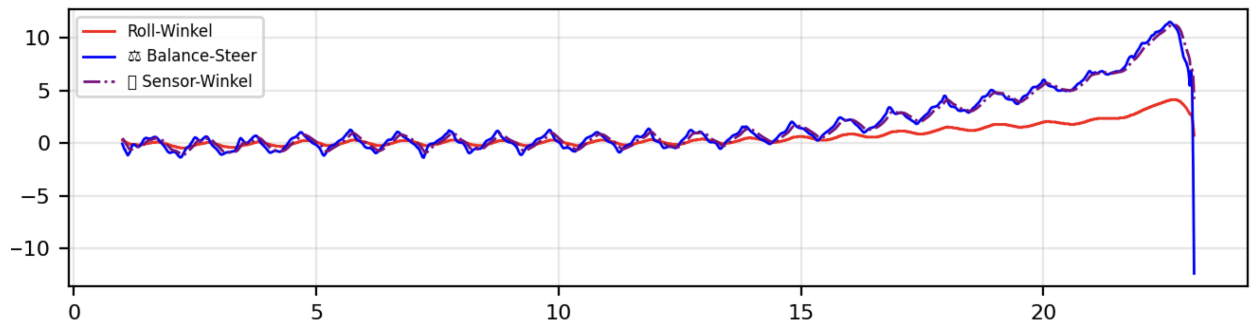


Figure 46: Zeitverläufe Rollwinkel/Balancelenkung (Fall B, $M = 2,0$ m, Baseline-Gewichte).

Fall C: Senke, $M = 2,0$ m mit angepassten Balance-PID-Gewichten Zur Wiederherstellung der Querstabilität wurden die Balance-Gains gezielt erhöht: $K_p = 3,5$, $K_i = 0,2$, $K_d = 0,4$. Die

resultierende Trajektorie (Figure 47) bleibt spurtreu; die Zeitreihen in Figure 48 zeigen gedämpfte Rollschwingungen mit rascher Rückkehr in den stationären Bereich. Die Anpassung adressiert die dominanten Effekte bei großem M : (i) stärkere proportionale Rückführung (K_p) zur Reduktion des Querfehlers und (ii) erhöhte Dämpfung (K_d) zur Phasenführung gegen die durch den Krümmungswechsel angeregten Eigenschwingungen. Der Integrationsanteil bleibt moderat und dient primär der Eliminierung kleiner stationärer Abweichungen (Anti-Windup aktiv).

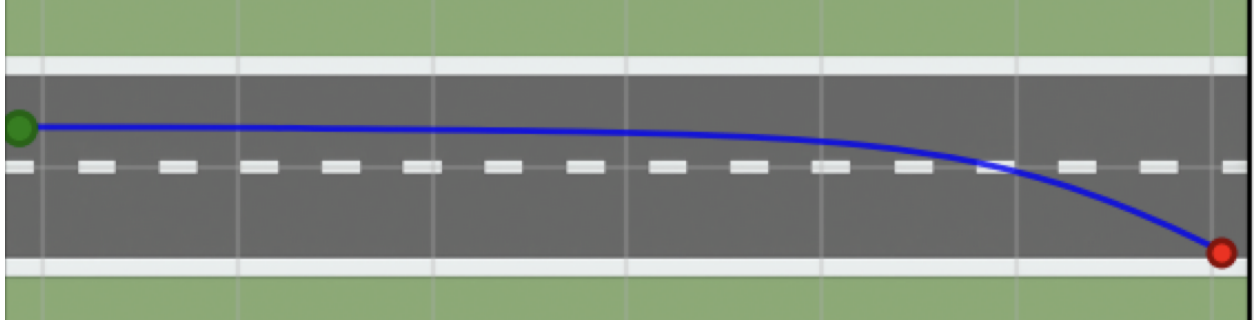


Figure 47: Trajektorie mit retuntem Balance-PID (Fall B, $M = 2,0$ m).

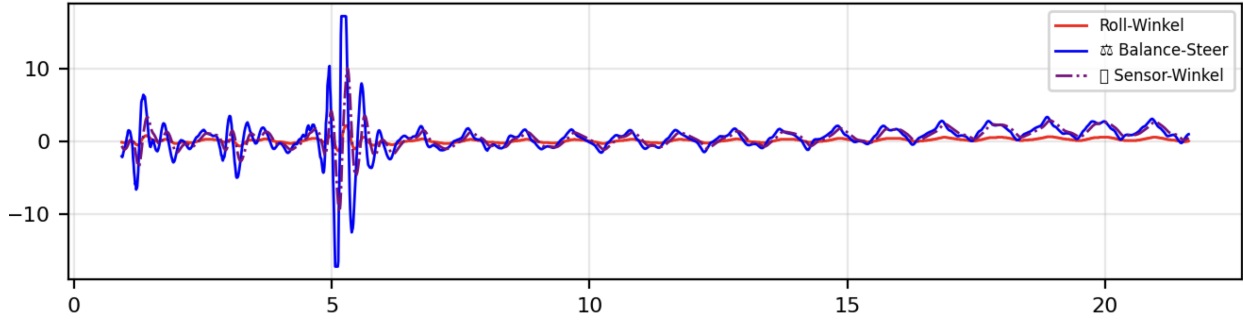


Figure 48: Zeitverläufe mit retuntem Balance-PID (Fall B, $M = 2,0$ m).

Zusammenfassung und Ableitungen Die Ergebnisse zeigen, dass mit zunehmender Höhendifferenz M die erforderlichen Balance-Gains systematisch ansteigen, um Querstabilität und Dämpfung sicherzustellen. In der hier verwendeten Konfiguration (Reifenbreite 2 cm, Streckenlänge 64 m mit Tiefpunkt bei $s = 32$ m) liegt die praktikable Grenze im Fall B bei $M \approx 2$ m; ohne Retuning treten darüber Instabilität bzw. Umkippen auf. Perspektivisch bietet sich eine steigungsadaptive Gain-Schedulierung entlang des Höhenprofils an, um manuelles Retuning zu vermeiden und die Robustheit gegenüber starken Längsneigungen weiter zu erhöhen.

Fall D: Hügel, $M = 2,0\text{ m}$ mit angepassten Balance PID Gewichten

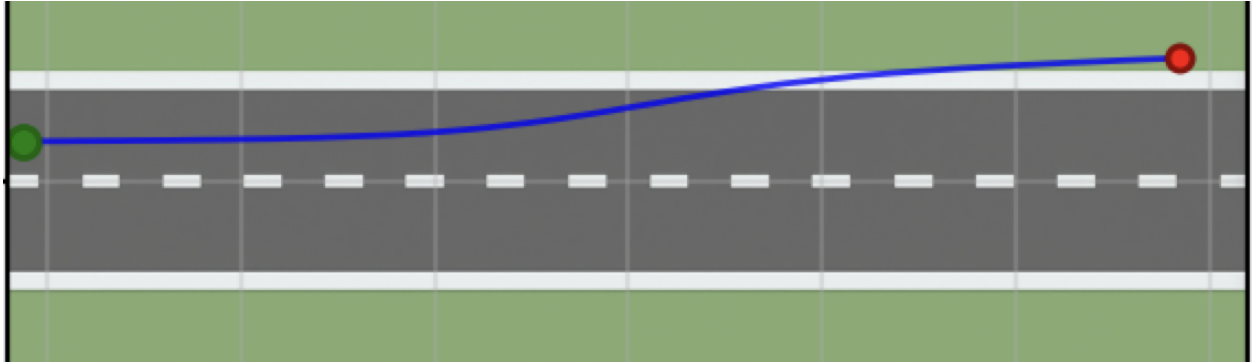


Figure 49: Trajektorie durch den Hügel (Fall D, $M = 2,0$ m).

Wir betrachten nun die Auffahrt auf einen Hügel mit einer maximalen Höhe von $+2$ m. Der Zwischenfall mit $+1$ m zeigt qualitativ das gleiche Verhalten und wird daher ausgelassen. Ziel ist die Beurteilung der Stabilität bei umgekehrtem Höhenprofil im Vergleich zur Senke.

Die Trajektorie in Figure 49 bleibt über weite Strecken spurtreu. Geringe laterale Abweichungen treten vor allem im Bereich des Scheitelpunkts auf. Dort ändert sich die Steigung am stärksten und die Balance Regelung muss zusätzliche Störmomente kompensieren. Die Zeitreihen in Figure 50 stützen diese Beobachtung. Rollwinkel und Balancelenkswinkel überlagern sich weitgehend und zeigen kleine gedämpfte Schwingungen. Kurz vor dem Scheitelpunkt ist eine moderate Amplitudenzunahme sichtbar. Nach der Überfahrt klingen die Schwingungen rasch ab und der Verlauf nähert sich wieder dem stationären Bereich.

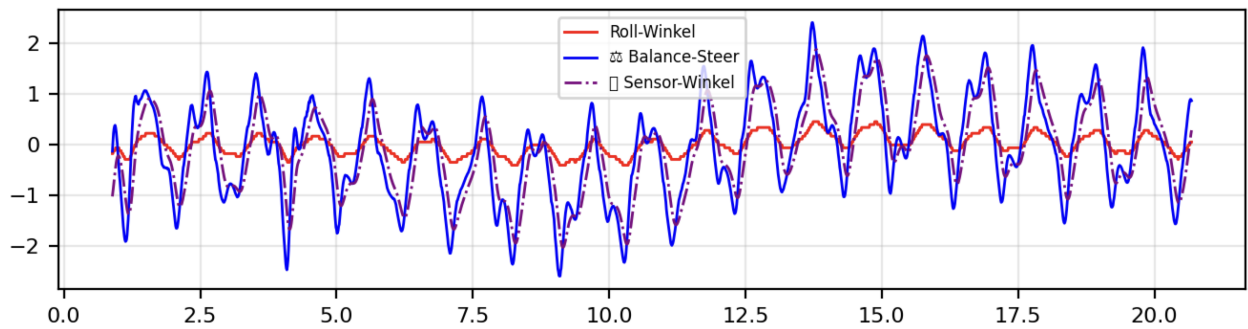


Figure 50: Zeitverläufe mit angepasstem Balance PID (Fall D, $M = 2,0$ m).

Fall E: Hügel, $M = 4,0$ m mit angepassten Balance PID Gewichten Mit den Parametern $K_p = 4,5$, $K_i = 0,2$ und $K_d = 0,4$ lässt sich eine Hügelhöhe von etwa 4 m beherrschen. Als Gütekriterium diente das sichere Vermeiden eines Umkippens. Spurtreue wurde nicht erzwungen.

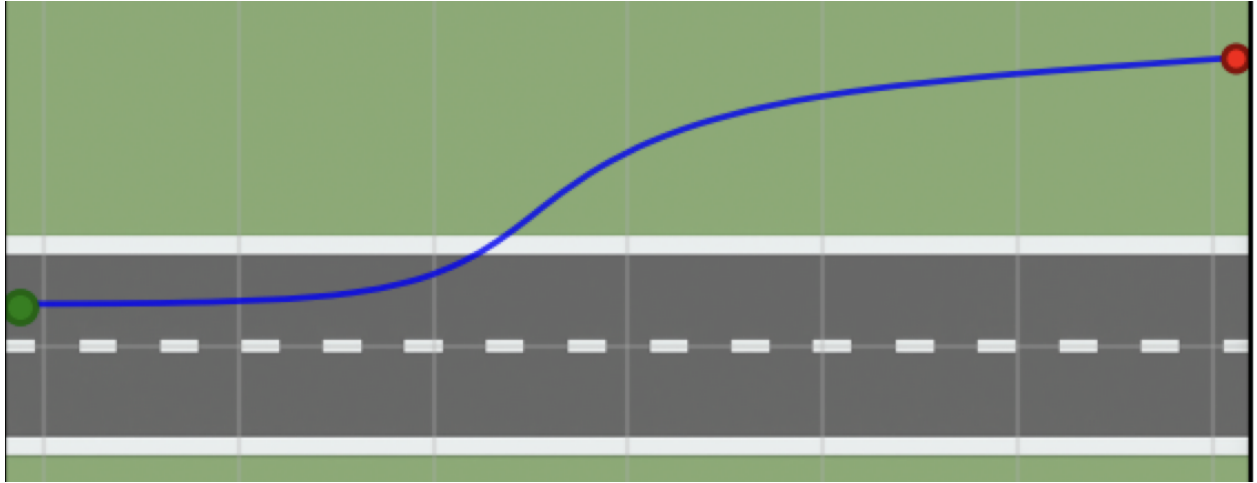


Figure 51: Trajektorie durch den Hügel (Fall E, $M = 4,0$ m).

Die Trajektorie in Figure 51 zeigt eine zentrale Annäherung an den Hügel. Nach rund 20 m setzt eine deutliche Ausweichbewegung ein. Bis zu diesem Punkt wurde bereits ein Höhengewinn von knapp 2 m erreicht. Die Regelung priorisiert in dieser Phase die Aufrechterhaltung der Balance gegenüber der Spurhaltung. Dadurch kann eine laterale Abdrift entstehen und der Scheitel wird mit seitlichem Versatz umfahren. Nach dem Scheitel stabilisiert sich die Fahrt in der anschließenden Abfahrt wieder und der Verlauf nähert sich einer geraden Spur.

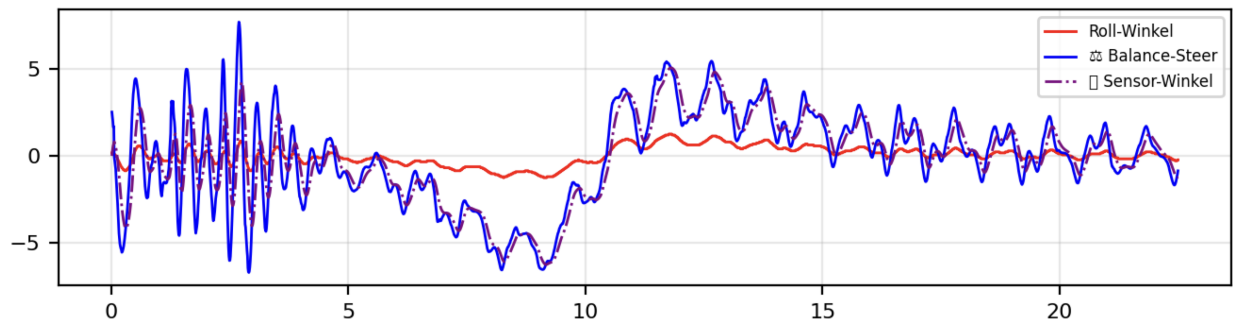


Figure 52: Zeitverläufe mit angepasstem Balance PID (Fall E, $M = 4,0$ m).

Die Zeitreihen in Figure 52 verdeutlichen die Dynamik. Zu Beginn treten stärkere Schwingungen auf, die im Bereich des steilen Anstiegs zunehmen. Kurz nach dem Scheitel gehen die Amplituden spürbar zurück und der Verlauf wird ruhiger. Dieses Muster ist konsistent mit den zusätzlichen Störmomenten durch die größere Längsneigung und den ausgeprägten Krümmungswechsel. Die gewählten Verstärkungen erhöhen die proportionale Rückführung und die Dämpfung. Der Integrationsanteil bleibt moderat und dient vor allem der Beseitigung kleiner stationärer Abweichungen. In Summe wird Balance erreicht, während die Spurhaltung bei $M = 4$ m nur eingeschränkt gewährleistet ist.

Zusammenfassung und Ableitungen Die Experimente mit Senken und Hügeln zeigen übereinstimmend, dass mit wachsender Höhendifferenz M die erforderlichen Balance Gains ansteigen,

damit Querstabilität und Dämpfung erhalten bleiben. Die Teststrecke besitzt eine Länge von 64 m und ein zentrales Extremum bei $s = 32$ m. In dieser Konfiguration beträgt die Reifenbreite 2 cm.

Senken. Bei $M = 1,0$ m bleibt das System mit Baseline Parametern stabil und spurtreu. Beim Übergang aus der Senke treten nur kleine transiente Abweichungen auf, die rasch abklingen. Bei $M = 2,0$ m führt das Baseline Tuning zu wachsender Rollschwingung und lateraler Drift. Ein Retuning mit $K_p = 3,5$, $K_i = 0,2$ und $K_d = 0,4$ stellt die Querstabilität wieder her. Die Talfahrt erweist sich als robust, weil die tiefste Stelle mittig liegt und die seitlichen Anteile der Hangabtriebskraft symmetrisch wirken.

Hügel. Für $M = 2,0$ m zeigt die Auffahrt mit angepassten Gewichten ein stabiles Verhalten. Bei $M = 4,0$ m bleibt das Fahrrad mit $K_p = 4,5$, $K_i = 0,2$ und $K_d = 0,4$ aufrecht, weicht jedoch seitlich aus und umgeht den Scheitelpunkt. Die stärkste Beanspruchung entsteht im Anstieg nahe dem Scheitel, wo sich Steigung und Krümmungswechsel überlagern. Nach dem Scheitel beruhigt sich das System, und die Abfahrt verläuft deutlich ruhiger als der Anstieg.

Gemeinsame Schlussfolgerungen.

- Die erforderlichen Gains wachsen mit M . Der proportionale Anteil reduziert den Querfehler. Der derivative Anteil erhöht die Dämpfung und verbessert die Phasenlage. Der integrale Anteil bleibt moderat und beseitigt kleine stationäre Abweichungen unter aktivem Anti Windup.
- Die kritischen Bereiche liegen an den Extrema des Höhenprofils. Dort ändern sich Neigung und Krümmung am stärksten, was transiente Rollmomente verstärkt und Spurabweichungen auslösen kann.
- In der vorliegenden Konfiguration liegt für Senken die praktikable Grenze ohne Retuning bei etwa $M = 2$ m. Größere Werte erfordern angepasste Gains. Hügel mit $M = 4$ m sind mit erhöhten Gewichten beherrschbar, jedoch mit Einbußen in der Spurhaltung.

Implikationen für die Regelung. Eine steigungsadaptive Gain Scheduling entlang der Strecke ist sinnvoll. Dabei können Vorsteueranteile aus der geschätzten Neigung und ihrer Änderung genutzt werden. Empfehlenswert sind zudem Begrenzungen der Lenkraten und Amplituden sowie eine konsequente Anti Windup Behandlung. Damit lässt sich das Retuning reduzieren, und die Robustheit gegenüber starken Längsneigungen steigt. 

6.3 Rechenzeit & Echtzeitfähigkeit in Webots (Vergleich zum Realfahrrad)

Motivation Dieses Teilkapitel ordnet die in den vorangehenden Abschnitten dargestellten Fahrversuche (Kurvenfahrt, S-Kurve, Senken-/Hügelfahrten) hinsichtlich Rechenzeit und Echtzeitfähigkeit ein. Im Fokus steht die Frage, welche zeitlichen Anforderungen die Simulation in Webots an den Code stellt und wie sich diese Anforderungen vom realen Prototyp unterscheiden.

Rechenzeit in Webots: virtuelle Zeit statt harter Echtzeit Webots trennt streng zwischen *virtueller Zeit* (Simulationszeit) und *Realzeit*. Die Schrittweite der Physik wird durch den `WorldInfo.basicTimeStep` vorgegeben (hier: $T_s = 2$ ms, entsprechend $f_s \approx 500$ Hz). Jeder Aufruf von `wb_robot_step(duration)` lässt genau die angegebene *virtuelle Zeit* vergehen; die tatsächlich benötigte *Rechenzeit* kann jedoch größer oder kleiner als `duration` sein.⁵ Damit gilt:

⁵Vgl. Webots Reference Manual: *Robot* – Synchronisation und `wb_robot_step`.

- Ist die Berechnung schneller als echtzeitfähig, läuft die Simulation schneller als Realzeit.
- Ist die Berechnung langsamer, *verlangsamt* Webots die Ausführung (Speedometer < 1); die Physikschritte bleiben konsistent, die Simulationsuhr schreitet lediglich langsamer fort.⁶

Für unsere Konfiguration bedeutet das: Der schnelle Balance-Controller (C, 500 Hz) und die langsamere Visionschleife (Python, ≈ 20 Hz) müssen in Webots *nicht* harte Deadlines erfüllen, solange die Synchronisation aktiv ist (`synchronization = TRUE`). Die Simulationsdynamik bleibt deterministisch; nur die Ausführungsdauer in Realzeit steigt bei hoher Rechenlast.

Implikationen für die hier durchgeführten Versuche Die in Kap. 5 dargestellten Szenarien (u. a. Kurve/S-Kurve mit YOLO sowie ROI-Fallback, Senken/Hügel mit ansteigenden Balance-Gains) profitieren von dieser Entkopplung:

1. **Parametervariationen ohne Echtzeitdruck:** Gain-Sweeps, Hysterese-Tests und Filterstudien (EMA/Spline-Glättung) können mit identischer virtueller Zeitskala reproduziert werden, auch wenn einzelne Schritte in Realzeit länger dauern.
2. **Stabilitätsaussagen bleiben gültig:** Das Kippverhalten, Querfehler-Verlauf und die Dämpfungseigenschaften ergeben sich aus der Physik im festen T_s ; eine temporär langsamere Ausführung verzerrt diese Aussagen nicht.
3. **Architekturpriorität:** Für die Simulation ist eine *klare, modulare Code-Architektur* (Trennung Balance vs. Vision, wohldefinierte Schnittstellen, Logging) wichtiger als die Optimierung auf harte Echtzeitgrenzen.

Warum harte Echtzeit erst auf dem Realfahrrad kritisch wird Die früheren Arbeiten am realen Fahrrad (u. a. Ranz 2021, Zander 2023, Yasin 2024) beschreiben eine kaskadierte PID-Architektur mit typischen Zykluszeiten von 5 ms (Balance), 1 ms (Lenkpositionsregelung) und 150 ms (Geschwindigkeit). Die Implementierung nutzt auf dem BeagleBone Black u. a. PRUs für deterministische PWM/GPIO-Signale und einen Watchdog für Deadline-Überwachung. Hier gilt:

- **Harte Deadlines:** Wird die 5 ms-Balanceperiode verfehlt, akkumuliert sich der Rollfehler; das System kann innerhalb weniger hundert Millisekunden instabil werden.
- **E/A-Latenzen sind real:** IMU, Encoder und Motoransteuerung verursachen nicht verschiebbare Verzögerungen und Jitter, die in der Regelung kompensiert werden müssen (Anti-Windup, Derivat-Anteil, Begrenzungen).
- **Kein „Zeitdilatations“-Sicherheitsnetz:** Im Gegensatz zu Webots lässt sich die Realzeit nicht „verlangsamen“; das Betriebssystem und die Elektronik laufen unverändert weiter.

Folglich ist auf dem realen System eine *echtzeitfähige* Umsetzung (deterministische Scheduling-Strategie, Interrupt-nahe E/A, Worst-Case-Betrachtungen) zwingend; in der Simulation dagegen nicht.

⁶Vgl. Webots User Guide: *The User Interface* – „Speedometer and Virtual Time“.

Konsequenzen für diese Arbeit und Folgeprojekte Aus den obigen Punkten leiten sich zwei zentrale Empfehlungen ab:

1. **Für Webots-Studien (diese Arbeit):** Beibehalten eines kleinen $T_s = 2\text{ ms}$ für die Physik, saubere Synchronisation (`wb_robot_step`), konsistente Logging-Zeitskalen. Rechenzeitoptimierung ist optional und zielt v. a. auf Durchsatz (mehr Szenarien/Parameter pro Zeiteinheit).
2. **Für den Transfer aufs reale Fahrrad:** Frühzeitige Vorbereitung einer echtzeitfähigen Architektur (PRU/ISR-basierte Ansteuerung, Pufferung von Sensordaten, feste Task-Zyklen, Watchdog, Timing-Telemetry). Die in Webots gefundenen Parameter/Strukturen bleiben nutzbar, müssen jedoch unter harten Zeitrestriktionen validiert werden.

Kurzfazit In Webots bestimmt die virtuelle Zeit (T_s) die physikalische Auflösung; Überschreitungen der Rechenzeit führen lediglich zu einer langsameren Ausführung in Realzeit, nicht zu dynamischen Artefakten. Daher ist für die Simulation keine strikte Echtzeitfähigkeit des Codes nötig. Auf dem realen Fahrrad hingegen sind harte Deadlines konstitutiv für die Stabilität; hier entscheidet Echtzeitfähigkeit unmittelbar über das Regelungsvermögen und die Sicherheit des Systems.

7 Grenzen der Arbeit

8 Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)

9 Fazit

10 Literaturverzeichnis

References

- [1] D. Andreo, V. Cerone, D. Dzung, and D. Regruto. Experimental results on l_{pv} stabilization of a riderless bicycle. In *Proc. American Control Conference (ACC)*, pages 3124–3129, 2009.
- [2] Bosch Sensortec. Bno055 intelligent 9-axis absolute orientation sensor. <https://www.bosch-sensortec.com>, 2020.
- [3] Concurrent Real-Time. Inverted pendulum, 2024.
- [4] Cyberbotics. Webots – open source robot simulator. <https://cyberbotics.com>, 2025.
- [5] Cyberbotics Ltd. Controller programming. <https://cyberbotics.com/doc/guide/controller-programming>, 2025. Zugriff am 27.08.2025.
- [6] Cyberbotics Ltd. Emitter. <https://cyberbotics.com/doc/reference/emitter>, 2025. Zugriff am 27.08.2025.
- [7] Cyberbotics Ltd. Receiver. <https://cyberbotics.com/doc/reference/receiver>, 2025. Zugriff am 27.08.2025.
- [8] Devantech Ltd. Md49 dual motor driver datasheet. <https://www.robot-electronics.co.uk>, 2019.
- [9] do-mpc development team. Basics of model predictive control, 2024. Accessed 23 August 2025.
- [10] Yassin Ghidaei. Video- und sensorgestütztes autonomes fahren für ein selbstbalancierendes fahrrad. Master’s thesis, Goethe-Universität Frankfurt am Main, 2024.
- [11] Yassin Ghidaei and Amine Karabila. Yolo-based vision control for self-balancing bicycle. Internal project documentation, 2024. Vision controller implementation using YOLO for path detection and MPC for trajectory planning.
- [12] Yasin Giray, Amine Karabila, Jonah Zander, et al. Vision controller readme for self-balancing bicycle. https://github.com/arabila/selfbalancingbicycle/tree/master/Regelsteuerung/controllers/vision_control_py/README.md, 2023. Describes the MPC-based vision controller and its integration with the balance controller.
- [13] GitHub contributors. Webots simulation framework. <https://github.com/cyberbotics/webots>, 2025. Zugriff am 24.08.2025.
- [14] R. Gupta and S. Lee. Evaluating webots as digital-twin platform for multi-robot systems. In *Proc. IEEE ICRA*, 2023.
- [15] M. et al. Hadi. Soft-gripper simulation in webots. *Soft Robotics*, 11(2):123–135, 2024.
- [16] John Hedengren et al. Proportional integral derivative (pid). <https://apmonitor.com/pdc/index.php/Main/ProportionalIntegralDerivative>, 2024.
- [17] Alex Hunziker. Autonomous steering of an electric bicycle based on sensor fusion using model predictive control. Master’s thesis, Goethe-Universität Frankfurt am Main, 2019.
- [18] Infineon Technologies. Bts7960 high-current half-bridge driver. <https://www.infineon.com>, 2018.

- [19] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Yolo by ultralytics. <https://github.com/ultralytics/ultralytics>, 2023.
- [20] K. Kanjanawanishkul. Lqr and mpc controller design and comparison for a stationary self-balancing bicycle robot with a reaction wheel. *Kybernetika*, 54(1):173–191, 2015.
- [21] Amine Karabila. Auf dem weg zum selbstfahrenden fahrrad: Analyse und implementierung einer regelung der balance. Master’s thesis, Goethe-Universität Frankfurt, 2022.
- [22] J. D. G. Kooijman, J. P. Meijaard, Jim M. Papadopoulos, Andy Ruina, and A. L. Schwab. A bicycle can be self-stable without gyroscopic or caster effects. *Science*, 332(6027):339–342, 2011.
- [23] Wenjia Li, Paris Francis, Robert McClosky, and Alexander Brown. WeBikes: a configurable, open-source add-on for simulating single-track vehicle dynamics in the Webots robot simulation software. *Advances in Transportation Studies*, Special Issue Vol. 4:73–88, 2024.
- [24] Cyberbotics Ltd. Inertialunit. <https://cyberbotics.com/doc/reference/inertialunit>, 2025. Geräte-Referenz, Zugriff: 24.08.2025.
- [25] J. P. Meijaard, Jim M. Papadopoulos, Andy Ruina, and A. L. Schwab. Linearized dynamics equations for the balance and steer of a bicycle: a benchmark and review. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 463(2084):1955–1982, 2007.
- [26] Olivier Michel. Cyberbotics Ltd. Webots: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):39–42, 2004.
- [27] OpenMMLab and Ultralytics. Dive into yolov8: How does this state-of-the-art model work?, 2023. Accessed 23 August 2025.
- [28] Sang-Hyun Park and Seung-Yun Yi. Active Balancing Control for Unmanned Bicycle Using Scissored-pair Control Moment Gyroscope. *International Journal of Control, Automation and Systems*, 18(1):217–224, 2020.
- [29] Jing Pei, Lei Deng, Sen Song, Mingguo Zhao, Shuang Zhang, et al. Towards artificial general intelligence with hybrid tianjic chip architecture. *Nature*, 572(7767):106–111, 2019.
- [30] Niklas Persson, Martin C. Ekström, Mikael Ekström, and Alessandro V. Papadopoulos. Trajectory tracking and stabilisation of a riderless bicycle. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 1859–1866, Indianapolis, IN, USA, 2021. IEEE.
- [31] Anna Ranz. Entwicklung eines selbstbalancierenden fahrradprototyps. Master’s thesis, HWR Berlin, 2021.
- [32] Wikipedia contributors. Model predictive control, 2024. Accessed 23 August 2025.
- [33] Wikipedia contributors. Image segmentation, 2025. Accessed 23 August 2025.
- [34] Wikipedia contributors. You only look once, 2025. Accessed 23 August 2025.
- [35] Jonah Zander. Entwurf einer Steuerung für ein selbstbalancierendes Fahrrad. Bachelorarbeit, Goethe Universität Frankfurt, 2023.

- [36] Xu Zhao, Wenchao Ding, Yongqi An, Yinglong Du, Tao Yu, Min Li, Ming Tang, and Jinqiao Wang. Fast segment anything. arXiv:2306.12156 [cs.CV], 2023.